Important Information

Thank you for choosing Freenove products!

Getting Started

If you have not yet downloaded the zip file, associated with this kit, please do so now and unzip it.

Get Support and Offer Input

Freenove provides free and responsive product and technical support, including but not limited to:

- Product quality issues
- Product use and build issues
- Questions regarding the technology employed in our products for learning and education
- Your input and opinions are always welcome
- We also encourage your ideas and suggestions for new products and product improvements

For any of the above, you may send us an email to:

support@freenove.com

Safety and Precautions

Please follow the following safety precautions when using or storing this product:

- Keep this product out of the reach of children under 6 years old.
- This product should be **used only when there is adult supervision present** as young children lack necessary judgment regarding safety and the consequences of product misuse.
- This product contains small parts and parts, which are sharp. This product contains electrically conductive parts. **Use caution with electrically conductive parts near or around power supplies, batteries and powered (live) circuits.**
- When the product is turned ON, activated or tested, some parts will move or rotate. **To avoid injuries to hands and fingers keep them away from any moving parts!**
- It is possible that an improperly connected or shorted circuit may cause overheating. **Should this happen, immediately disconnect the power supply or remove the batteries and do not touch anything until it cools down!** When everything is safe and cool, review the product tutorial to identify the cause.
- Only operate the product in accordance with the instructions and guidelines of this tutorial, otherwise parts may be damaged or you could be injured.
- Store the product in a cool dry place and avoid exposing the product to direct sunlight.
- After use, always turn the power OFF and remove or unplug the batteries before storing.

About Freenove

Freenove provides open source electronic products and services worldwide.

Freenove is committed to assist customers in their education of robotics, programming and electronic circuits so that they may transform their creative ideas into prototypes and new and innovative products. To this end, our services include but are not limited to:

- Educational and Entertaining Project Kits for Robots, Smart Cars and Drones
- Educational Kits to Learn Robotic Software Systems for Arduino, Raspberry Pi and micro: bit
- Electronic Component Assortments, Electronic Modules and Specialized Tools
- **Product Development and Customization Services**

You can find more about Freenove and get our latest news and updates through our website:

http://www.freenove.com

sale@freenove.com

Copyright

Contents

List

Freenove ESP32 Mini TV is currently available in four color options: **yellow, black, white, and blue**.
It is important to note that the differences between these versions are strictly limited to the external case color. All internal components—including the hardware architecture, circuit design, and software interfaces—remain identical across all models. Therefore, regardless of which color version you use, this tutorial is fully applicable for all learning and development activities.

If you have any concerns, please feel free to contact us via support@freenove.com

Freenove ESP32 Mini TV x 1	USB data cable x 1
	

Preface

ESP32-WROOM

The ESP32-WROOM offers two antenna options: the PCB on-board antenna and the IPEX antenna.

- The PCB on-board antenna is an integrated antenna within the chip module itself, making it compact and convenient for both portability and design.
- The IPEX antenna is an external metal antenna connected to the module's integrated antenna, providing enhanced signal performance.

PCB on-board antenna	IPEX antenna
	

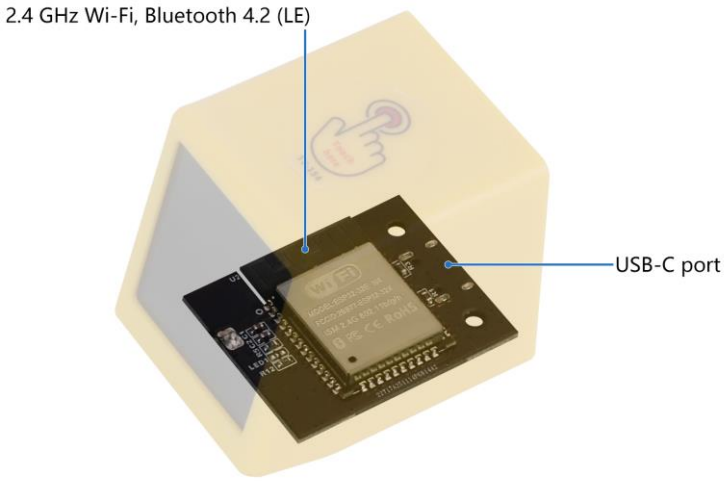
The ESP32-WROOM of this product is based on the ESP32-WROOM-32E module with built-in PCB on-board antenna.

Freenove ESP32 Mini TV

ESP32 Module

A powerful, generic and small-sized controller with onboard wireless.

Dual-core 32-bit microprocessor	520 KB SRAM
Up to 240 MHz clock frequency	448 KB ROM
Compatible with Arduino IDE	4 MB flash



Color Screen

A delicate screen integrated on the mini TV.

1.54 inch	TFT screen
240x240 pixel	ST7789 driver
IPS wide viewing angle	4-wire SPI communication

CH340 (Required)

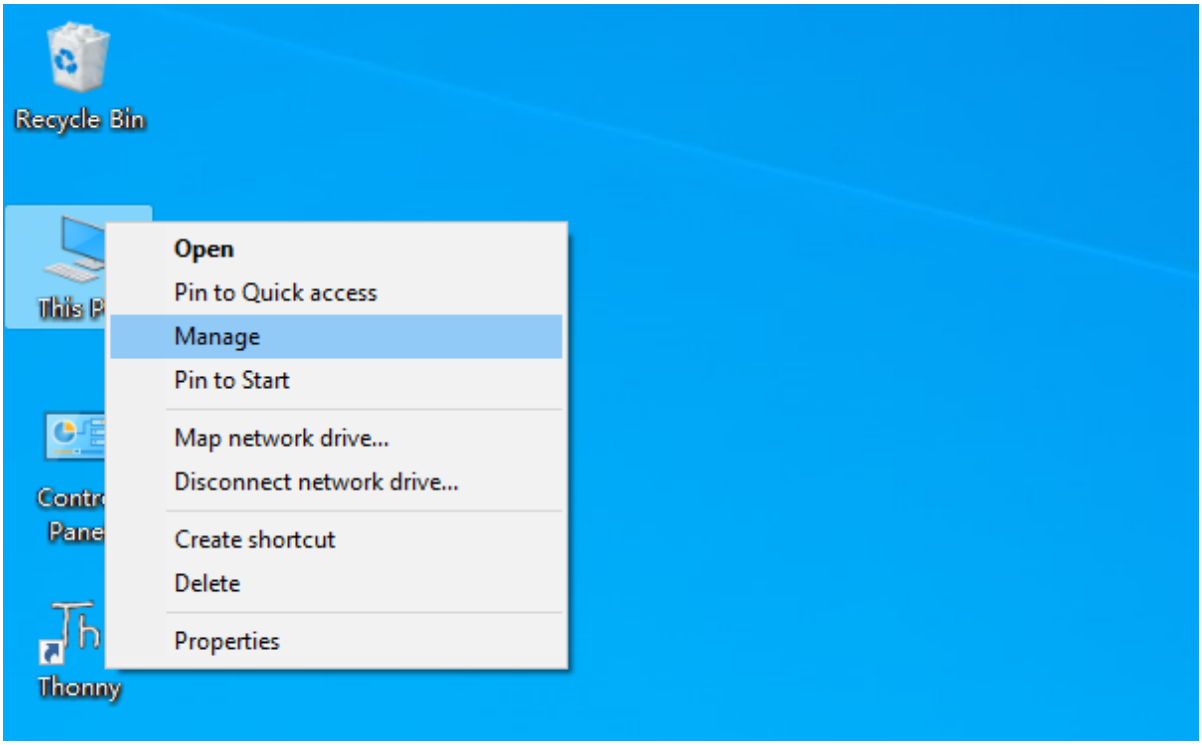
ESP32 uses CH340 to download codes. So before using it, we need to install CH340 driver in our computers.

Windows

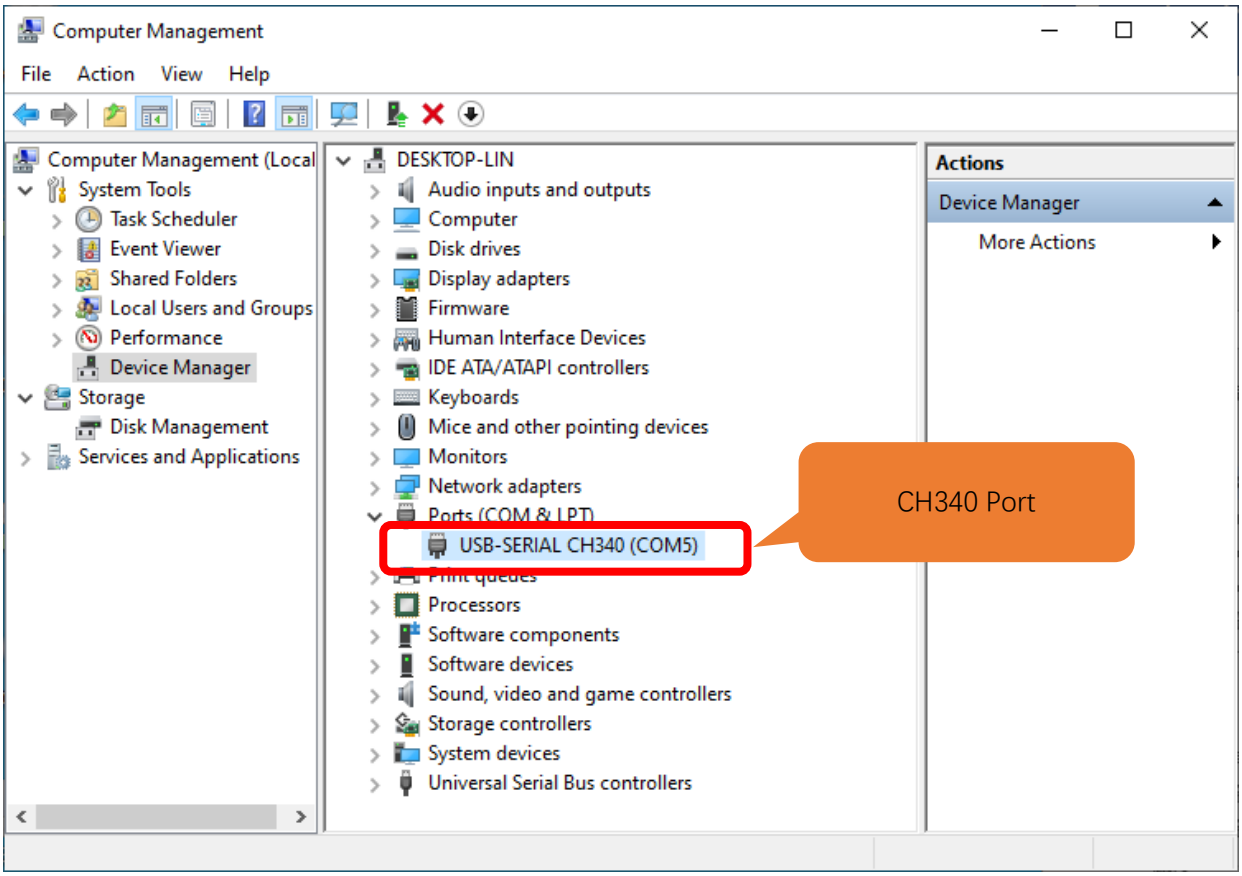
Check whether CH340 has been installed

1. Connect your computer and ESP32-WROOM with a USB cable.

2. Turn to the main interface of your computer, select "**This PC**" and right-click to select "**Manage**".

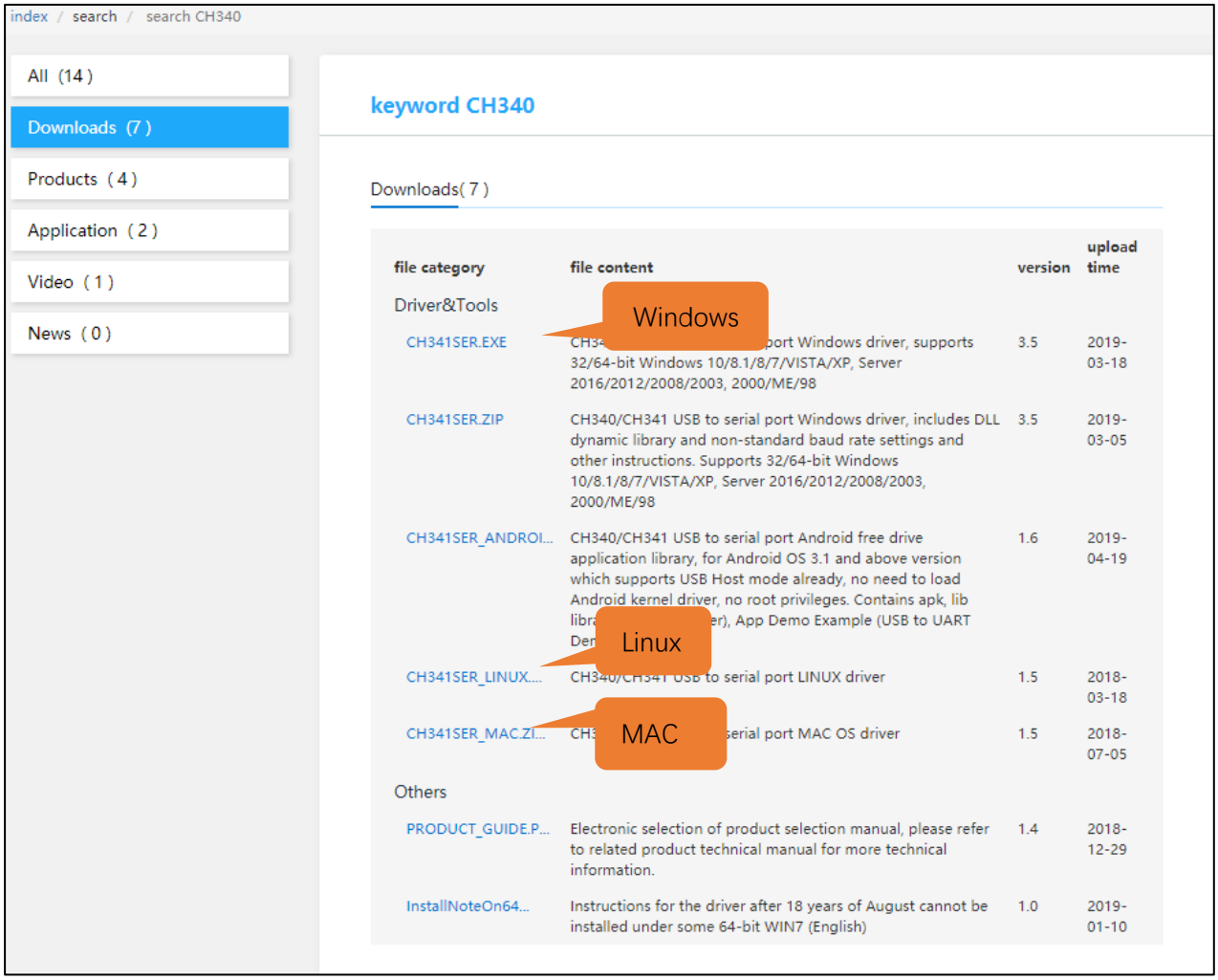


3. Click "Device Manager". If your computer has installed CH340, you can see"USB-SERIAL CH340 (COMx)". And you can click here to move to the next step.

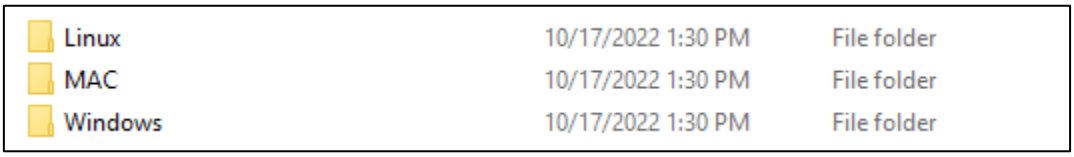


Installing CH340

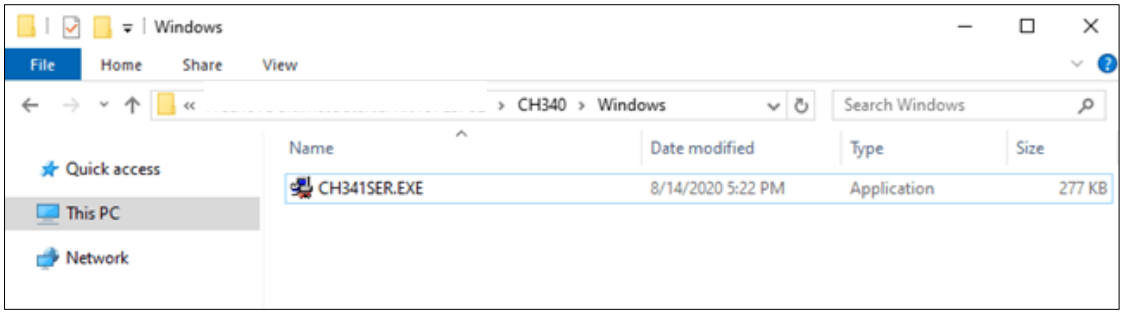
1. First, download CH340 driver, click http://www.wch-ic.com/search?q=CH340&t=downloads to download the appropriate one based on your operating system.



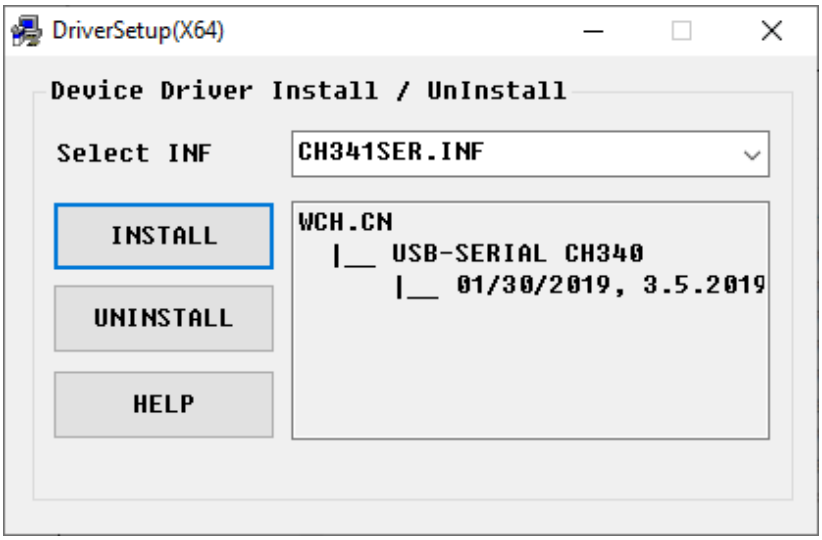
If you would not like to download the installation package, you can open **"Freenove_ESP32_Mini_TV /CH340"**, we have prepared the installation package.



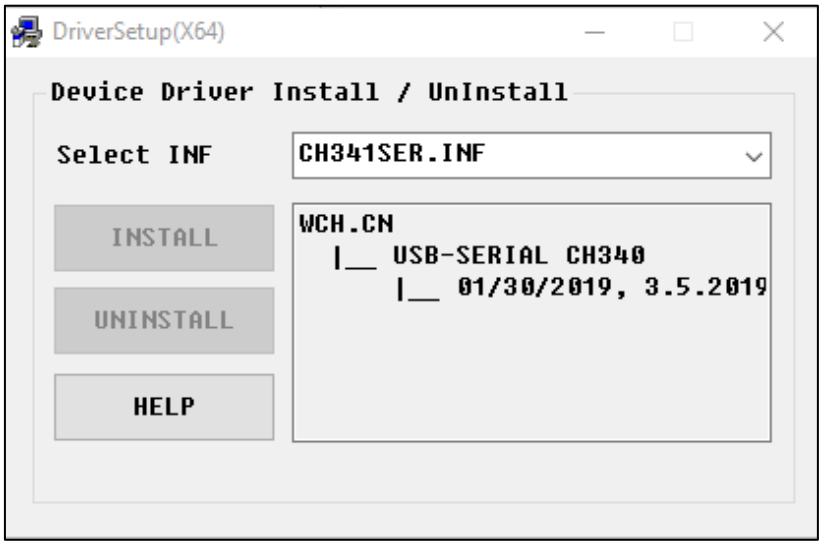
2. Open the folder "**Freenove_ESP32_Mini_TV/CH340/Windows/**"



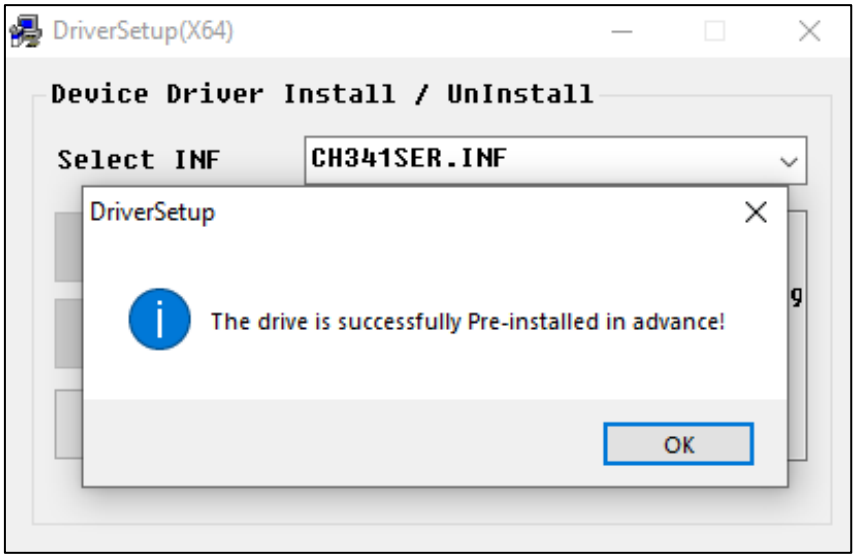
3. Double click "**CH341SER.EXE**".



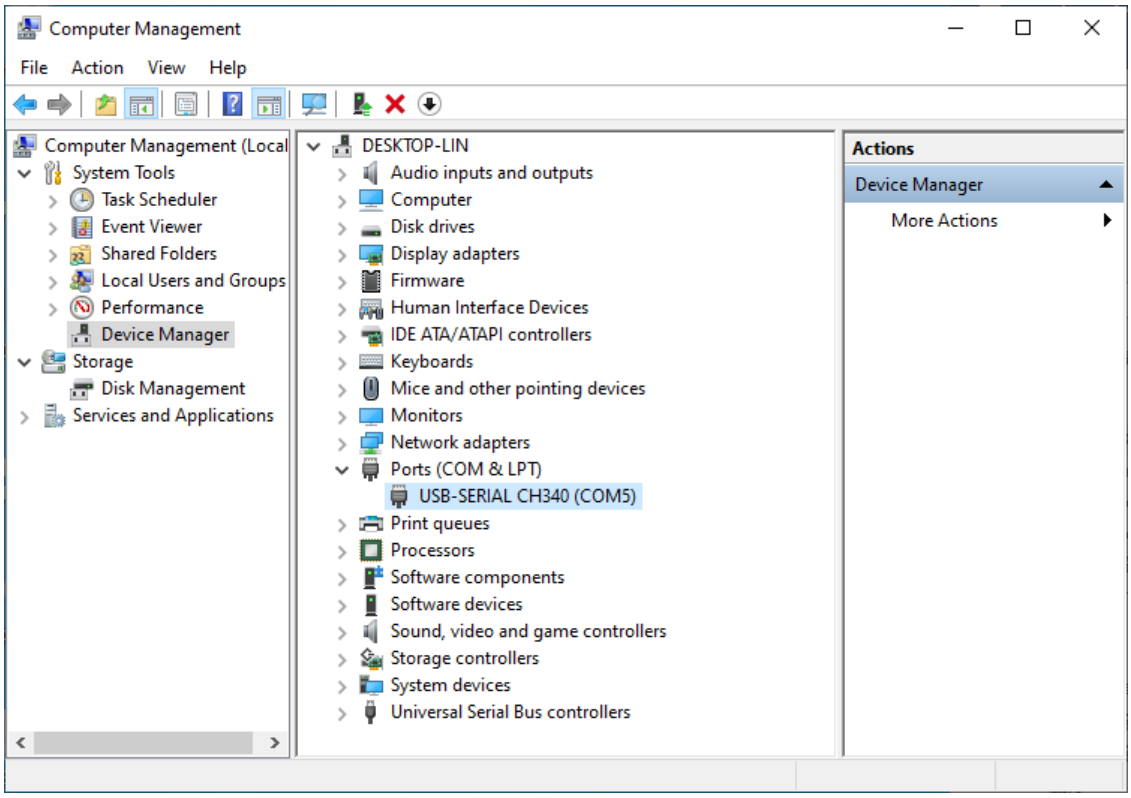
4. Click "INSTALL" and wait for the installation to complete.



5. Install successfully. Close all interfaces.



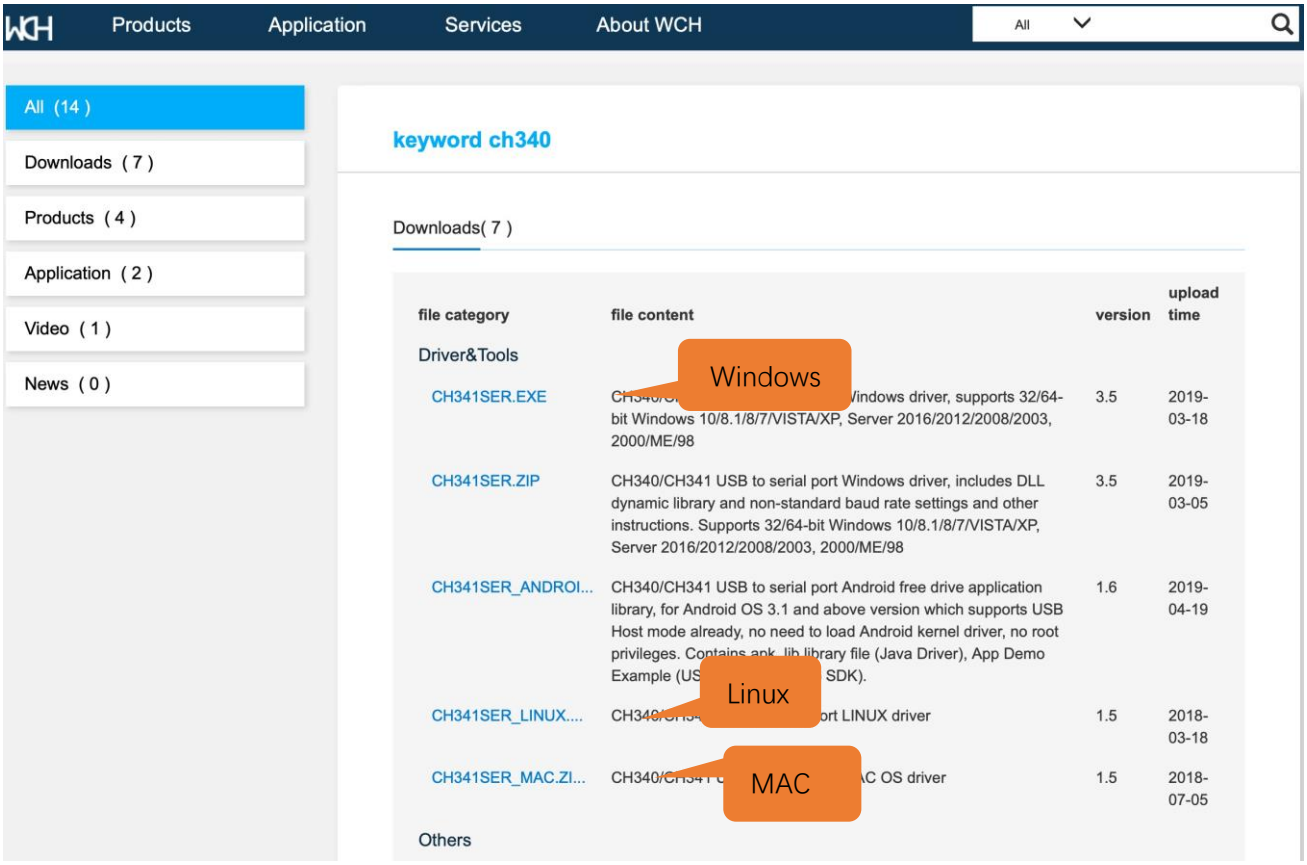
6. When ESP32 is connected to computer, select “This PC”, right-click to select “Manage” and click “Device Manager” in the newly pop-up dialog box, and you can see the following interface.



7. So far, CH340 has been installed successfully. Close all dialog boxes.

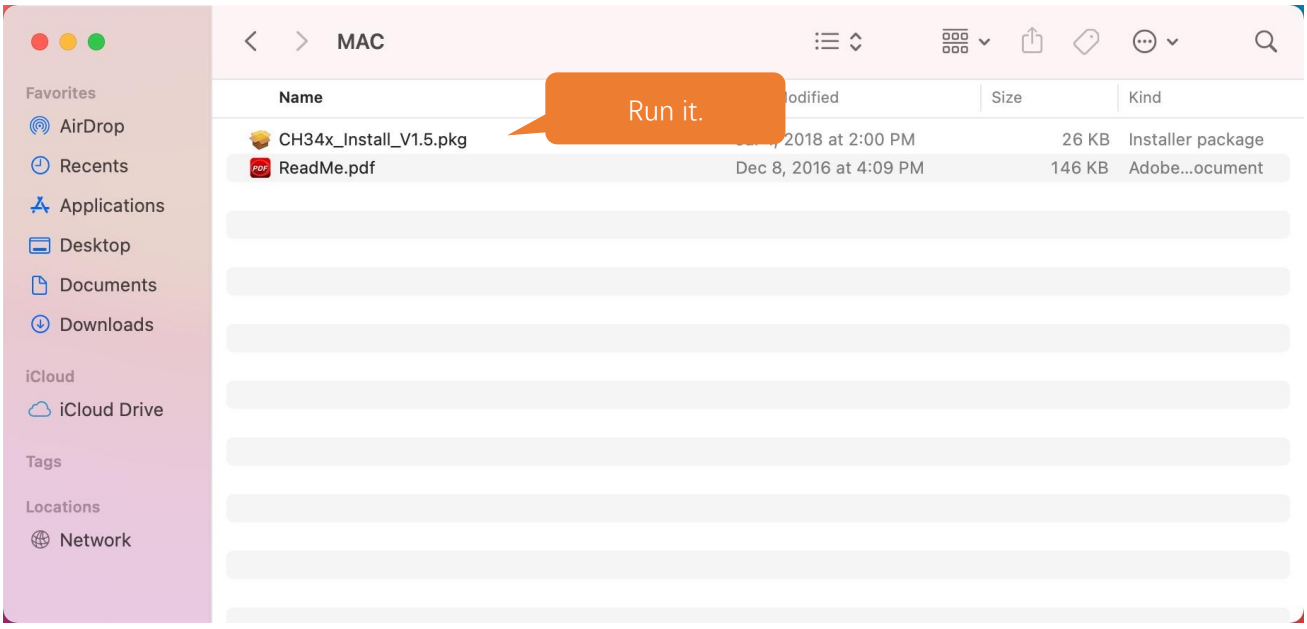
MAC

First, download CH340 driver, click http://www.wch-ic.com/search?q=CH340&t=downloads to download the appropriate one based on your operating system.

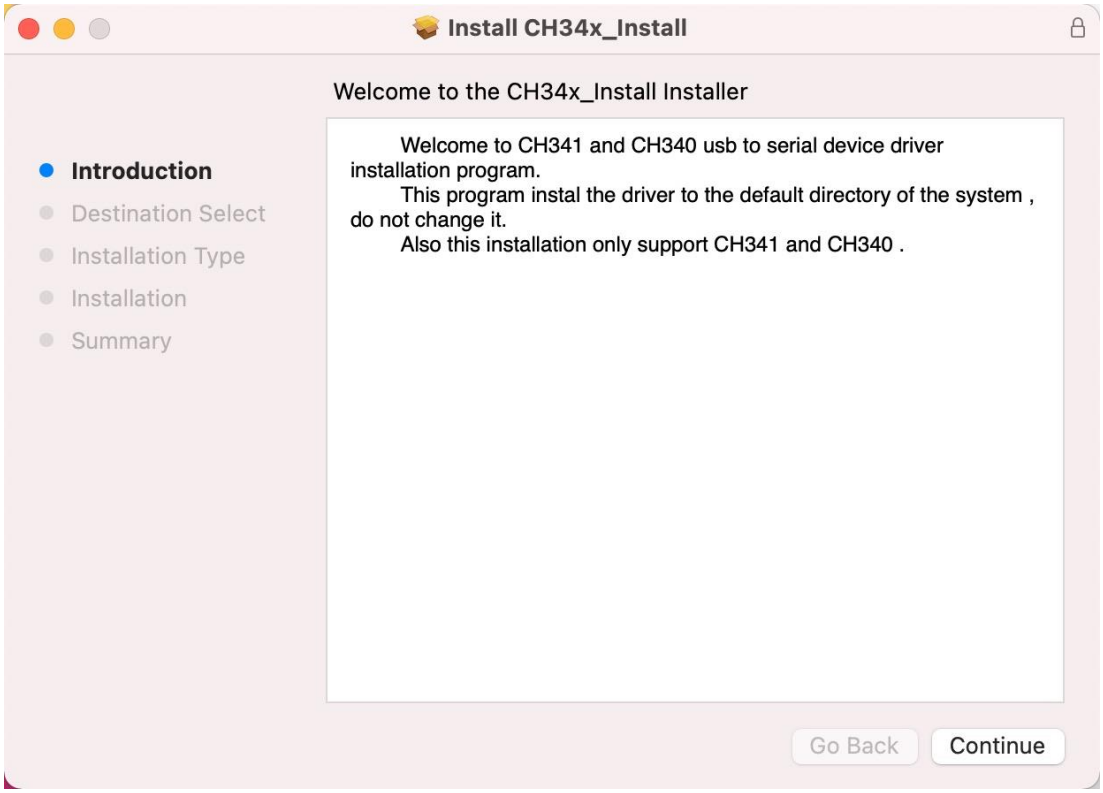


If you would not like to download the installation package, you can open **"Freenove_ESP32_Mini_TV/CH340"**, we have prepared the installation package.

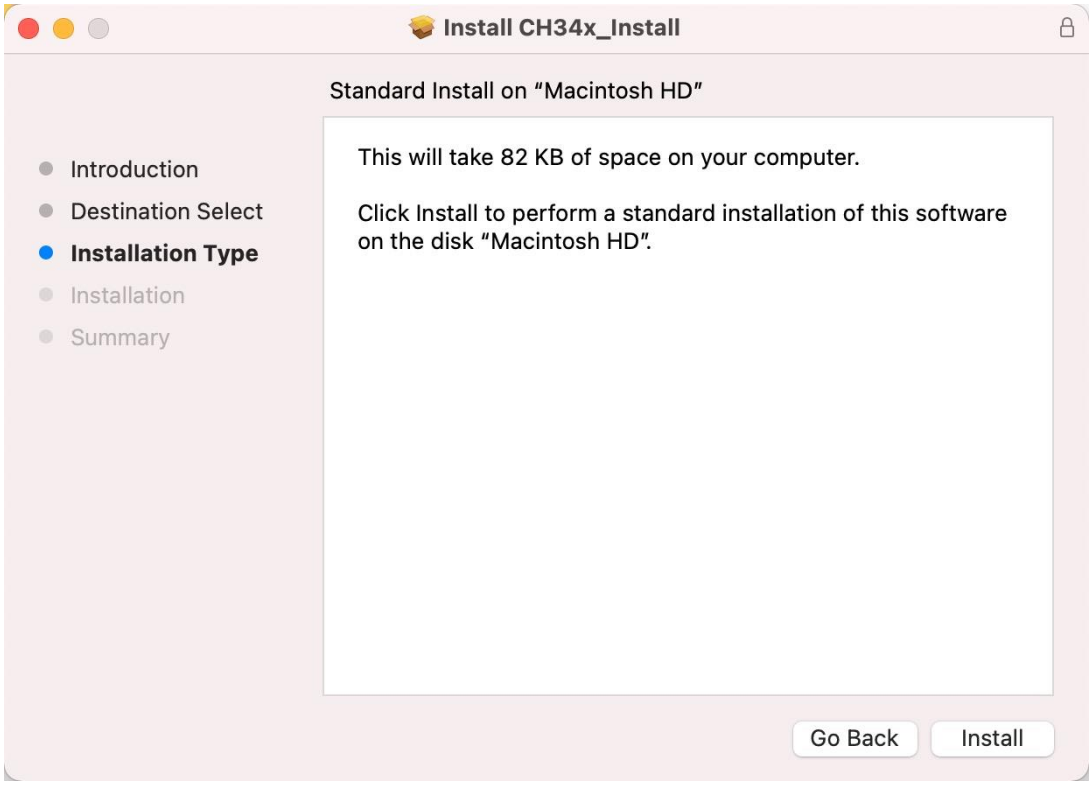
Second, open the folder "**Freenove_ESP32_Mini_TV /CH340/MAC/**"



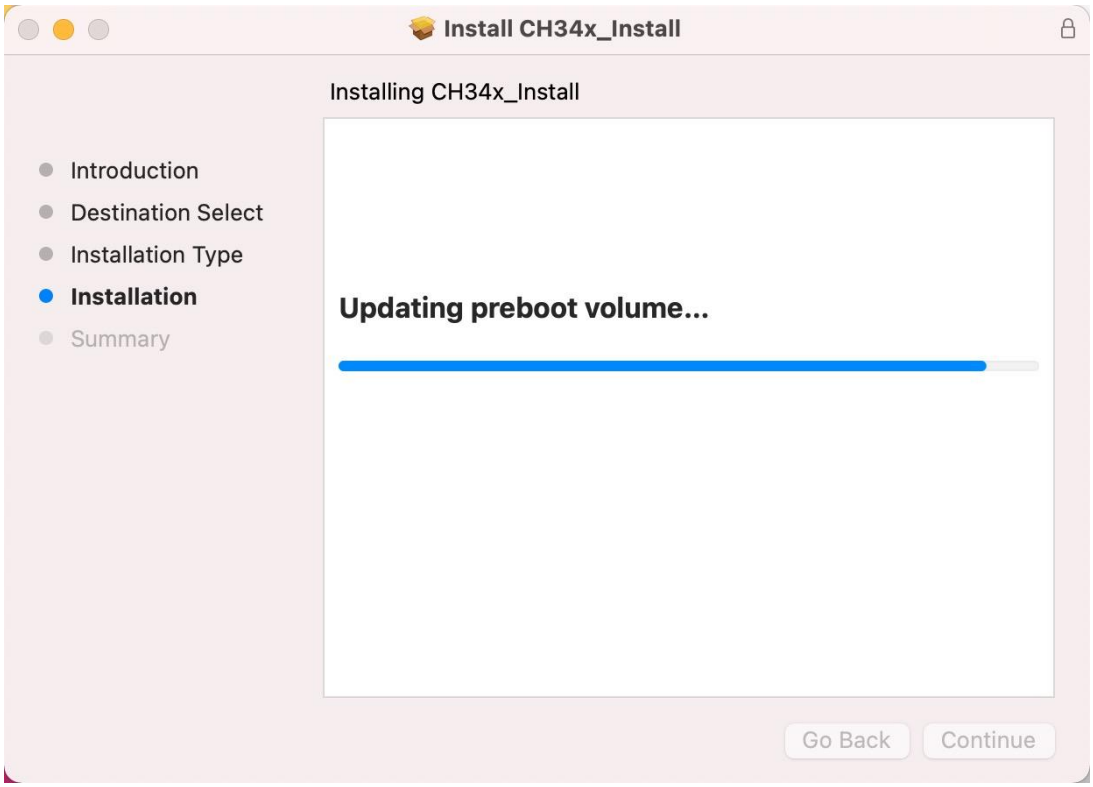
Third, click Continue.



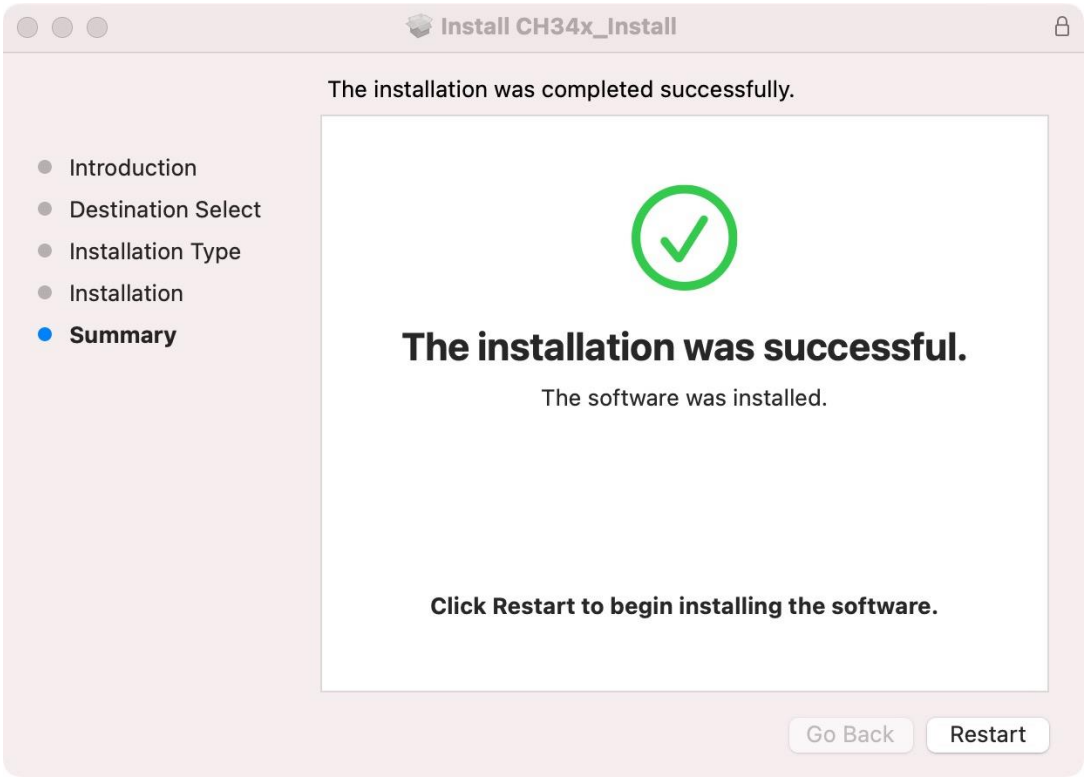
Fourth, click Install.



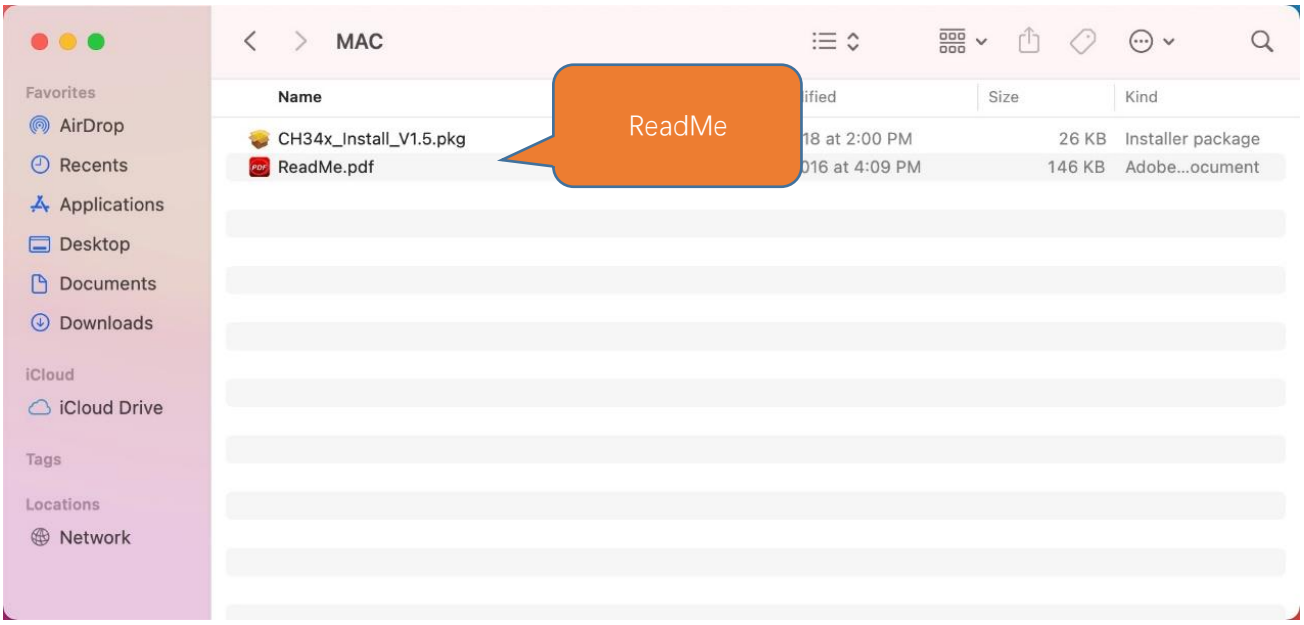
Then, waiting Finsh.



Finally, restart your PC.



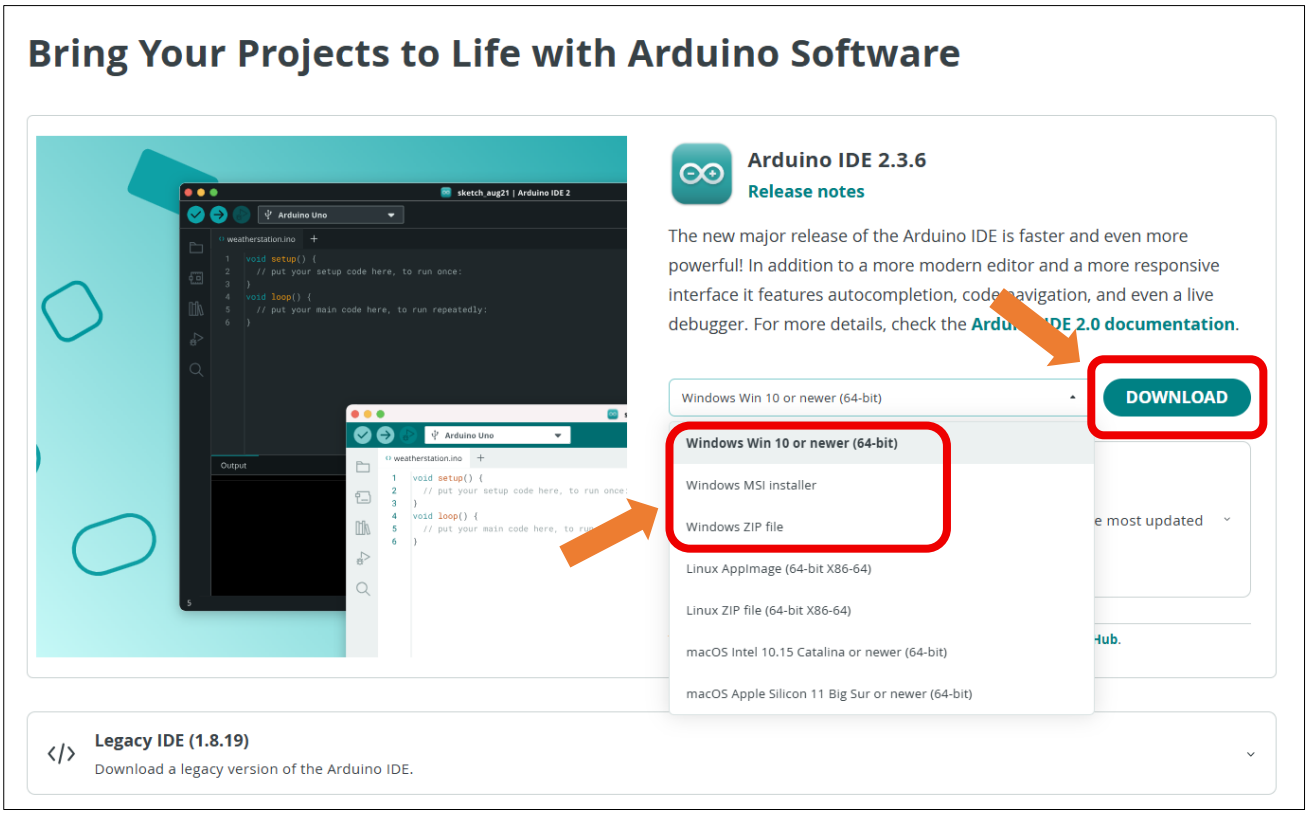
If you still haven't installed the CH340 by following the steps above, you can view readme.pdf to install it.



Programming Software

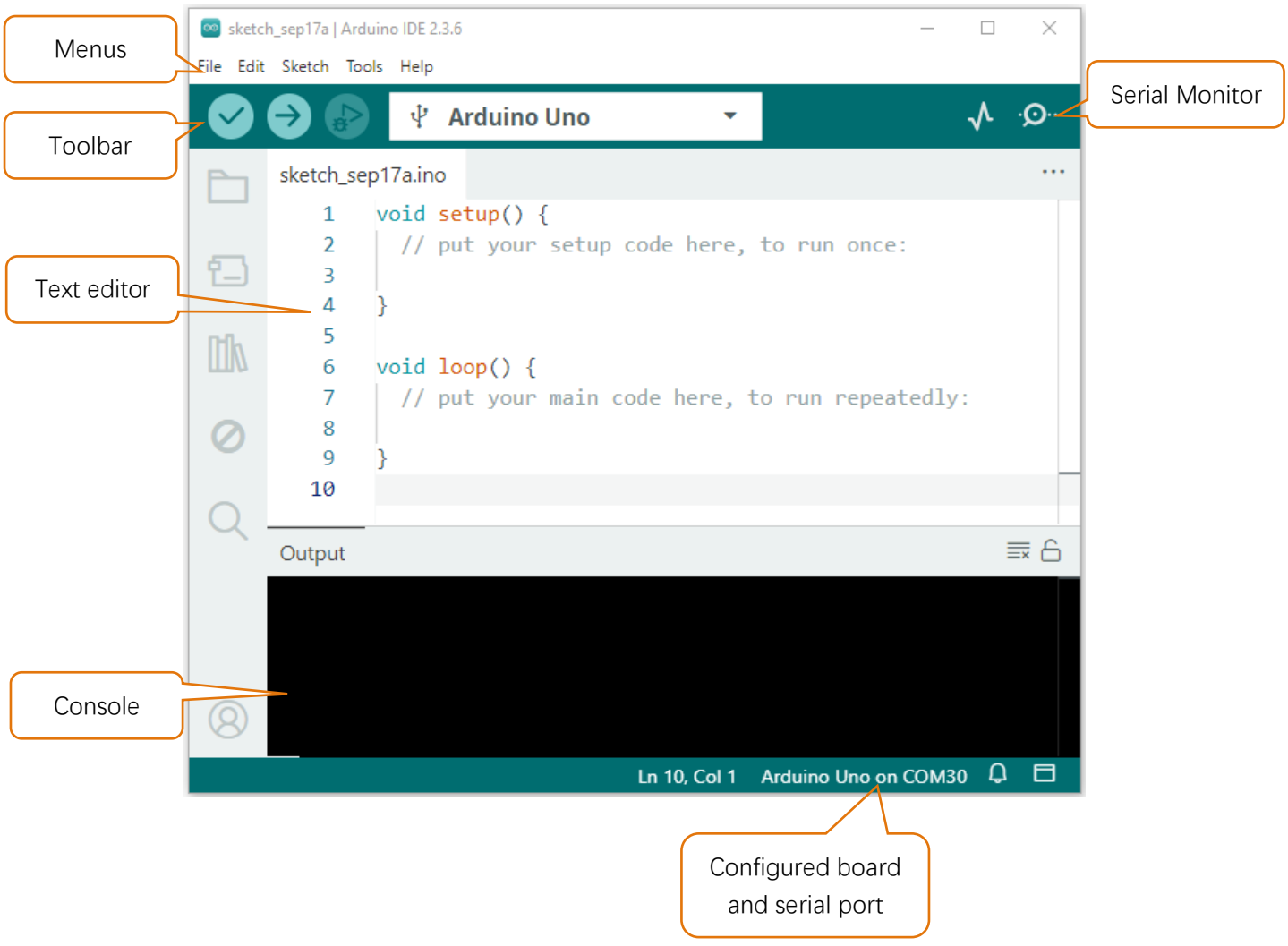
Arduino Software (IDE) is used to write and upload the code for Arduino Board.
First, install Arduino Software (IDE): visit https://www.arduino.cc/en/software/

Select and download corresponding installer based on your operating system. If you are a Windows user, please select the "Windows" to download and install the driver correctly.



After the downloading completes, run the installer. For Windows users, there may pop up an installation dialog box of driver during the installation process. When it is popped up, please allow the installation.
After installation is completed, an shortcut will be generated in the desktop.

Run it. The interface of the software is as follows:

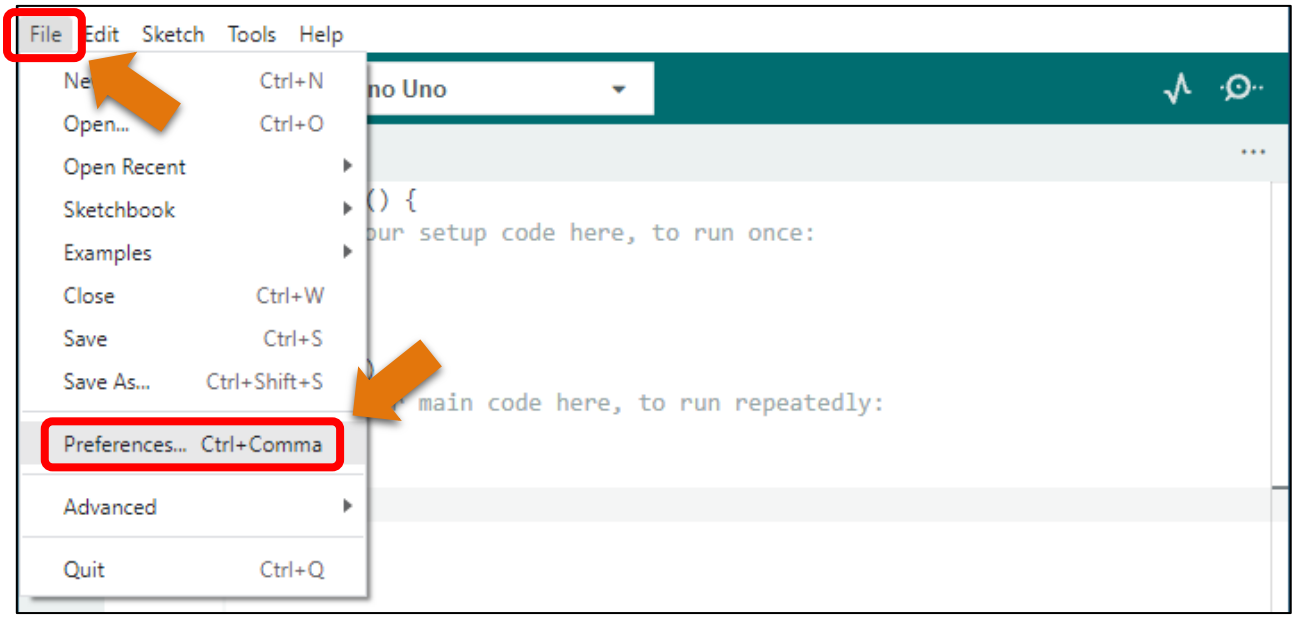


Programs written with Arduino IDE are called sketches. These sketches are written in a text editor and are saved with the file extension.ino. The editor has features for cutting/pasting and for searching/replacing text. The console displays text output by the Arduino IDE, including complete error messages and other information. The bottom right-hand corner of the window displays the configured board and serial port. The toolbar buttons allow you to verify and upload programs, open the serial monitor, and access the serial plotter.

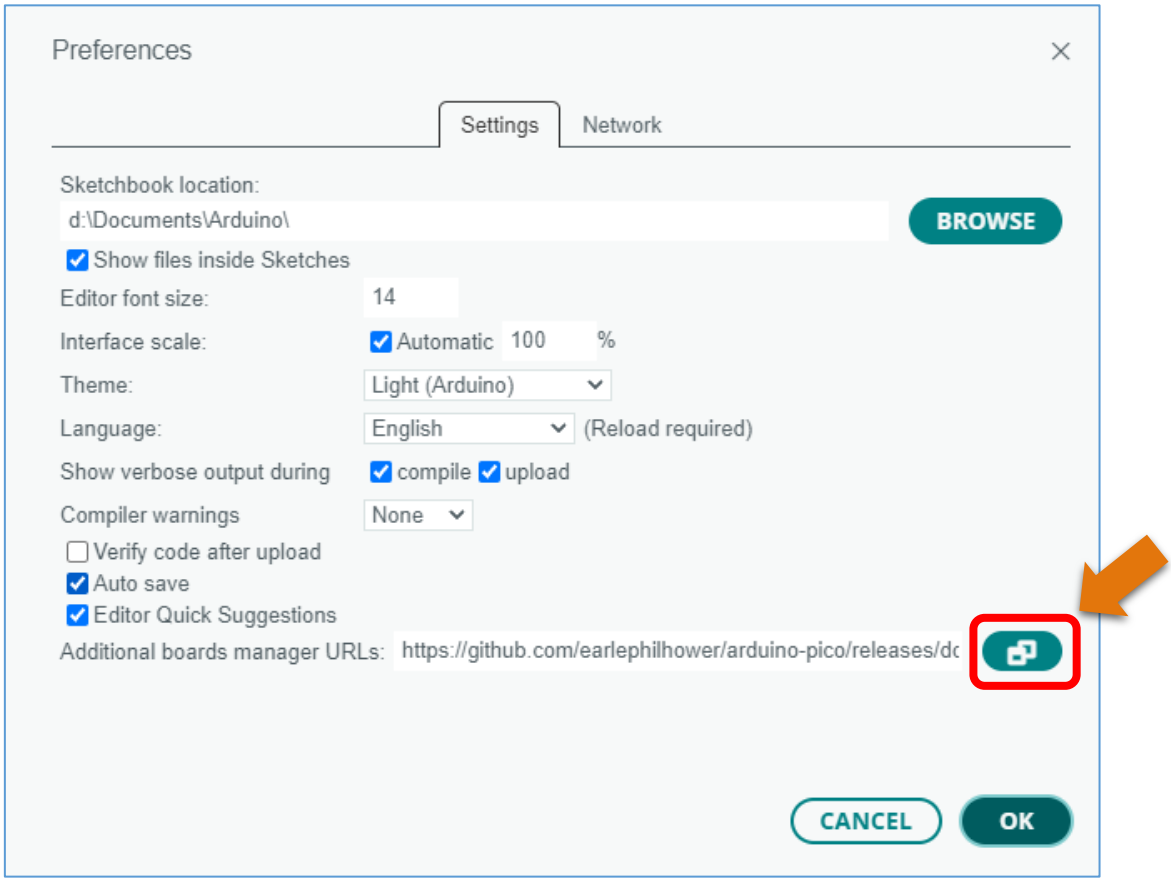
	Verify Checks your code for errors compiling it.
	Upload Compiles your code and uploads it to the configured board.
	Debug Troubleshoot code errors and monitor program running status.
	Serial Plotter Real-time plotting of serial port data charts.
	Serial Monitor Used for debugging and communication between devices and computers.

Environment Configuration

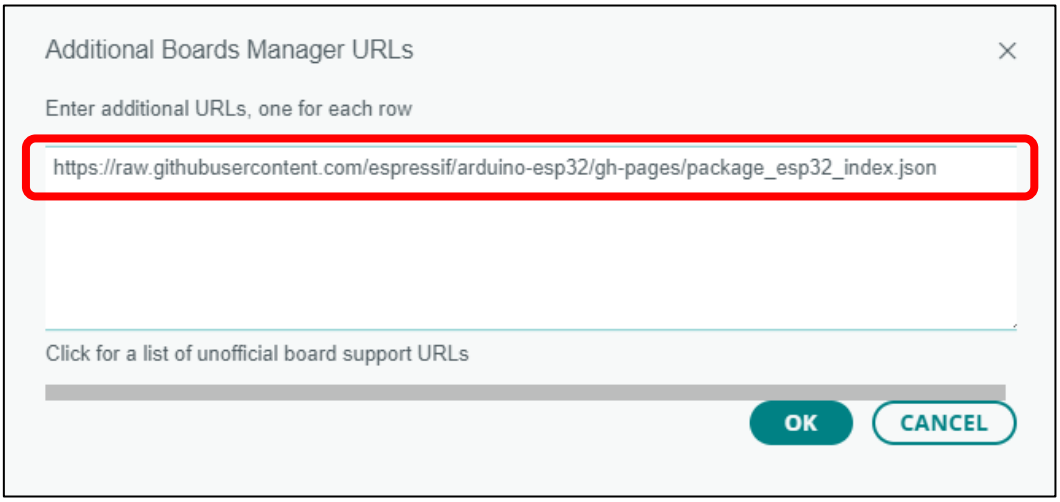
First, open the software platform arduino, and then click File in Menus and select Preferences.



Second, click on the symbol behind "Additional Boards Manager URLs"



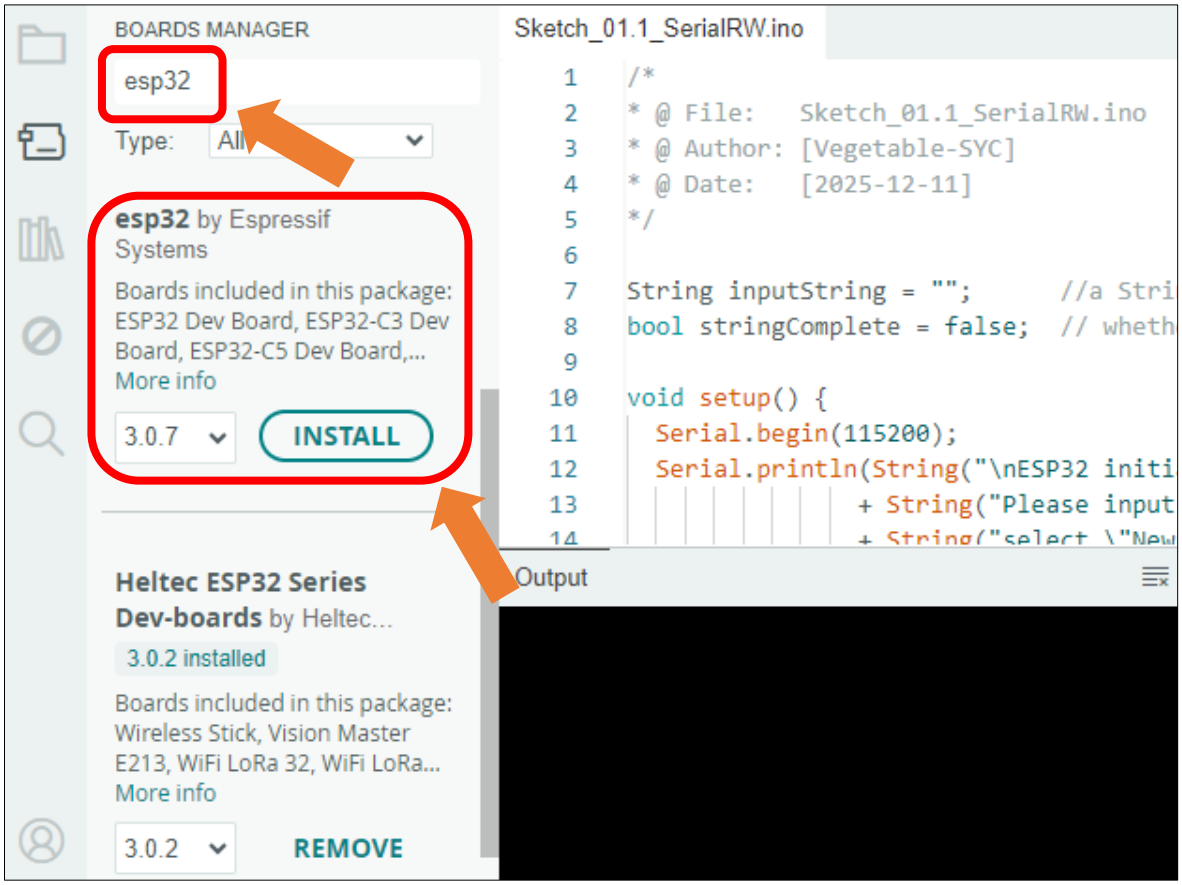
Third, fill in https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json in the new window, click OK, and click OK on the Preferences window again.



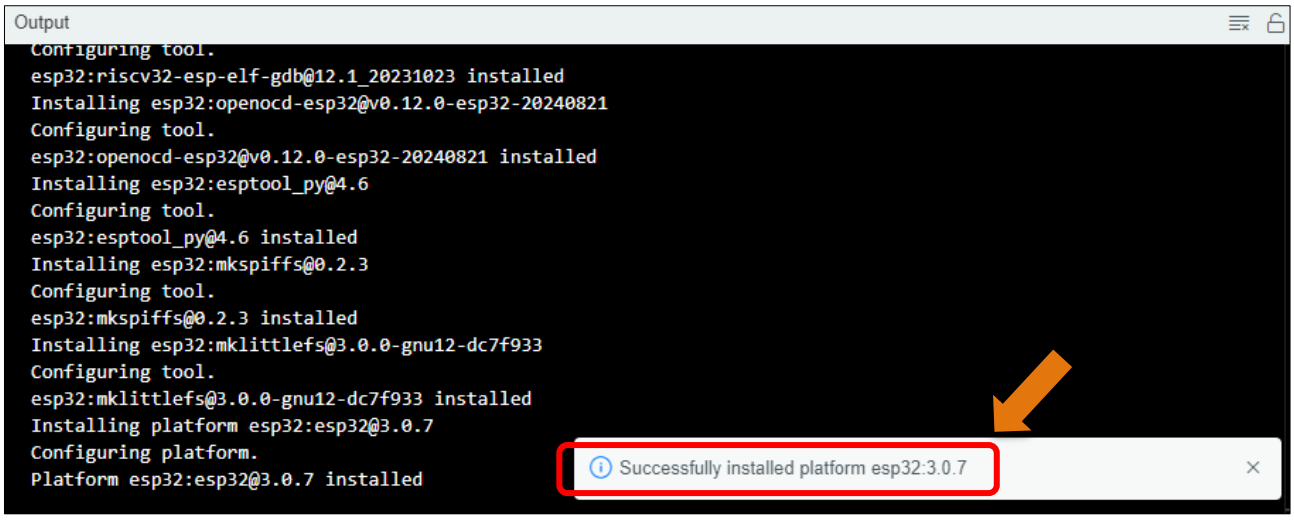
Note: if you copy and paste the URL directly, you may lose the "-". Please check carefully to make sure the link is correct.

Fourth, click "Boards Manager". Enter "**esp32**" in Boards manager, select **3.0.7**, and click "**INSTALL**".

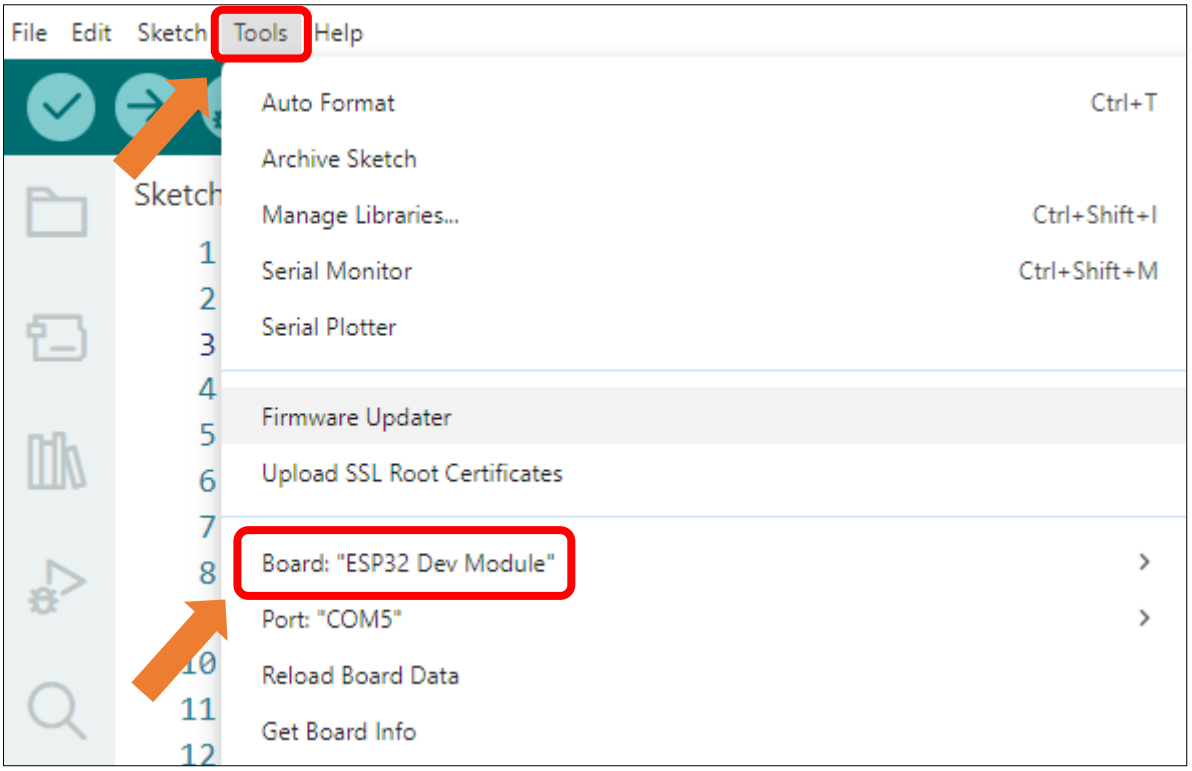
Please note: Repeated testing has revealed that higher versions of the board support package may cause compatibility issues with certain peripherals. It is recommended to use the more stable version 3.0.7.



Arduino will download these files automatically. Wait for the installation to complete.



When finishing installation, click Tools in the Menus again and select Board: "ESP32 Dev Module", and then you can see information of ESP32.

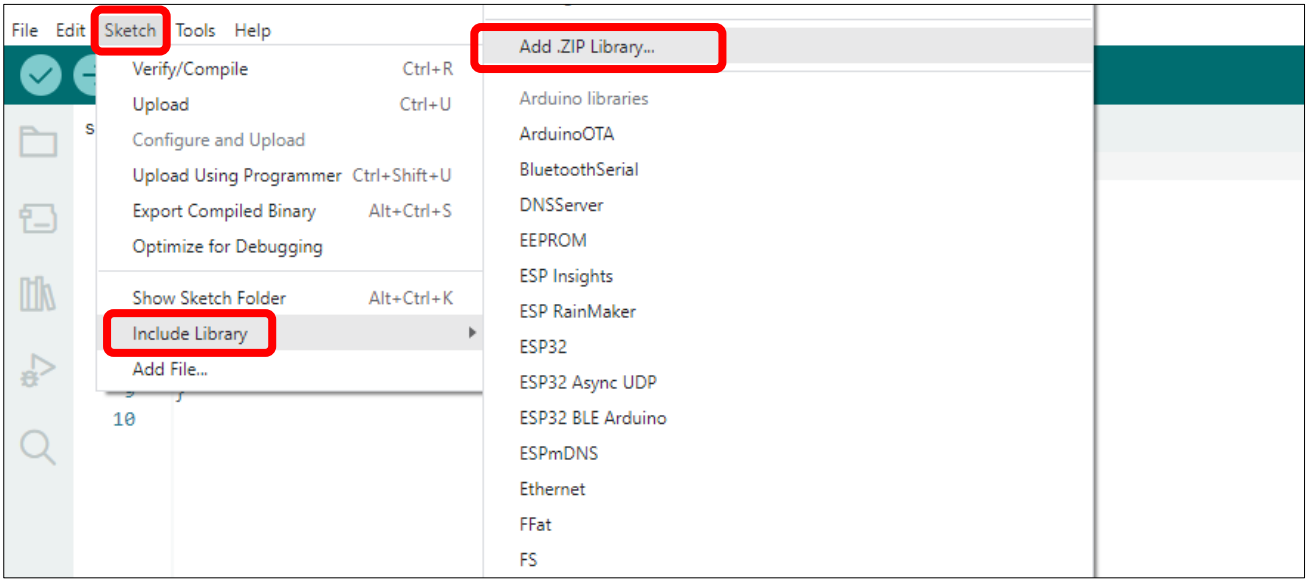


Library Installation

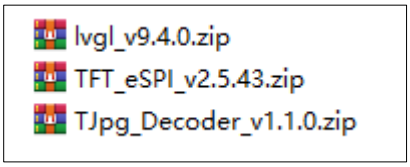
Before starting the learning process, it is necessary to install some libraries in advance to enable the code to be compiled properly. For convenience, we have already packaged these libraries and placed them in the **Freenove_ESP32_Mini_TV/Libraries** folder. Please refer to the following steps to install these libraries into the Arduino IDE.

1. Open Arduino IDE.

2. Select Sketch->Include Library->Add .ZIP library….

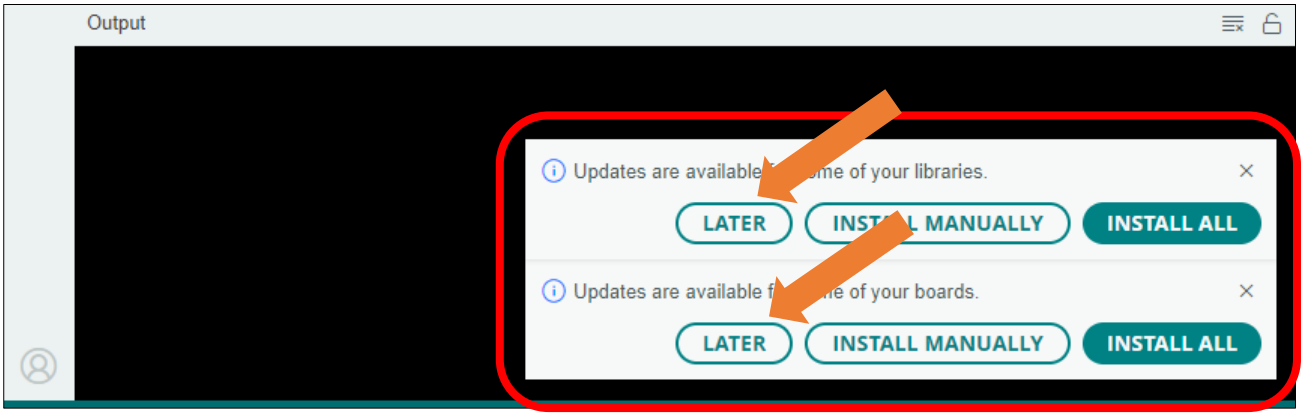


3. On the newly pop-up window, select the files from the **Freenove_ESP32_Mini_TV/Libraries**. Click Open to install the library.



4. **Repeat the above steps until all the three libraries are installed to Arduino.** So far, all libraries have been installed.

Note: Some libraries are not the latest version. Please do not update them even if it prompts every time you open the IDE. Just click LATER. Otherwise, it may lead to compilation failure.



Chapter 1 Serial

Project 1.1 SerialRW

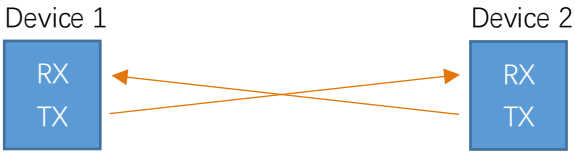
Related Knowledge

Serial and parallel communication

Serial communication uses one data cable to transfer data one bit by another in turn, while parallel communication means that the data is transmitted simultaneously on multiple cables. Serial communication takes only a few cables to exchange information between systems, which is especially suitable for computers to computer, long distance communication between computers and peripherals. Parallel communication is faster, but it requires more cables and higher cost, so it is not appropriate for long distance communication.

Serial communication

Serial communication generally refers to the Universal Asynchronous Receiver/Transmitter (UART), which is commonly used in electronic circuit communication. It has two communication lines, one is responsible for sending data (TX line) and the other for receiving data (RX line). The serial communication connections of two devices use is as follows:



Before serial communication starts, the baud rate in both sides must be the same. Only use the same baud rate can the communication between devices be normal. The baud rates commonly used are 9600 and 115200.

Component List

Freenove ESP32 Mini TV 	USB cable x1 

Circuit

Connect Freenove ESP32 Mini TV to the computer with USB cable.

Sketch

Open "**Sketch_01.1_SerialRW**" folder under **"Freenove_ESP32_Mini_TV\Sketch"** and double-click "**Sketch_01.1_SerialRW.ino**".

Sketch_01.1_SerialRW

```
1  String inputString = "";      //a String to hold incoming data
2  bool stringComplete = false;  // whether the string is complete
3
4  void setup() {
5     Serial.begin(115200);
6     Serial.println(String("\nESP32 initialization completed!\n")
7                    + String("Please input some characters,\n")
```

```
8                   + String("select \"Newline\" below and click send button. \n"));
9   }
10
11  void loop() {
12    if (Serial.available()) {         // judge whether data has been received
13      char inChar = Serial.read();          // read one character
14      inputString += inChar;
15      if (inChar == '\n') {
16        stringComplete = true;
17      }
18    }
19    if (stringComplete) {
20      Serial.printf("InputString: %s", inputString);
21      inputString = "";
22      stringComplete = false;
23    }
24  }
```

Code Explanation

Set the baud rate to 115200.

```
5   Serial.begin(115200);
```

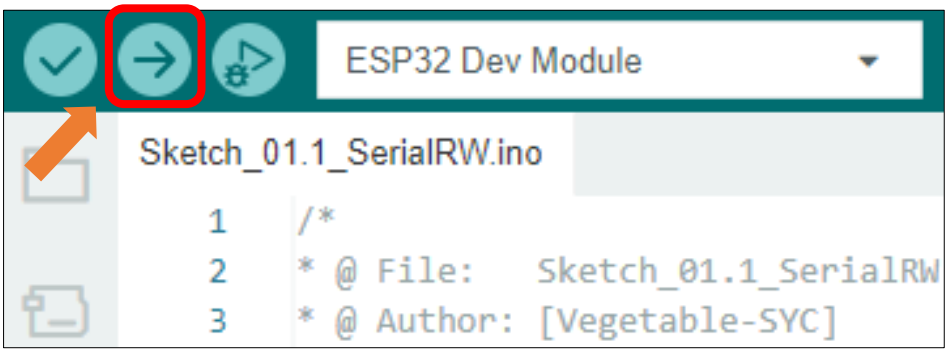
Determine whether there is data in the serial port buffer.

```
8   if (Serial.available())
```

Receive serial port data and save it in the inputString string.

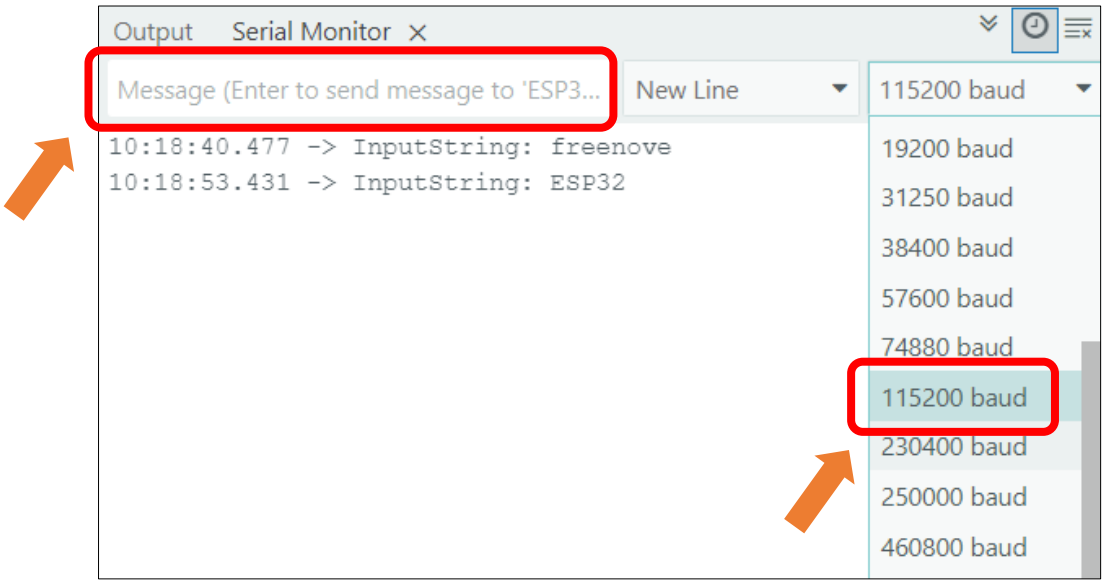
```
13  char inChar = Serial.read();          // read one character
14  inputString += inChar;
```

This code achieves the function of printing data on serial monitor. Click "**Upload**" to upload the code to Freenove ESP32 Mini TV.



After downloading the code, open the serial port monitor, and set the baud rate to **115200**, input any data in the messages bard and press Enter key, Freenove ESP32 Mini TV will print the received data.

Please note: This chapter does not involve the use of the screen. After the code for this chapter, the screen may not light up, which is normal and not a hardware malfunction. If you need to verify whether the screen is functioning properly, please refer to Chapter 6 for testing.



Reference

`int available()`
Serial.available()checks the number of bytes currently available to read in the Serial receive buffer. It returns the number of bytes available (int type), or 0 if the buffer is empty.
`int read ()`
Serial.read()reads one byte of data from the Serial receive buffer and returns it as an int. If no data is available to read, it returns -1.

Chapter 2 Touch

Project 2.1 Touch Test

Component List

Freenove ESP32 Mini TV 	USB cable x1 

Component knowledge

The fundamental principle of the ESP32's capacitive touch sensing is to monitor variations in the capacitance of its GPIO pins. Physically, a touch-capable pin and its associated sensing pad constitute an equivalent capacitor. The proximity of a human finger, which possesses inherent capacitance, increases the overall capacitance at the pin. For detection, the ESP32 employs an internal circuit to cycle through charging and discharging the pin at a high frequency. It records the number of pulses (the count of completed cycles) in a fixed duration. As a larger capacitance extends the charge/discharge time, the recorded pulse count within that period is correspondingly reduced.

Freenove ESP32 Mini TV utilizes capacitive touch sensing technology in its design. Internally, a wire connect the touch-sensitive pins to a copper foil pad affixed to the inside of the casing. This configuration transforms specific areas on the exterior shell into an active touch sensor, allowing the device to be controlled by simply tapping the designated spots on the casing.

Important Note: The internal disassembly diagram shown below is provided for illustrative purposes only. Disassembling the device yourself is strongly discouraged. The unit is secured with extremely small screws that are highly susceptible to being lost or damaged during removal, which can compromise the structural integrity of the device.

Circuit

Connect Freenove ESP32 Mini TV to the computer with USB cable.

Sketch

Open "**Sketch_02.1_Touch_Test**" folder under **"Freenove_ESP32_Mini_TV\Sketch"** and double-click "**Sketch_02.1_Touch_Test.ino**".

Sketch_02.1_Touch_Test

```
1  const int TOUCH_PIN = 32;
2
3  void setup() {
4     Serial.begin(115200);
5     delay(1000);
6     Serial.println("ESP32 Touch Test");
7     touchSetCycles(0xf000, 0x1000);
8  }
9
10 void loop() {
11    Serial.println(touchRead(TOUCH_PIN));
12    delay(100);
13 }
```

Code Explanation

Set the baud rate to 115200.

```
4  Serial.begin(115200);
```

Configure touch detection parameters

```
7  touchSetCycles(0xf000, 0x1000);
```

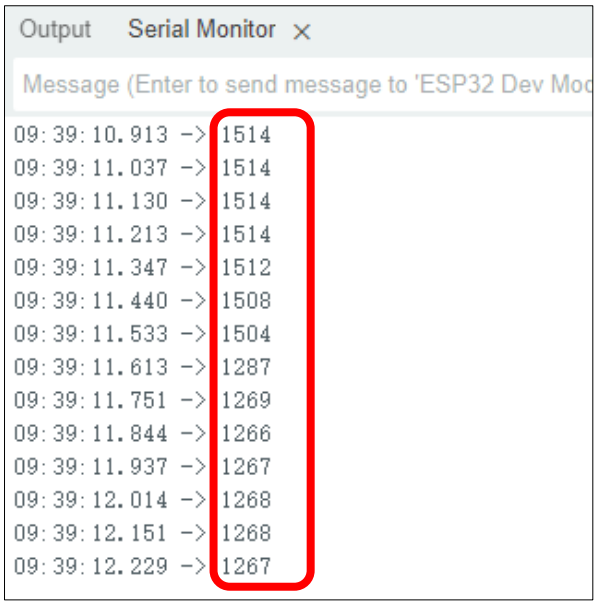
Print capacitance values on the serial monitor

```
7  Serial.println(touchRead(TOUCH_PIN));
```

The purpose of this code is to display data on the serial monitor. Click "**Upload**" to upload the code to Freenove ESP32 Mini TV.

After downloading the code, open the serial port monitor, and set the baud rate to **115200**

Touching the capacitive touch area of the Freenove ESP32 Mini TV with your finger will significantly reduce the capacitance reading.



Please note: Repeated testing has revealed that newer versions of the board support package may cause compatibility issues with certain peripherals. If abnormal serial port data transmission occurs, please use the more stable version 3.0.7.

Project 2.2 Touch Button

Building on the understanding of capacitive touch operation principles from the previous section, this part will focus on accurately distinguishing between short and long presses using these principles.

Component List

Freenove ESP32 Mini TV 	USB cable x1 

Circuit

Connect Freenove ESP32 Mini TV to the computer with USB cable.

Sketch

Open "**Sketch_02.2_Touch_Button**" folder under **"Freenove_ESP32_Mini_TV\Sketch"** and double-click "**Sketch_02.2_Touch_Button.ino**".

Sketch_02.2_Touch_Button

```
1  /*
2  * @ File:   Sketch_02.2_Touch_Button.ino
3  * @ Author: [Vegetable-SYC]
4  * @ Date:   [2025-12-19]
```

```
*/

const int TOUCH_PIN = 32;
const int TOUCH_THRESHOLD = 80; // Touch sensitivity threshold
const int PRESS_Time = 3000;    // Threshold duration for long press (ms)
uint32_t Touch_Time = 0;        // Timestamp of the initial touch
int Touch_data = 0, Touch_old_data = 0;
int short_press_num = 0;
int long_press_num = 0;
bool LONG_PRESS_FLAG = 0;

void setup() {
  Serial.begin(115200);
  delay(1000);  // Wait for serial monitor to initialize

  /* Configure touch sensor cycles */
  touchSetCycles(0xf000, 0x1000);

  /* Initial calibration and baseline data reading */
  Touch_old_data = touchRead(TOUCH_PIN);
  Serial.println(Touch_old_data);
}

void loop() {
  // Read touch sensor data
  Touch_data = touchRead(TOUCH_PIN);

  // Track and update the maximum baseline value
  if(Touch_old_data < Touch_data)
    Touch_old_data = Touch_data;

  // Press Detection Logic
  if((Touch_old_data - Touch_data) > TOUCH_THRESHOLD)
  {
    // Start timer on initial touch detection
    if(Touch_Time == 0)
      Touch_Time = millis();

    // Evaluate if the press duration meets Long Press requirement
    if(millis() - Touch_Time > PRESS_Time)
    {
      Touch_Time = 0;
      LONG_PRESS_FLAG = 1;

```

```
49          Serial.print("Long Press Detected: ");
50          long_press_num++;
51          Serial.println(long_press_num);
52        }
53      }
54      else {
55        // Release Detection Logic
56        if(Touch_Time != 0)
57        {
58          Touch_Time = 0;
59
60          if(LONG_PRESS_FLAG != 0)
61          {
62            // Event: Long press released
63            LONG_PRESS_FLAG = 0;
64          }
65          else
66          {
67            // Event: Short press released
68            Serial.print("Short Press Detected: ");
69            short_press_num++;
70            Serial.println(short_press_num);
71          }
72        }
73      }
74  }
```

Code Explanation

Preset basic parameters

```
7   const int TOUCH_PIN = 32;
8   const int TOUCH_THRESHOLD = 80; // Touch sensitivity threshold
9   const int PRESS_Time = 3000;    // Threshold duration for long press (ms)
```

Set the baud rate to 115200.

```
17  Serial.begin(115200);
```

Configure touch detection parameters.

```
21  touchSetCycles(0xf000, 0x1000);
```

Obtain capacitive touch reference value.

```
24  Touch_old_data = touchRead(TOUCH_PIN);
```

Continuously reading the capacitive touch values.

```
30  Touch_data = touchRead(TOUCH_PIN);
```

Determine if the difference between the current capacitive touch value and the reference value exceeds the preset touch threshold.

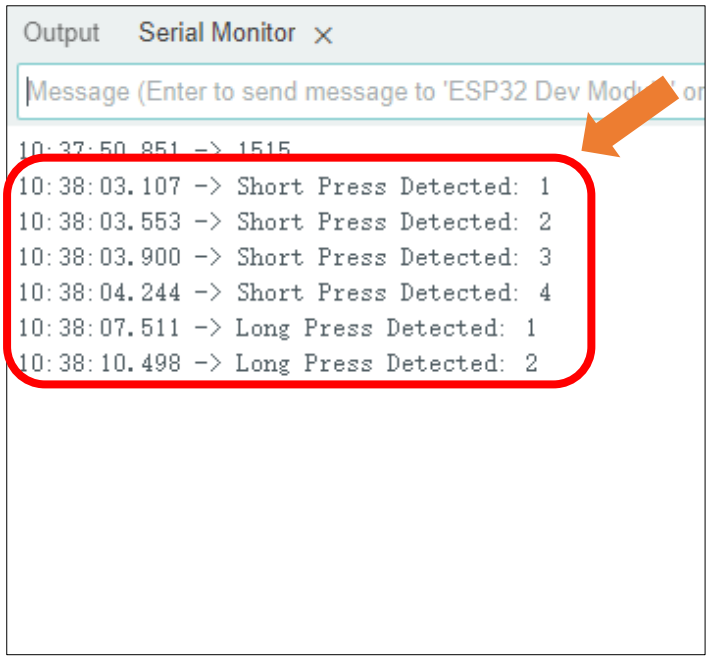
```
37  if((Touch_old_data - Touch_data) > TOUCH_THRESHOLD)
```

Determine if the touch duration exceeds the long-press time threshold.

```
44  if(millis() - Touch_Time > PRESS_Time)
```

The purpose of this code is to display data on the serial monitor. Click "**Upload**" to upload the code to Freenove ESP32 Mini TV.

After downloading the code, open the serial port monitor, and set the baud rate to **115200**
Touch on the capacitive area of the Freenode ESP32 Mini TV, if the duration exceeds 3 seconds, it is classfied as a long press; otherwise, it is considered a short press.



Please note: This chapter does not involve the use of the screen. After the code for this chapter, the screen may not light up, which is normal and not a hardware malfunction. If you need to verify whether the screen is functioning properly, please refer to Chapter 6 for testing.

Chapter 3 EEPROM

Project 3.1 EEPROM

Component List

Freenove ESP32 Mini TV 	USB cable x1 

Component knowledge

EEPROM

The primary function of EEPROM (Electrically Erasable Programmable Read-Only Memory) is to enable non-volatile data storage, ensuring critical information remains preserved after power loss or system restart. In the ESP32 architecture, EEPROM is not a physical chip but a logical storage space emulated using a section of the internal Flash memory. Due to the limited erase-write cycles of Flash memory, frequent repeated write operations should be avoided in practical applications.

Circuit

Connect Freenove ESP32 Mini TV to the computer with USB cable.

Sketch

Open "**Sketch_03.1_EEPROM**" folder under **"Freenove_ESP32_Mini_TV\Sketch"** and double-click "**Sketch_03.1_EEPROM.ino**".

Sketch_03.1_EEPROM

```
#include "EEPROM.h"

#define EEPROM_SIZE 1
int addr = 0;
void setup() {
  // put your setup code here, to run once:
  Serial.begin(115200);
  EEPROM.begin(EEPROM_SIZE);
  int val = EEPROM.read(addr) + 1;
  delay(5);
  EEPROM.write(addr, val);
  EEPROM.commit();
  delay(1000);
  Serial.print("The data in EEPROM(0) is:");
  Serial.println(EEPROM.read(addr));
}

void loop() {
  // put your main code here, to run repeatedly:

}
```

Code Explanation

Introduce the necessary libraries.

```
1  #include "EEPROM.h"
```

Define the size of non-volatile storage space (unit: bytes).

```
3  #define EEPROM_SIZE 1
```

Define the target starting address of the storage operation.

```
7  int addr = 0;
```

Set the baud rate to 115200.

```
3  Serial.begin(115200);
```

Initialize EEPROM and allocate buffer size

```
8  EEPROM.begin(EEPROM_SIZE);
```

Read the target address data and increment it by 1

```
9  int val = EEPROM.read(addr) + 1;
```

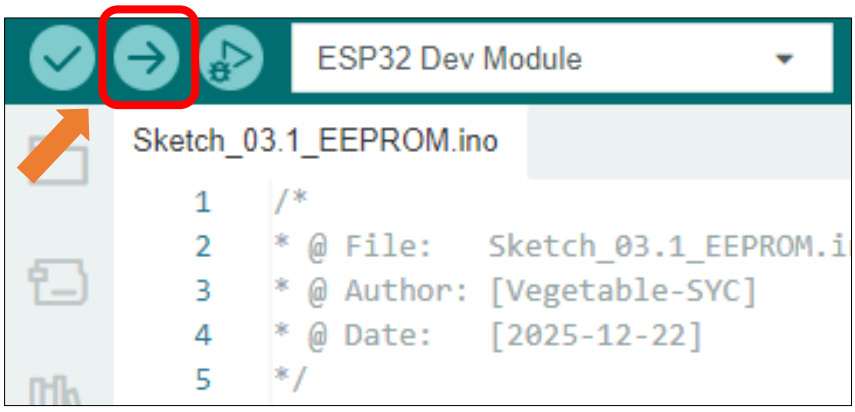
Write the new data to the memory buffer.

```
11 EEPROM.write(addr, val);
```

Update data to EEPROM.

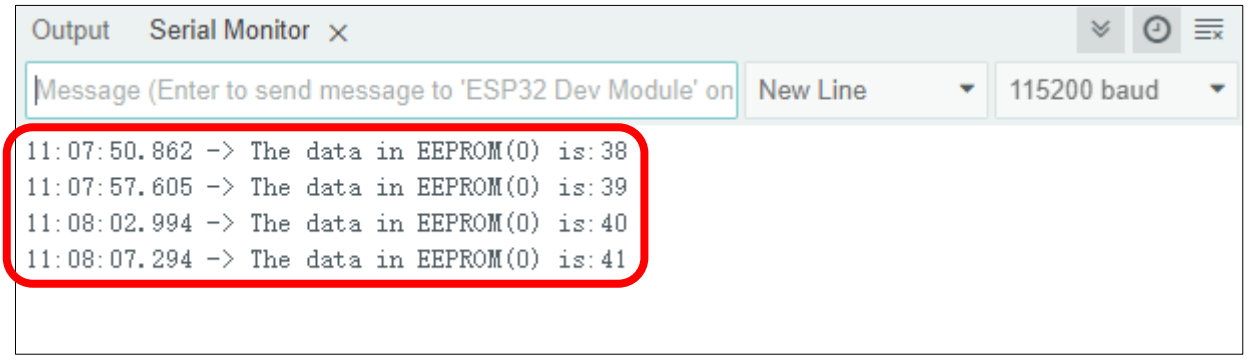
```
12 EEPROM.commit();
```

The purpose of this code is to display data on the serial monitor. Click "**Upload**" to upload the code to Freenove ESP32 Mini TV.



After downloading the code, open the serial port monitor, and set the baud rate to **115200**

After viewing the current value on the serial monitor, disconnect and then reconnect the USB cable. You will observe that the value stored in the EEPROM persists through the power cycle. Upon reinitialization after reboot, the value increments from its previously stored state.



Please note: This chapter does not involve the use of the screen. After the code for this chapter is uploaded, the screen may not light up, which is normal and not a hardware malfunction. If you need to verify whether the screen is functioning properly, please refer to Chapter 6 for testing.

Chapter 4 BLE

Project 4.1 BLE USART

Component List

Freenove ESP32 Mini TV 	USB cable x1 

Component knowledge

BLE（Bluetooth Low Energy）

Low Energy Bluetooth (BLE) is a new feature introduced in the Bluetooth 4.0 specification, specifically designed for low-power devices and suitable for applications involving intermittent transmission of small amounts of data.

The BLE architecture follows a client-server model and consists of the following key components:
GATT (Generic Attribute Protocol): The foundational protocol for BLE communication, defining a hierarchical data structure of services and characteristics.
Service: A collection of data that performs a specific function or feature, containing one or more characteristics.
Characteristic: A specific data point within a service, consisting of a value and descriptors.
UUID: A 128-bit universally unique identifier used to distinguish services and characteristics.

BLE device can function simultaneously as a Peripheral (providing services) and a Central (connecting to peripherals). In this section, we will learn how to configure the Freenove ESP32 Mini TV as a peripheral device to provide services.

Circuit

Connect Freenove ESP32 Mini TV to the computer with USB cable.

Sketch

Open "**Sketch_04.1_BLE_USART**" folder under **"Freenove_ESP32_Mini_TV\Sketch"** and double-click "**Sketch_04.1_BLE_USART.ino**".

Sketch_04.1_BLE_USART

The following is the program code:

```
#include "BLEDevice.h"
#include "BLEServer.h"
#include "BLEUtils.h"
#include "BLE2902.h"
#include "String.h"

BLECharacteristic *pCharacteristic;
bool deviceConnected = false;
uint8_t txValue = 0;
long lastMsg = 0;
String rxload="Test\n";

#define SERVICE_UUID           "6E400001-B5A3-F393-E0A9-E50E24DCCA9E"
#define CHARACTERISTIC_UUID_RX "6E400002-B5A3-F393-E0A9-E50E24DCCA9E"
#define CHARACTERISTIC_UUID_TX "6E400003-B5A3-F393-E0A9-E50E24DCCA9E"

class MyServerCallbacks: public BLEServerCallbacks {
    void onConnect(BLEServer* pServer) {
```

```
        deviceConnected = true;
      };
      void onDisconnect(BLEServer* pServer) {
        deviceConnected = false;
      }
};

class MyCallbacks: public BLECharacteristicCallbacks {
      void onWrite(BLECharacteristic *pCharacteristic) {
        String rxValue = pCharacteristic->getValue();
        if (rxValue.length() > 0) {
          rxload="";
          for (int i = 0; i < rxValue.length(); i++){
            rxload +=(char)rxValue[i];
          }
        }
      }
};

void setupBLE(String BLEName){
    const char *ble_name=BLEName.c_str();
    BLEDevice::init(ble_name);
    BLEServer *pServer = BLEDevice::createServer();
    pServer->setCallbacks(new MyServerCallbacks());
    BLEService *pService = pServer->createService(SERVICE_UUID);
    pCharacteristic=
pService->createCharacteristic(CHARACTERISTIC_UUID_TX,BLECharacteristic::PROPERTY_NOTIFY);
    pCharacteristic->addDescriptor(new BLE2902());
    BLECharacteristic *pCharacteristic =
pService->createCharacteristic(CHARACTERISTIC_UUID_RX,BLECharacteristic::PROPERTY_WRITE);
    pCharacteristic->setCallbacks(new MyCallbacks());
    pService->start();
    pServer->getAdvertising()->start();
    Serial.println("Waiting a client connection to notify...");
}

void setup() {
    Serial.begin(115200);
    setupBLE("ESP32_BLE");
}

void loop() {
    long now = millis();
    if (now - lastMsg > 100) {
```

```
63        if (deviceConnected&&rxload.length()>0) {
64            Serial.println(rxload);
65            rxload="";
66        }
67        if(Serial.available()>0){
68            String str=Serial.readString();
69            const char *newValue=str.c_str();
70            pCharacteristic->setValue(newValue);
71            pCharacteristic->notify();
72        }
73        lastMsg = now;
74     }
75  }
```

Code Explanation

Include the necessary header libraries.

```
1  #include "BLEDevice.h"
2  #include "BLEServer.h"
3  #include "BLEUtils.h"
4  #include "BLE2902.h"
5  #include "String.h"
```

Define service UUID and characteristic UUID.

```
13 #define SERVICE_UUID           "6E400001-B5A3-F393-E0A9-E50E24DCCA9E"
14 #define CHARACTERISTIC_UUID_RX "6E400002-B5A3-F393-E0A9-E50E24DCCA9E"
15 #define CHARACTERISTIC_UUID_TX "6E400003-B5A3-F393-E0A9-E50E24DCCA9E"
```

The MyServerCallbacks class handles device connection and disconnection events and updates the deviceConnected status.

```
17 class MyServerCallbacks: public BLEServerCallbacks {
18       void onConnect(BLEServer* pServer) {
19         deviceConnected = true;
20       };
21       void onDisconnect(BLEServer* pServer) {
22         deviceConnected = false;
23       }
24 };
```

The MyServerCallbacks class handles the received data and save them to the rxload string.

```
26 class MyCallbacks: public BLECharacteristicCallbacks {
27       void onWrite(BLECharacteristic *pCharacteristic) {
28         String rxValue = pCharacteristic->getValue();
29         if (rxValue.length() > 0) {
30           rxload="";
31           for (int i = 0; i < rxValue.length(); i++){
32             rxload +=(char)rxValue[i];
33           }
34         }
```

```
        }
};
```

Set the baud rate to 115200.

```
Serial.begin(115200);
```

BLE Device Initialization, Service Creation, and Characteristic Setup

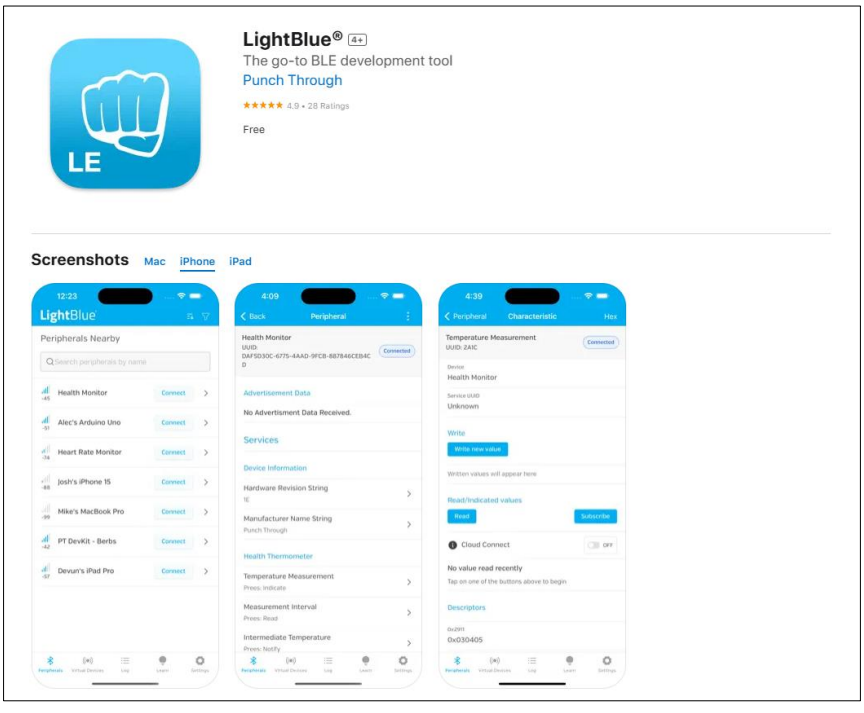
```
setupBLE("ESP32_BLE");
```

The Loop function check the connection status and serial data every 100 milliseconds.

```
void loop() {
  long now = millis();
  if (now - lastMsg > 100) {
    if (deviceConnected&&rxload.length()>0) {
        Serial.println(rxload);
        rxload="";
    }
    if(Serial.available()>0){
        String str=Serial.readString();
        const char *newValue=str.c_str();
        pCharacteristic->setValue(newValue);
        pCharacteristic->notify();
    }
    lastMsg = now;
  }
}
```

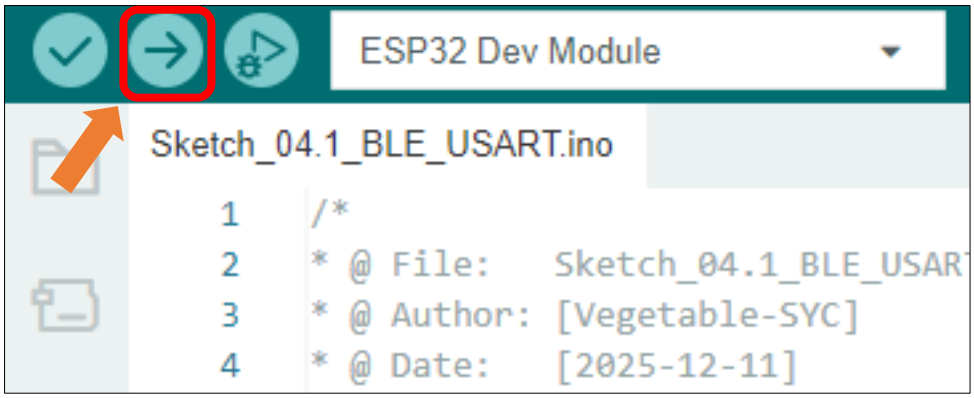
LightBlue

If you do not have this software installed on your phone, you can refer to this link:
https://apps.apple.com/us/app/lightblue/id557428110

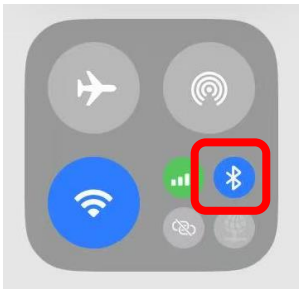


Click "**Upload**" to upload the code to Freenove ESP32 Mini TV.

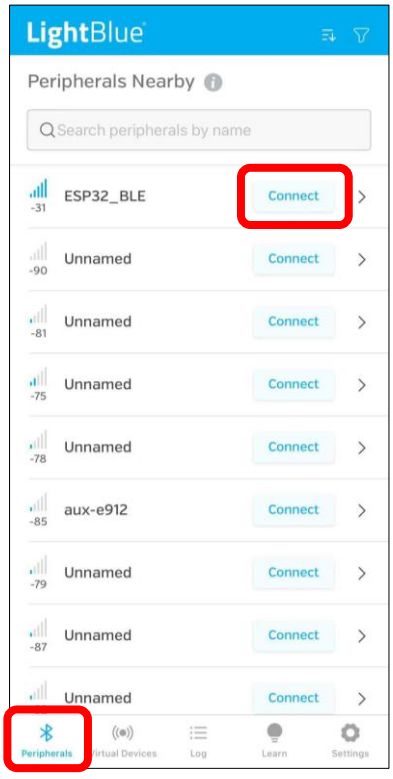
Please note: This chapter does not involve the use of the screen. After the code for this chapter, the screen may not light up, which is normal and not a hardware malfunction. If you need to verify whether the screen is functioning properly, please refer to Chapter 6 for testing.

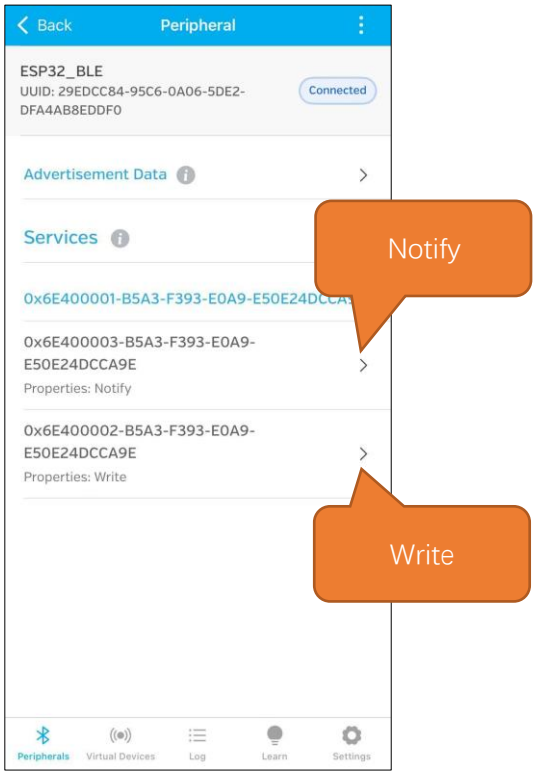


Turn ON Bluetooth on your phone, and open the LightBlue APP.



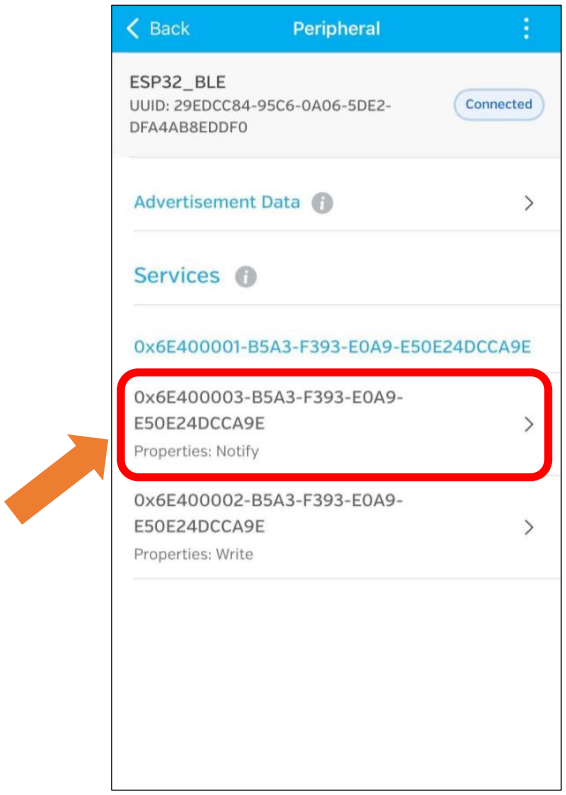
In the Scan page, swipe down to refresh the name of Bluetooth that the phone searches for. Click the Connection button of ESP32_BLE.



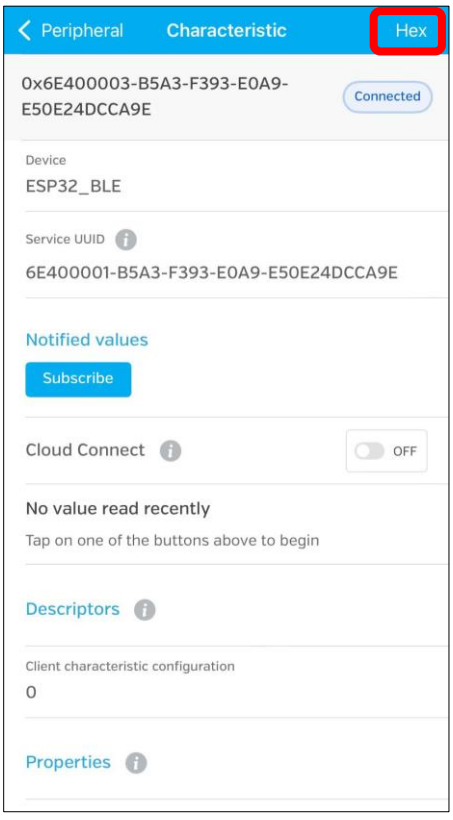


Receiving Data

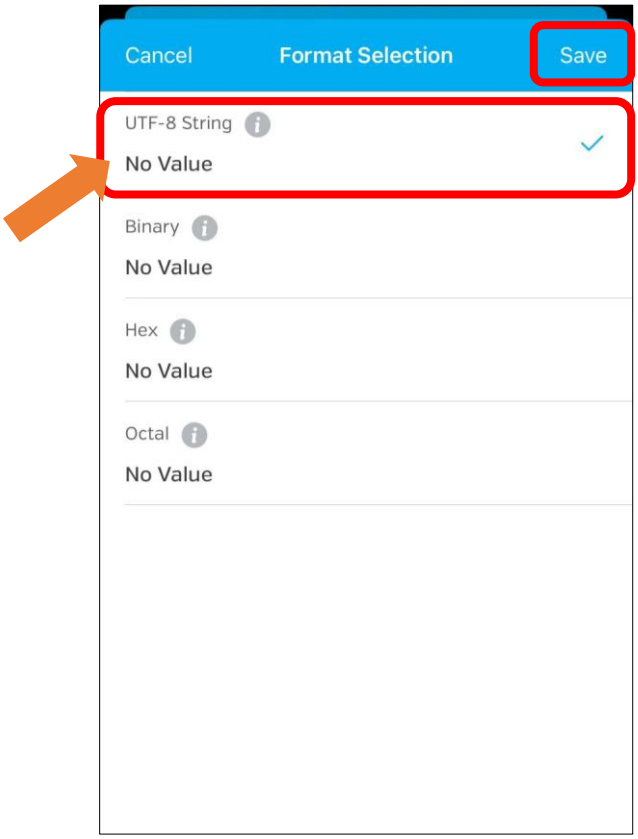
Click Notify



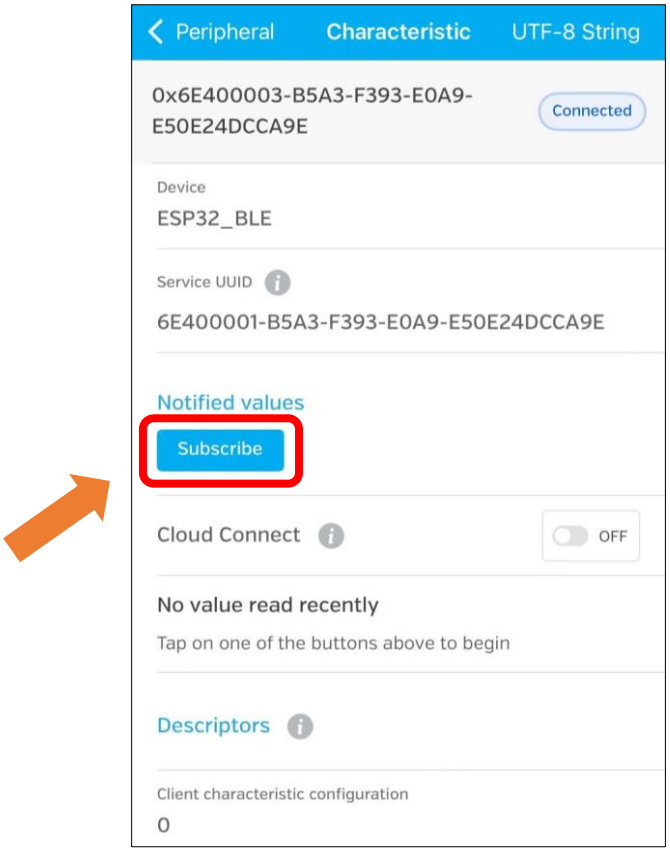
Click the top right corner to change the string type.



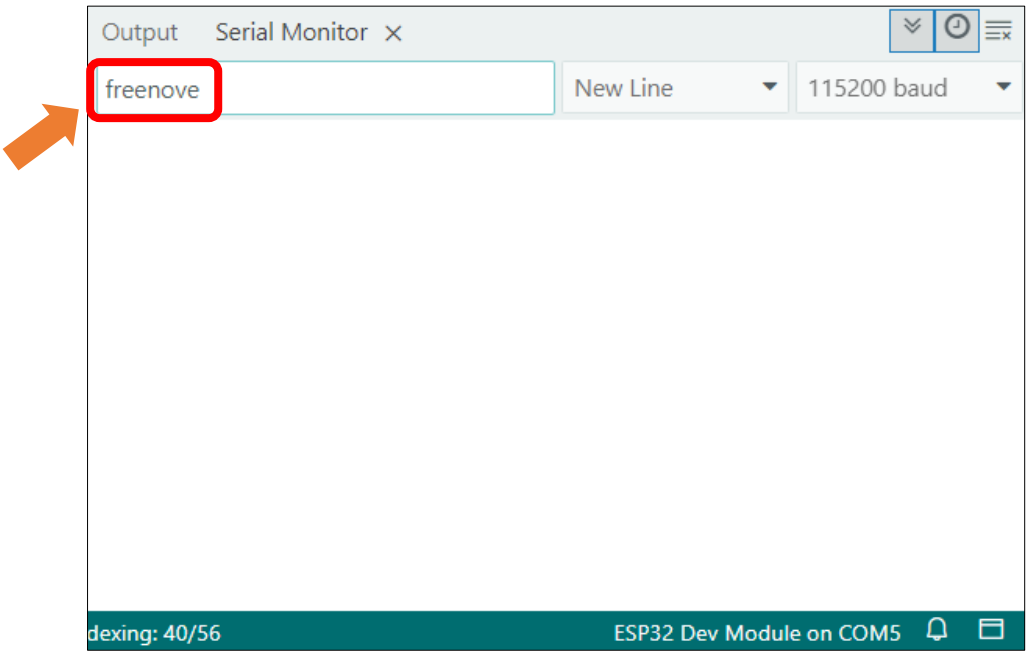
Select UTF-8 String, and click “Save”.



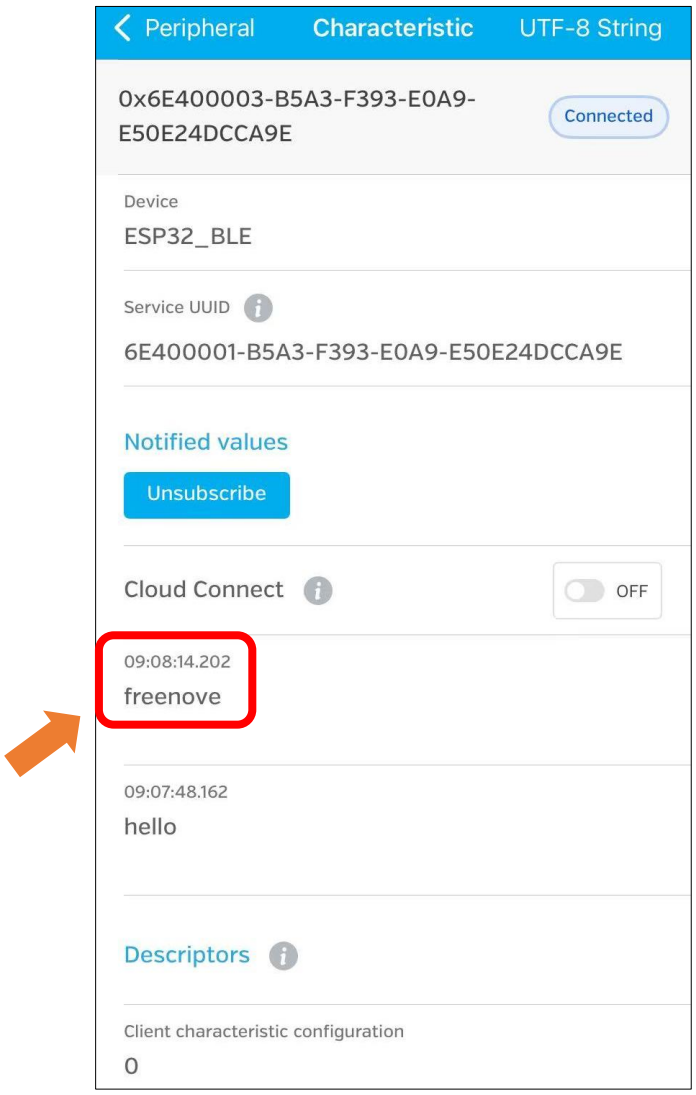
Click Subscribe



Send data on the serial monitor.

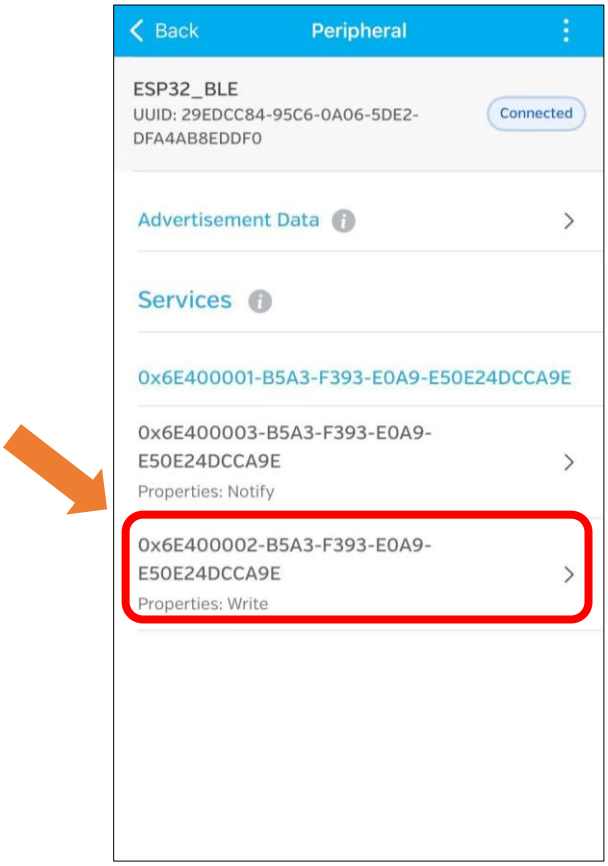


Data will be received on the LightBlue app.

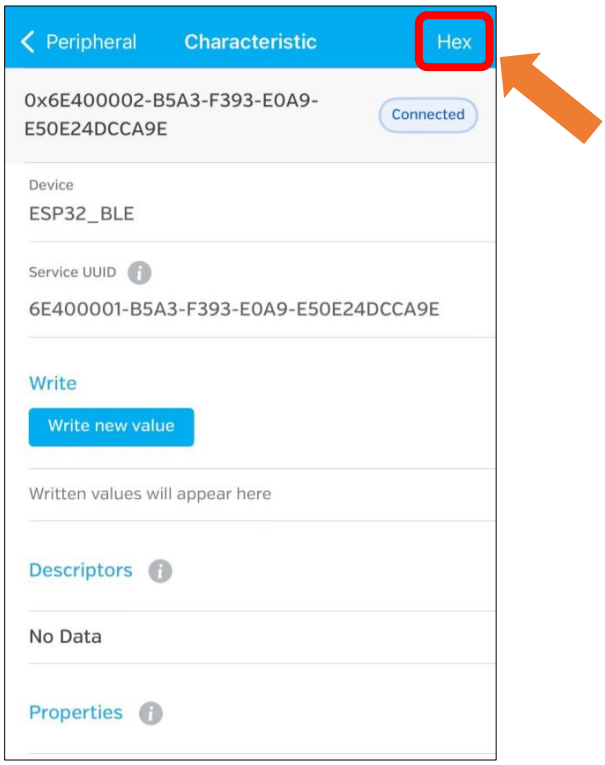


Sending Data

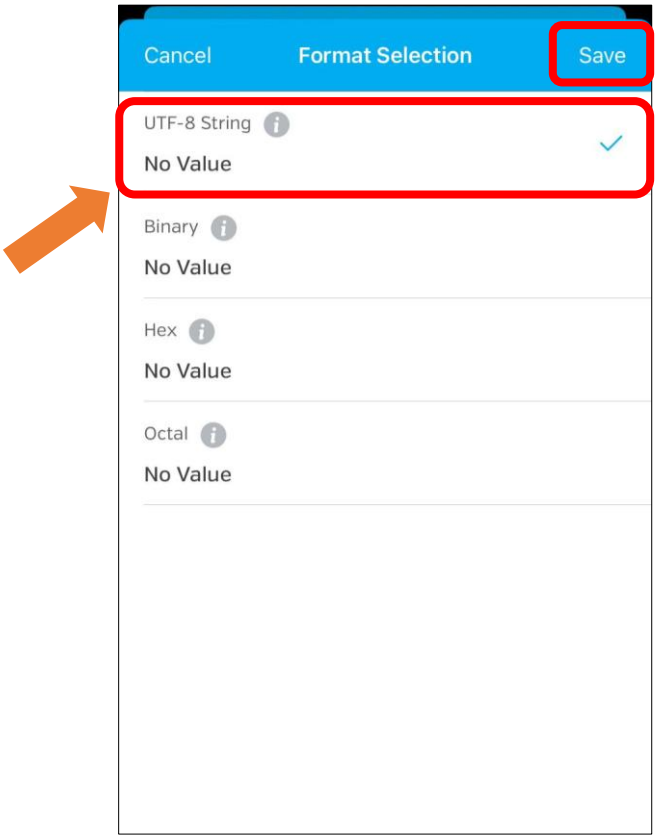
Click Write



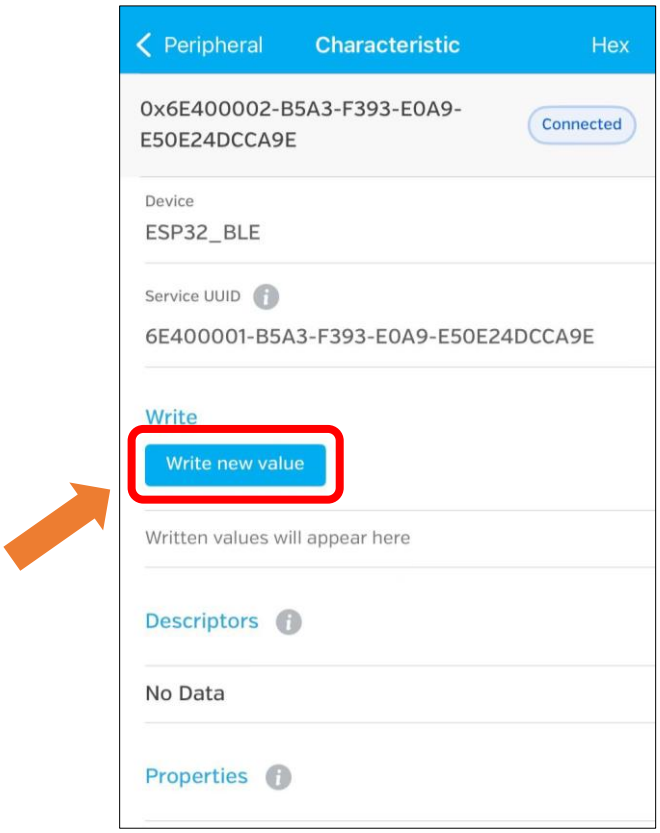
Click the top right corner to change the string type.



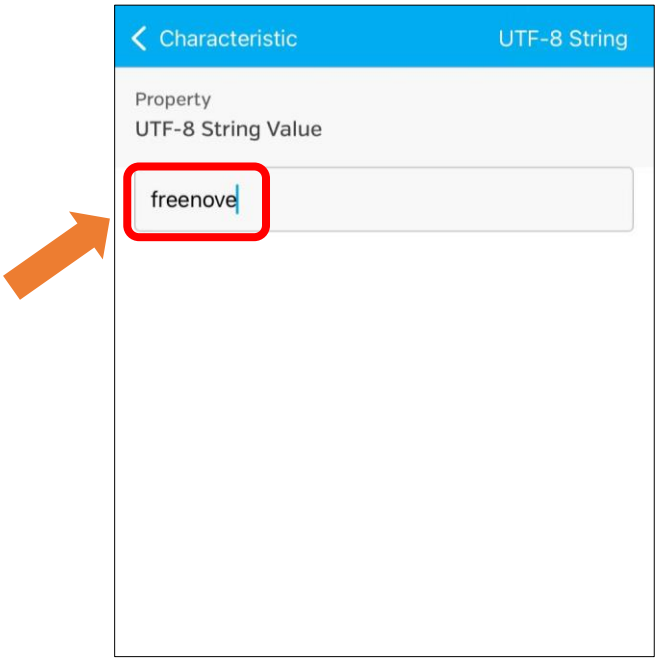
Select UTF-8 String and click “Save”.



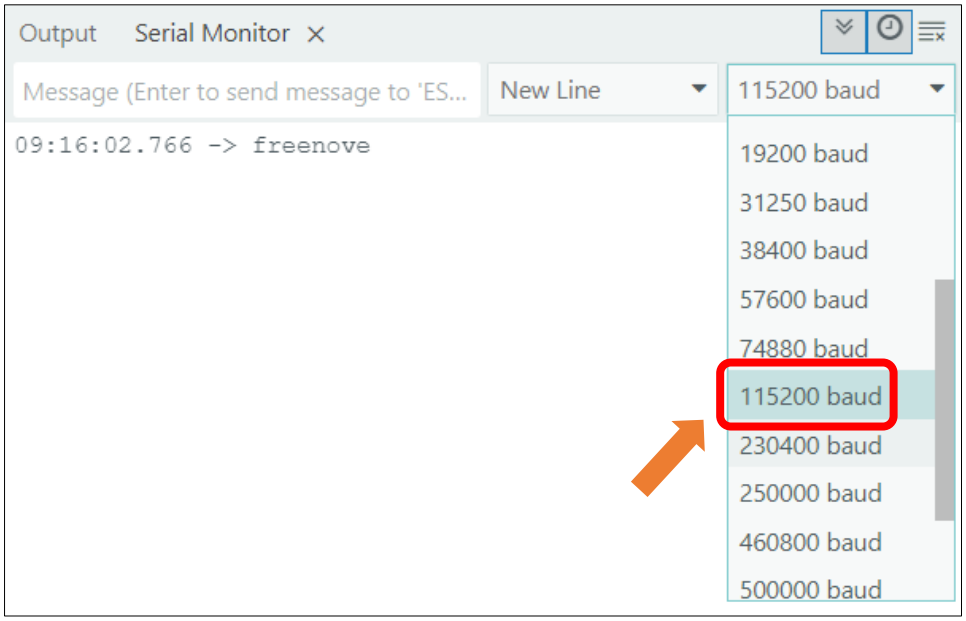
Click “Write new value”



Enter the messages to send.



Set the baud rate to 115200, and the serial monitor will print the data received.



Chapter 5 WIFI

Project 5.1 WIFI Web Servers LED

Component List

Freenove ESP32 Mini TV 	USB cable x1 

Component Knowledge

Wi-Fi

Wi-Fi is a wireless LAN (WLAN) technology compliant with the IEEE 802.11 standard, enabling devices to transmit data or connect to the internet via radio waves within short-range coverage (typically up to tens of meters). In embedded systems, Wi-Fi modules provide wireless communication capabilities, allowing devices to connect to routers, access cloud services, or interact with other devices.

Its core features include:

- Wireless connectivity (eliminating the need for physical cabling)
- Moderate to high-speed data transfer (depending on protocol versions such as 802.11n/ac)
- Secure communication (ensured by encryption protocols like WPA2/WPA3)

In embedded development, Wi-Fi is one of the key technologies for enabling IoT (Internet of Things) device connectivity.

Web

The Web (World Wide Web) is an internet-based information system that integrates global resources through hypertext links. In the embedded field, Web technology refers to devices equipped with lightweight built-in web servers, allowing users to remotely access the device interface via browsers (such as Chrome or Edge). By entering the device's IP address, users can open an interactive page to perform functions like status monitoring, parameter configuration, or firmware upgrades. Its core value lies in cross-platform compatibility (no need for dedicated software installation) and standardized protocols (HTTP/HTTPS), making it a mainstream solution for remote management of embedded devices.

HTML & CSS & JavaScript

- tags to define page elements, forming the foundational framework. In embedded web interfaces,

HTML describes the layout of components such as device status displays and configuration forms.

- **CSS (Cascading Style Sheets)** controls the visual presentation of web pages, including colors, fonts, spacing, and responsive layouts. Through CSS rules, developers style HTML elements into intuitive interactive interfaces. The combination of HTML and CSS enables the creation of lightweight device control pages without complex graphics libraries, reducing resource overhead in embedded systems.
- **JavaScript (a scripting language)** adds dynamic behavior and interaction logic to web pages. In embedded web interfaces, JavaScript plays a crucial role: it responds to user actions and enables asynchronous communication with the device backend through technologies like **AJAX** or **WebSocket**. This allows the webpage to fetch real-time device status data, update displays without refreshing, and send control commands or configuration parameters—delivering a smooth and efficient remote management experience.

Circuit

Connect Freenove ESP32 Mini TV to the computer with USB cable.

Sketch

Open "**Sketch_04.1_BLE_USART**" folder under **"Freenove_ESP32_Mini_TV\Sketch"** and double-click "**Sketch_04.1_BLE_USART.ino**".

Sketch_04.1_BLE_USART

The following is the program code:

```
1  #include <WiFi.h>
2  #include <WebServer.h>
3  #include "web.h"
4  // ---------------------------------------------------------
5  // WiFi configuration
```

```
6  // ---------------------------------------------------------------
7  const char* ssid =     "********";
8  const char* password = "********";
9
10 // Create a web server instance listening on HTTP port 80.
11 WebServer server(80);
12
13 // Serve an HTML page to the client.
14 void handleRoot() {
15    server.send(200, "text/html", index_html);
16 }
17
18 // Decode the POST request data.
19 void handleSubmit() {
20    if (server.hasArg("msg")) {
21      String receivedMsg = server.arg("msg");
22      Serial.println("-------------------------------------------------------------");
23      Serial.print("Received message: ");
24      Serial.println(receivedMsg);
25      Serial.println("-------------------------------------------------------------");
26
27      server.send(200, "text/plain", "OK");
28    } else {
29      server.send(400, "text/plain", "Bad Request");
30    }
31 }
32
33 // Process 404 (Not Found) error.
34 void handleNotFound() {
35    server.send(404, "text/plain", "Not found");
36 }
37
38 void setup() {
39    Serial.begin(115200);
40    delay(1000);
41
42    Serial.println();
43    Serial.print("Connect to WiFi: ");
44    Serial.println(ssid);
45
46    // Set up the WiFi connection.
47    WiFi.begin(ssid, password);
48
49    // Wait until WiFi connects successfully
```

```
50      while (WiFi.status() != WL_CONNECTED) {
51        delay(500);
52        Serial.print(".");
53      }
54
55      // Print the assigned local IP address
56      Serial.println("\nWiFi connected successfully!");
57      Serial.print("Access address: http://");
58      Serial.println(WiFi.localIP());
59
60      server.on("/", handleRoot);
61      server.on("/submit", handleSubmit);
62      server.onNotFound(handleNotFound);
63
64      // Start the HTTP server
65      server.begin();
66      Serial.println("HTTP Server is running");
67  }
68
69  void loop() {
70      // Continuously listen for and handle HTTP connection requests from clients
71      server.handleClient();
72  }
```

Code Explanation

Include the necessary header files.

```
1  #include <WiFi.h>
2  #include <WebServer.h>
3  #include "web.h"
```

Define WiFi name and password.

```
7  const char* ssid =     "********";
8  const char* password = "********";
```

Create a web server instance that listens on HTTP port 80..

```
11  WebServer server(80);
```

Connect WIFI.

```
47  WiFi.begin(ssid, password);
48
49  // Wait until WiFi connects successfully
50  while (WiFi.status() != WL_CONNECTED) {
51      delay(500);
52      Serial.print(".");
53  }
```

Continuously listen for and process HTTP connection requests from clients.

```
71  server.handleClient();
```

Input correct WiFi(2.4GHz) SSID and password.

```
// WiFi credentials
const char* ssid     = "**************";
const char* password = "**************";
```

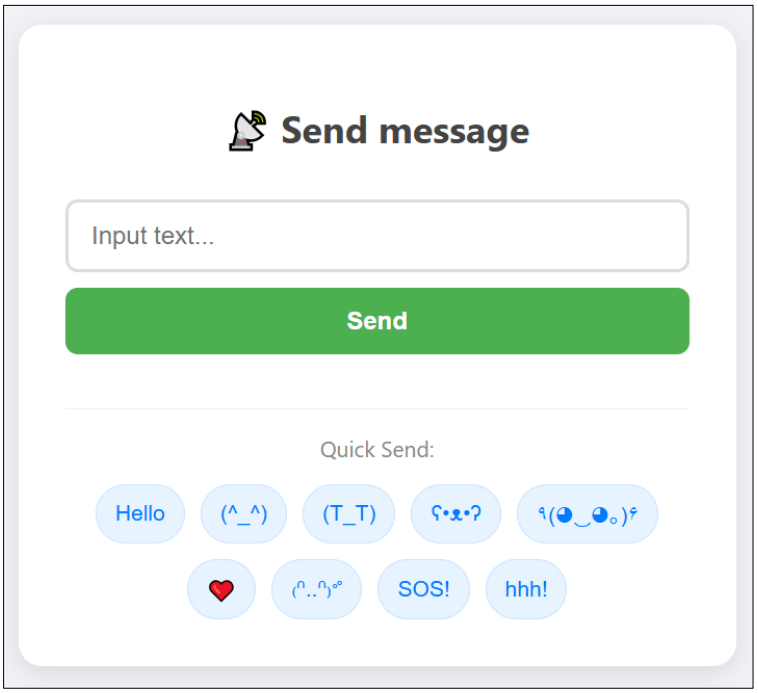
Click "**Upload**" to upload the code to Freenove ESP32 Mini TV, and set the baudrate to 115200.

When the serial monitor prints the following information, it means the network connection is successful. Use a mobile phone or computer browser to open the IP address printed by the serial monitor.

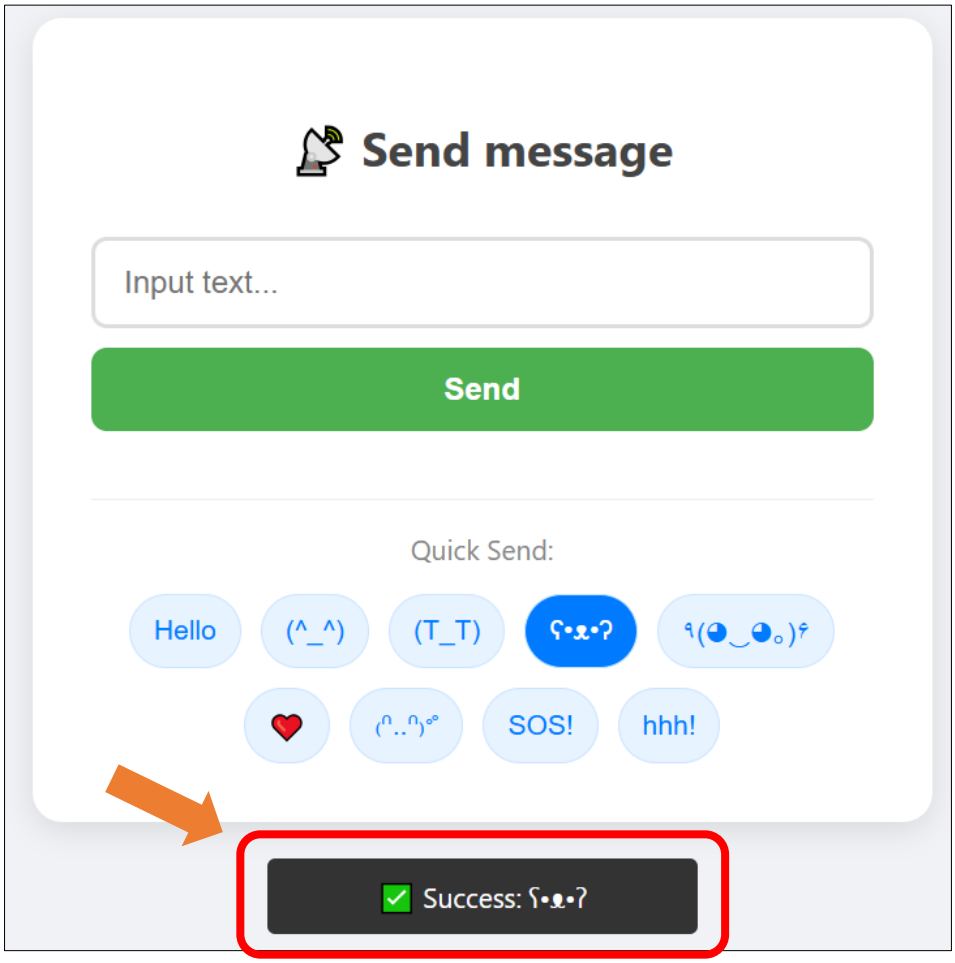
Please note: If it keeps printing dots, please confirm whether the WiFi is in the 2.4G band.

```
13:46:08.763 -> Connect to WiFi: FYI_2.4G
13:46:09.367 -> ...
13:46:10.332 -> WiFi connected successfully!
13:46:10.368 -> Access address: http://192.168.1.26
13:46:10.368 -> HTTP Server is running
```

The browser will display the following screen. You can click the shortcut below or enter your text in the text box and click "Send".



A notification message will pop up when the message is successfully sent.



The messages you sent will print on the serial monitor.

Reference

server.begin()
This function starts the server to monitor the port.
server.available()
This function is used to detect and return the connected client object.

Chapter 6 TFT

Project 6.1 TFT_Display

Component List

Freenove ESP32 Mini TV 	USB cable x1 

Component knowledge

TFT Display

TFT (Thin Film Transistor) is an electronic component that serves as the foundation for TFT displays, the mainstream display technology in modern laptops and desktop computers. In these displays, each individual liquid crystal pixel is controlled by its own dedicated thin-film transistor embedded directly behind it. This architecture classifies TFT screens as a form of active-matrix LCD (AMLCD) technology.

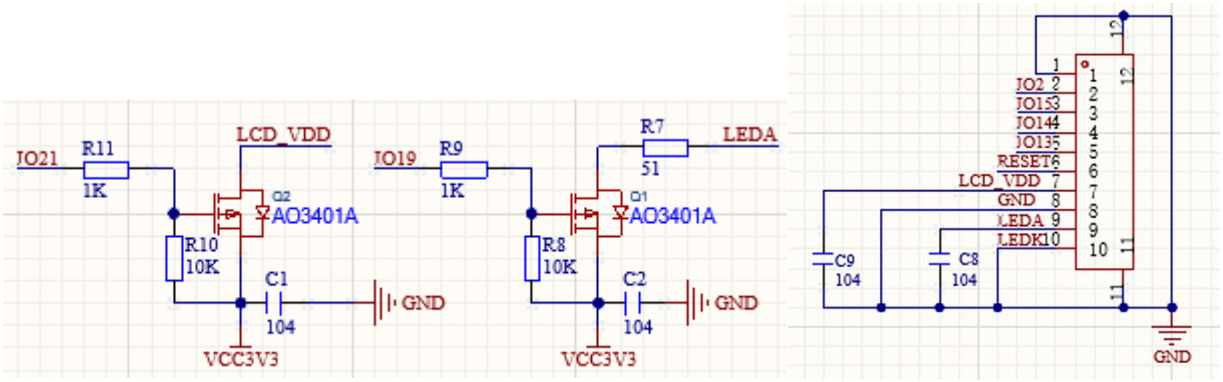
As one of the finest LCD color displays available, TFT screens offer superior performance characteristics including rapid response times, exceptional brightness levels, and outstanding contrast ratios.

Circuit

Connect Freenove ESP32 Mini TV to the computer with USB cable.

Schematic

The schematic diagram for the TFT screen is shown below. It is important to note that the screen's power supply (LCD_VDD) and backlight (LEDA) are driven by two independent GPIO pins, and both are active-low. The complete schematic diagram is located in the same directory as this tutorial document.



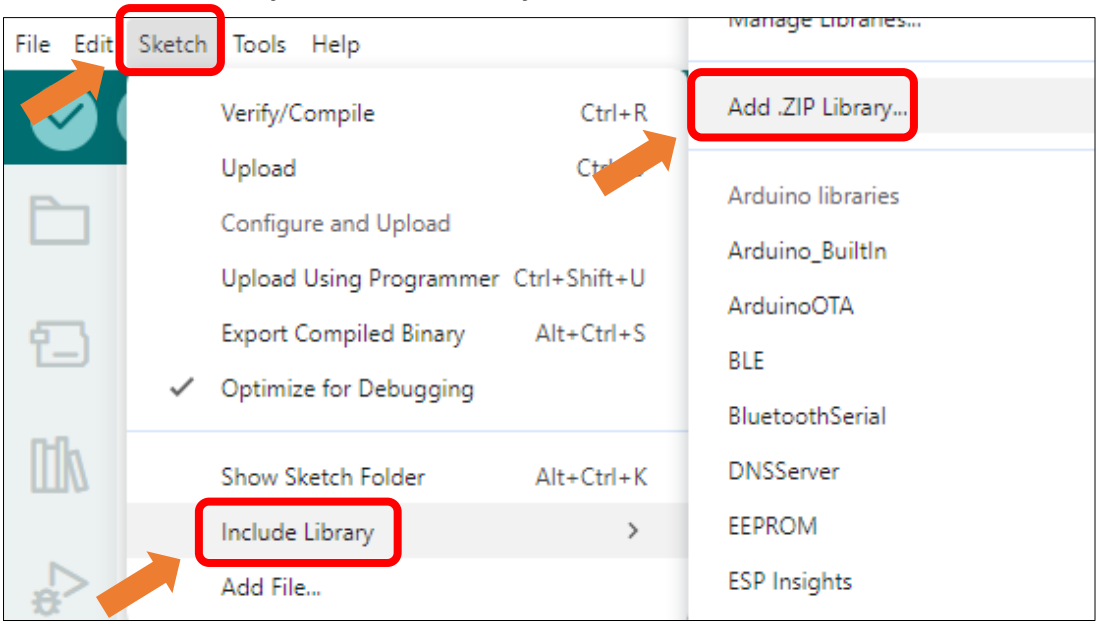
Sketch

Open "**Sketch_06.1_TFT_Clock.ino**" folder under **"Freenove_ESP32_Mini_TV\Sketches"** and double-click "**Sketch_06.1_TFT_Clock.ino**".

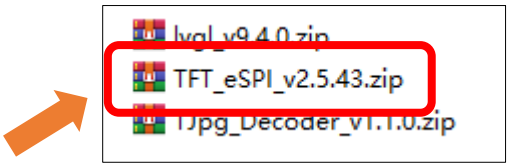
Install Libraries

Caution: It is critical to follow the installation steps below precisely. Our provided library comes pre-configured for the display. Using a version downloaded online or keeping an old one will prevent the screen from working. Please ensure you completely replace any other version with this one to guarantee correct driver operation.

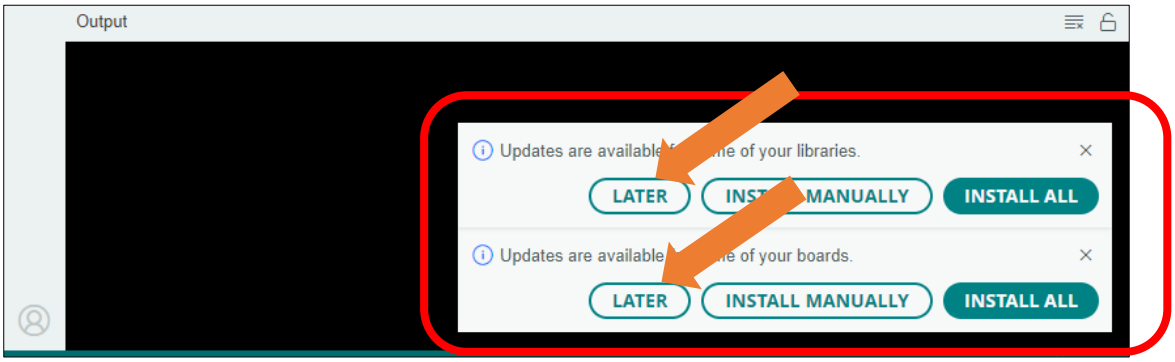
Click **Sketch** -> **Include Library** -> **Add .ZIP Library...**



Install **TFT_eSPI_v2.5.43.zip**



Note: Some libraries are not the latest version. Please do not update them even if it prompts every time you open the IDE. Just click LATER. Otherwise, it may lead to compilation failure.



Sketch_06.1_TFT_Clock

The following is the program code:

```
#include <TFT_eSPI.h>

const int TFT_VCC = 21;

// Initialize TFT instance
TFT_eSPI tft = TFT_eSPI();

uint16_t colors[] = {
    TFT_RED,      // Red
    TFT_GREEN,    // Green
    TFT_BLUE,     // Blue
    TFT_YELLOW,   // Yellow
    TFT_PURPLE    // Purple
};

int colorIndex = 0;

void setup() {
    Serial.begin(115200);

    // Power on and initialize display
    pinMode(TFT_VCC, OUTPUT);
    digitalWrite(TFT_VCC, LOW); // Active Low enable

    tft.begin();
    tft.fillScreen(TFT_BLACK);
```

```
27
28      Serial.println("Test Start...");
29  }
30
31  void loop() {
32      // Fill screen with the current color
33      tft.fillScreen(colors[colorIndex]);
34
35      // Configure text settings
36      tft.setTextColor(TFT_BLACK);       // Set text color
37      tft.setTextDatum(MC_DATUM);        // Set text datum to middle center
38      tft.drawString("TFT Test", 120, 100, 4); // Draw text string
39
40      // Update index
41      colorIndex++;
42      if (colorIndex >= 5) {
43        colorIndex = 0;
44      }
45
46      // Delay for 1 second
47      delay(1000);
48  }
```

Code Explanation

Include the necessary header files.

```
1   #include <TFT_eSPI.h>
```

Declare the TFT screen object.

```
6   TFT_eSPI tft = TFT_eSPI();
```

Enable screen power supply pin.

```
22  pinMode(TFT_VCC, OUTPUT);
23  digitalWrite(TFT_VCC, LOW);
```

Initialize theTFT screen.

```
19  tft.begin();
20  tft.fillScreen(TFT_BLACK);
```

Fill with the color corresponding to the index.

```
33  tft.fillScreen(colors[colorIndex]);
```

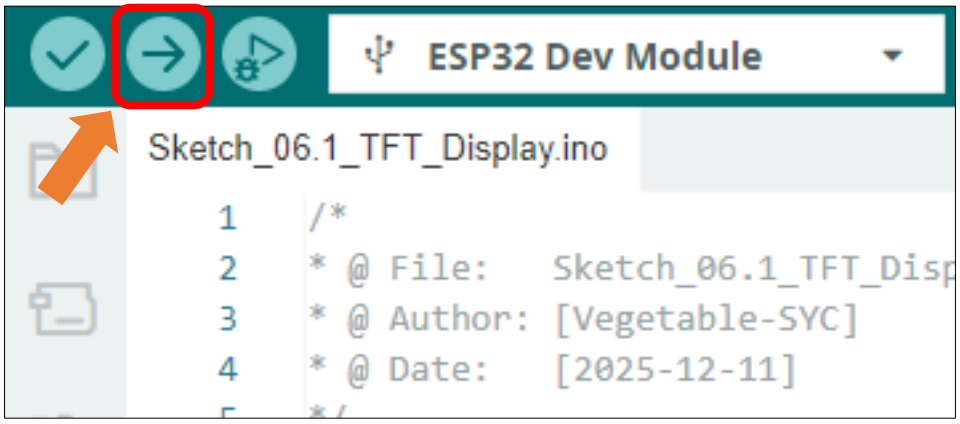
Set text style and draw.

```
38  tft.setTextColor(TFT_BLACK);       // Set text color
39  tft.setTextDatum(MC_DATUM);        // Set text datum to middle center
40  tft.drawString("TFT Test", 120, 100, 4); // Draw text string
```

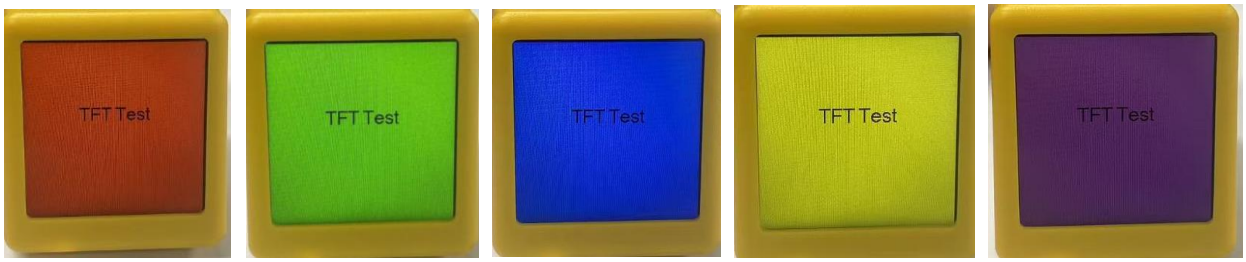
Update the index.

```
41  colorIndex++;
42  if (colorIndex >= 5) {
43      colorIndex = 0;
44  }
```

Click "**Upload**" to upload the code to Freenove ESP32 Mini TV.



The TFT screen will change colors in the order of red -> green -> blue -> yellow -> purple, while displaying the text "TFT Text".



Project 6.2 TFT Picture

Component List

Freenove ESP32 Mini TV 	USB cable x1 

Component knowledge

DMA

DMA, or Direct Memory Access, is a hardware feature that allows peripherals to transfer data to and from memory without needing the CPU to be directly involved, dramatically improving overall system efficiency while minimizing processor workload.

The core mechanism of DMA relies on a dedicated DMA controller taking over data transfer tasks. The CPU only needs to initialize the transfer parameters before offloading the operation, allowing computation and I/O operations to proceed in parallel.

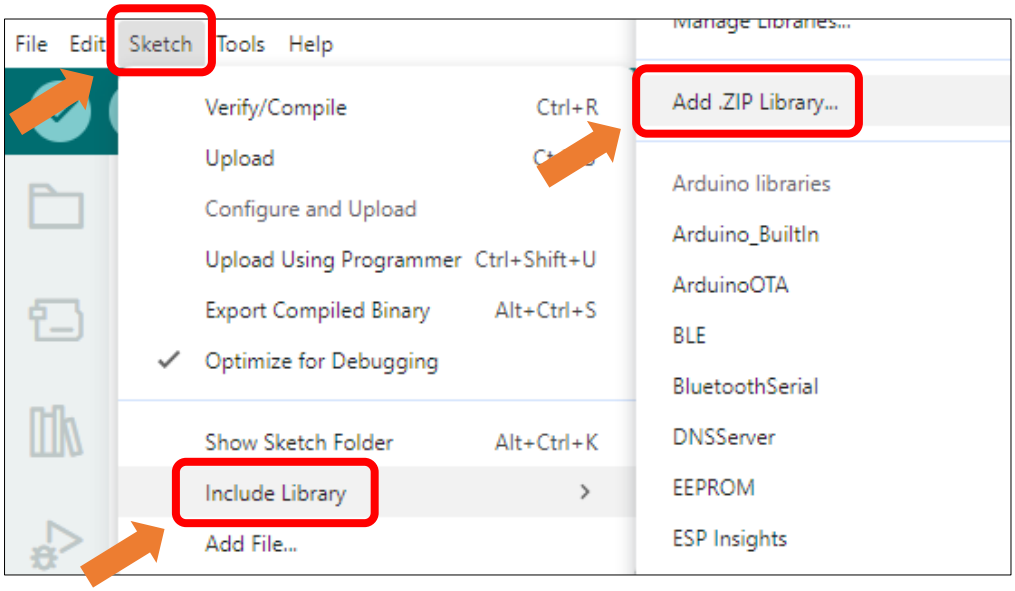
Circuit

Connect Freenove ESP32 Mini TV to the computer with USB cable.

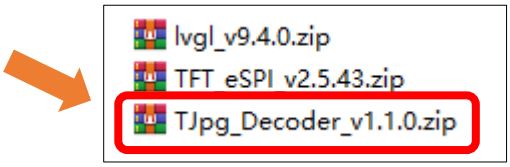
Sketch

Install Libraries

Click **Sketch** -> **Include Library** -> **Add .ZIP Library...**



Install **TJpg_Decoder_v1.1.0.zip**



Open "**Sketch_06.2_TFT_Picture**" folder under **"Freenove_ESP32_Mini_TV\Sketches"** and double-click "**Sketch_06.2_TFT_Picture.ino**".

Sketch_06.2_TFT_Picture

The following is the program code:

```
#include <TFT_eSPI.h>  // TFT display library
#define USE_DMA
#include "image.h"          // Include raw JPEG image data array
#include <TJpg_Decoder.h>  // Include JPEG decoder library

#ifdef USE_DMA
uint16_t dmaBuffer1[16 * 16];  // DMA buffer for 16x16 pixel block
uint16_t dmaBuffer2[16 * 16];  // Second DMA buffer for toggling
uint16_t* dmaBufferPtr = dmaBuffer1;
bool dmaBufferSel = 0;
#endif

const int TFT_VCC = 21;

TFT_eSPI tft = TFT_eSPI();  // Create TFT object instance

```

```
// Callback function to draw decoded JPEG blocks on screen
bool tft_output(int16_t x, int16_t y, uint16_t w, uint16_t h, uint16_t* bitmap) {
    if (y >= tft.height()) return 0;  // Stop decoding if off-screen

#ifdef USE_DMA
    if (dmaBufferSel) dmaBufferPtr = dmaBuffer2;
    else dmaBufferPtr = dmaBuffer1;
    dmaBufferSel = !dmaBufferSel;
    tft.pushImageDMA(x, y, w, h, bitmap, dmaBufferPtr);  // Start DMA transfer
#else
    tft.pushImage(x, y, w, h, bitmap);  // Draw without DMA
#endif

    return 1;  // Continue decoding next block
}

void setup() {
    Serial.begin(115200);
    Serial.println("\n\n Testing TJpg_Decoder library");
    pinMode(TFT_VCC, OUTPUT);
    digitalWrite(TFT_VCC, LOW);
    tft.begin();  // Initialize TFT display
    tft.setTextColor(TFT_WHITE, TFT_BLACK);
    tft.fillScreen(TFT_BLACK);  // Clear screen with black background

#ifdef USE_DMA
    tft.initDMA();  // Setup SPI DMA engine
#endif

    TJpgDec.setJpgScale(1);           // Set scale factor (1=full size)
    tft.setSwapBytes(true);           // Match color byte order
    TJpgDec.setCallback(tft_output);  // Register drawing callback
}

void loop() {
    uint16_t w = 0, h = 0;
    TJpgDec.getJpgSize(&w, &h, img_asset, sizeof(img_asset));  // Get image dimensions
    Serial.print("Width = ");
    Serial.print(w);
    Serial.print(", height = ");
    Serial.print(h);

    uint32_t dt = millis();
    tft.startWrite();
```

```
61      TJpgDec.drawJpg(0, 0, img_asset, sizeof(img_asset));  // Draw the JPEG image
62      tft.endWrite();
63
64      dt = millis() - dt;
65      Serial.print(", dt = ");
66      Serial.print(dt);
67      Serial.println(" ms");
68
69      delay(2000);  // Wait before redraw
70  }
```

Code Explanation

Include necessary header files.

```
1   #include <TFT_eSPI.h>  // TFT display library
    ...
3   #include "image.h"          // Include raw JPEG image data array
4   #include <TJpg_Decoder.h>  // Include JPEG decoder library
```

Configure DMA buffer

```
7   uint16_t dmaBuffer1[16 * 16];  // DMA buffer for 16x16 pixel block
8   uint16_t dmaBuffer2[16 * 16];  // Second DMA buffer for toggling
9   uint16_t* dmaBufferPtr = dmaBuffer1;
10  bool dmaBufferSel = 0;
```

Create TFT object instance.

```
19  TFT_eSPI tft = TFT_eSPI();  // Create TFT object instance
```

JPEG decoding callback function

```
15  // Callback function to draw decoded JPEG blocks on screen
16  bool tft_output(int16_t x, int16_t y, uint16_t w, uint16_t h, uint16_t* bitmap) {
17      if (y >= tft.height()) return 0;  // Stop decoding if off-screen
18
19  #ifdef USE_DMA
20      if (dmaBufferSel) dmaBufferPtr = dmaBuffer2;
21      else dmaBufferPtr = dmaBuffer1;
22      dmaBufferSel = !dmaBufferSel;
23      tft.pushImageDMA(x, y, w, h, bitmap, dmaBufferPtr);  // Start DMA transfer
24  #else
25      tft.pushImage(x, y, w, h, bitmap);  // Draw without DMA
26  #endif
27
28      return 1;  // Continue decoding next block
29  }
```

Get JPG size.

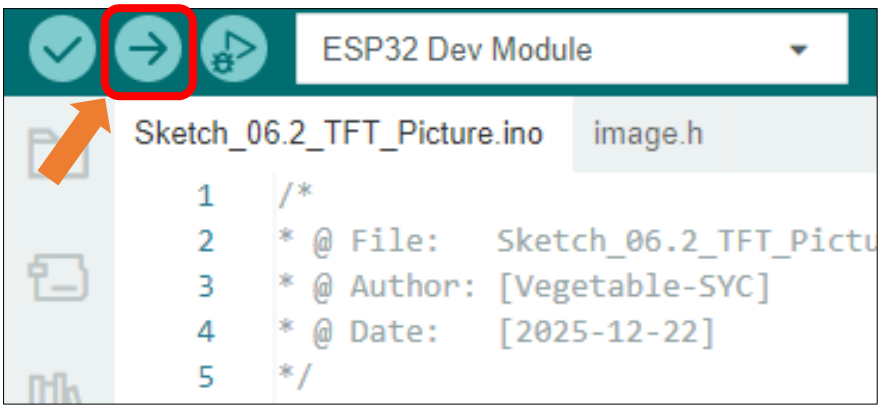
```
46  TJpgDec.getJpgSize(&w, &h, img_asset, sizeof(img_asset));  // Get image dimensions
```

Draw images on the TFT screen.

```
57  tft.startWrite();
58  TJpgDec.drawJpg(0, 0, img_asset, sizeof(img_asset));  // Draw the JPEG image
```

```
59    tft.endWrite();
```

Click "**Upload**" to upload the code to Freenove ESP32 Mini TV



After the code is uploaded, an image will be displayed on the TFT screen

Custom image display

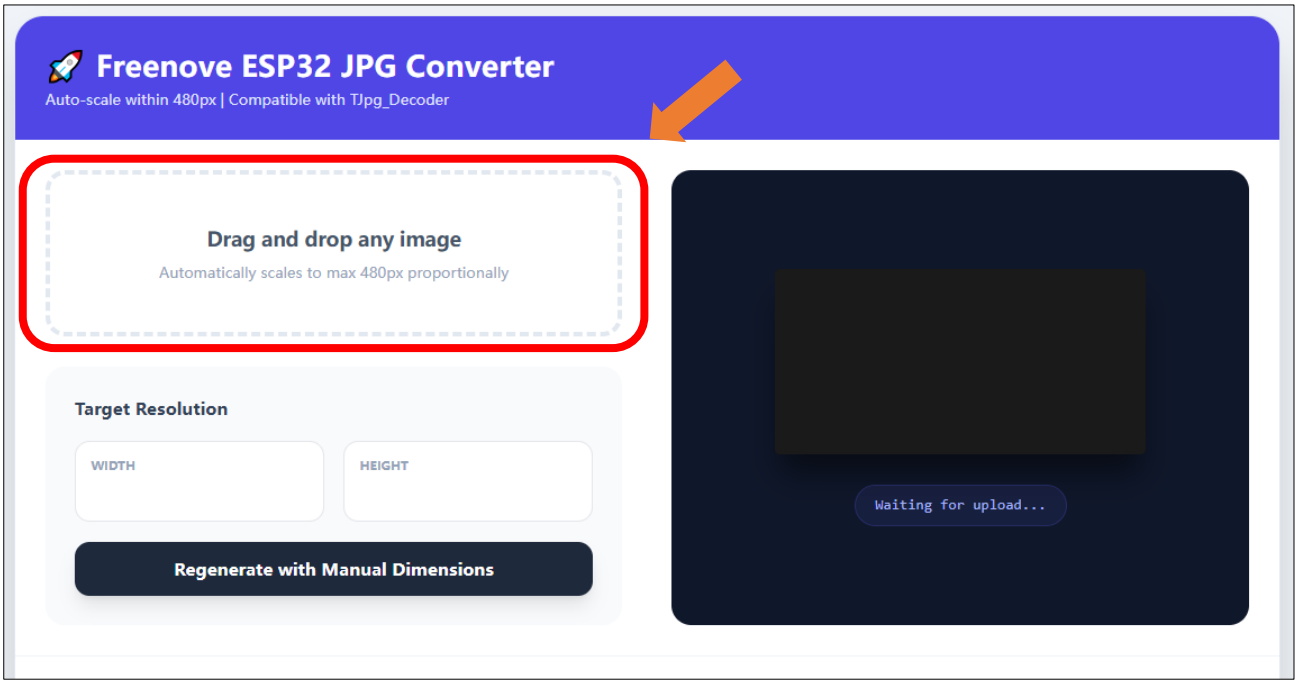
You can customize the image displayed on the display according to your personal preferences.

First, open **Freenove_ESP32_Mini_TV \Sketch\ Sketch_06.2_TFT_Picture\TFT_img.html**

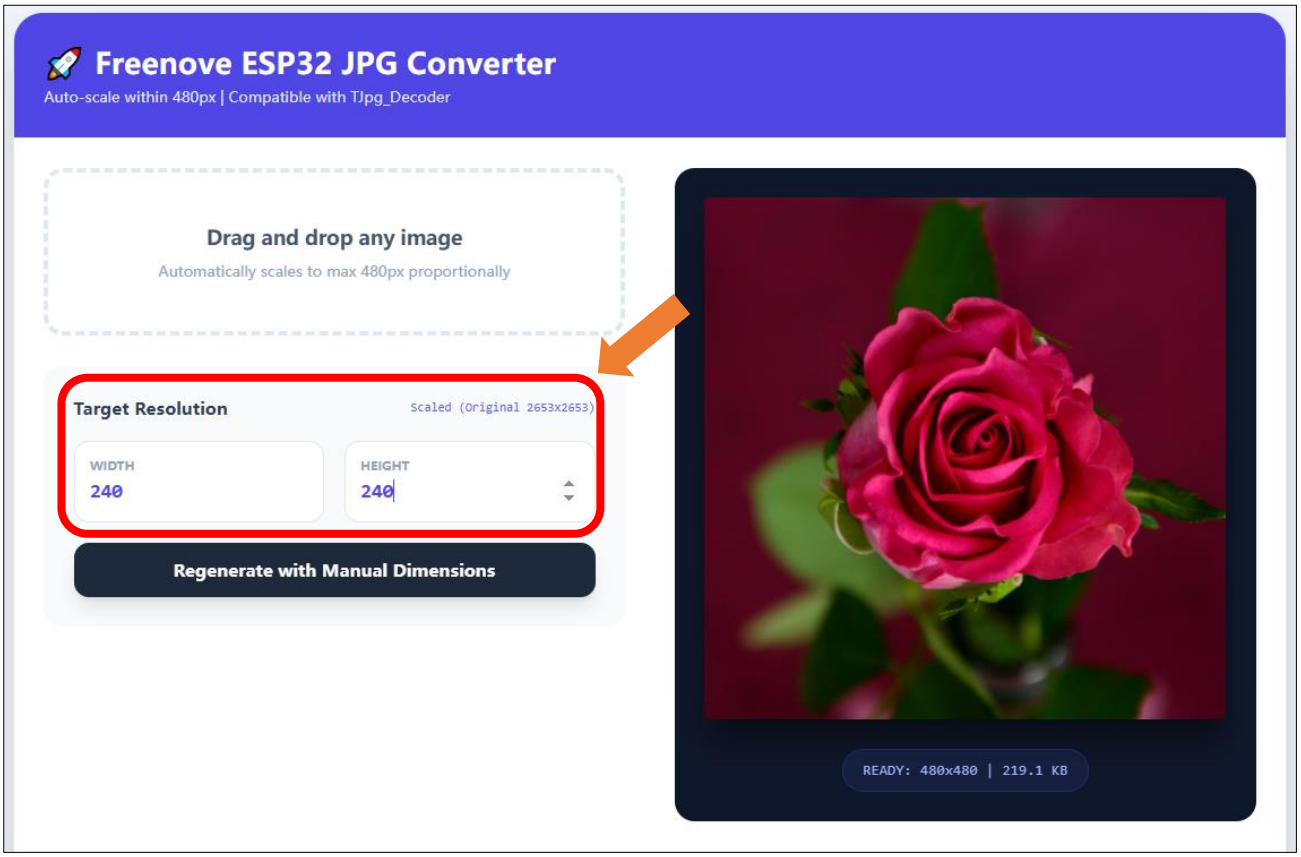
For optimal compatibility, please use Microsoft Edge or Google Chrome. You can open the file by dragging it directly into the browser window, or by right clicking it and selecting"Open with".

If you have any concerns, please feel free to contact us via support@freenove.com

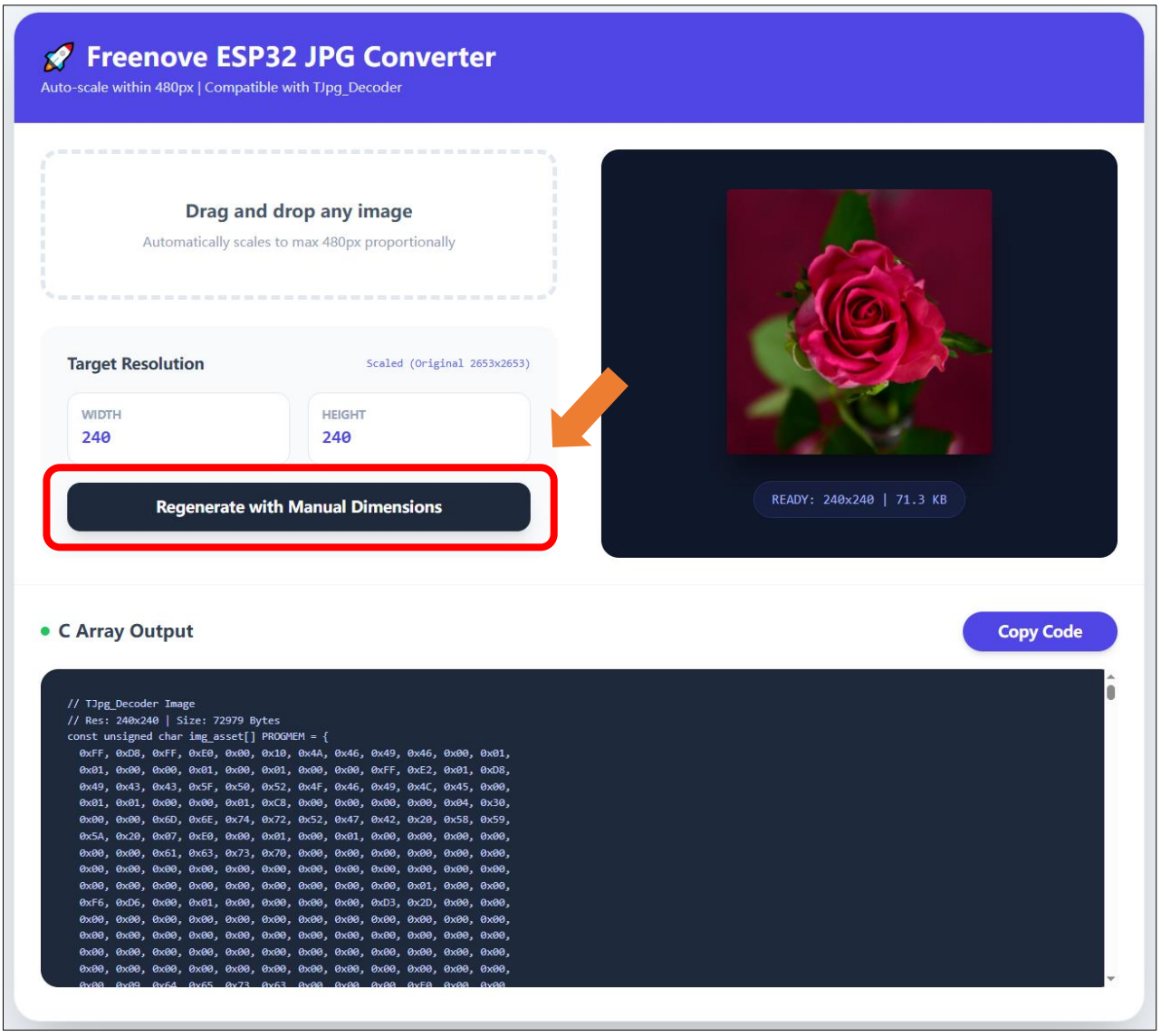
Click the "**Drag and drop any image**" area to select an image file, or directly drag and drop a file onto it.



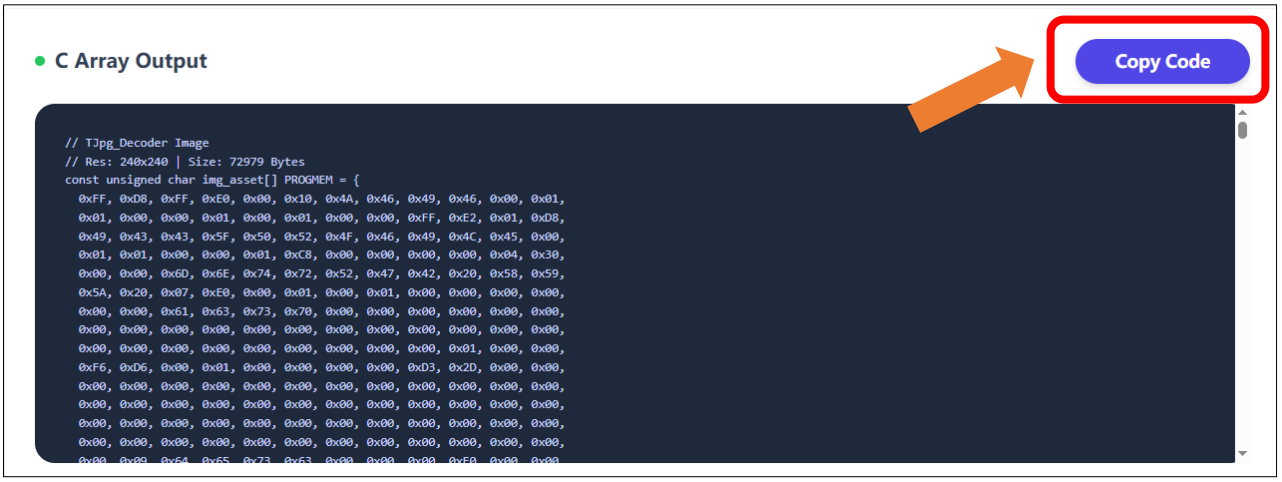
Set the resolution to **240x240.**



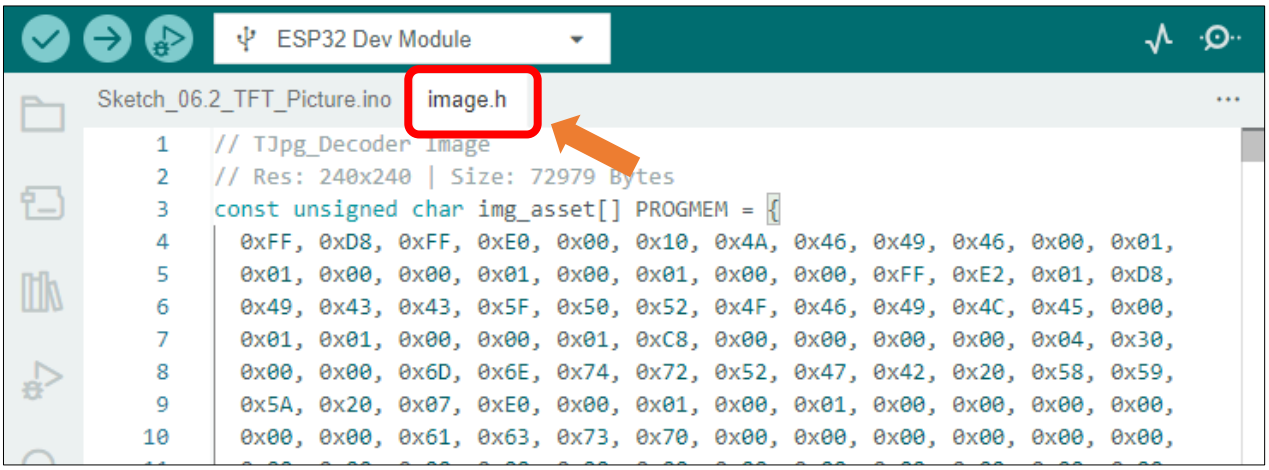
Click **"Regenerate with Manual Dimensions"** to generate arrays.



Click the **"Copy Code"** button to copy the arrays.



Replace all content in **image.h** with the copied arrays.



Upload the code and the custom image will be displayed.

Project 6.3 TFT Clock

Having explored key concepts such as EEPROM, Wi-Fi, capacitive touch, and TFT display in previous chapters, we will now synthesize this knowledge to create a practical and visually appealing classic project: a WiFi clock. Upon completing this chapter, you will be able to customize the interface to your preference.

Component List

Freenove ESP32 Mini TV 	USB cable x1 

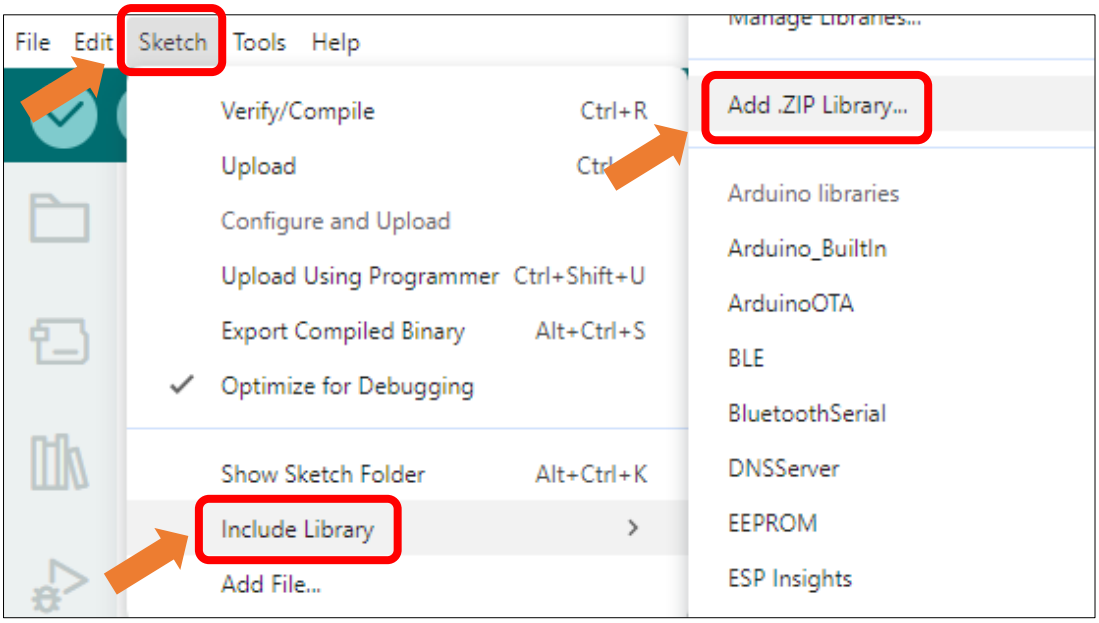
Circuit

Connect Freenove ESP32 Mini TV to the computer with USB cable.

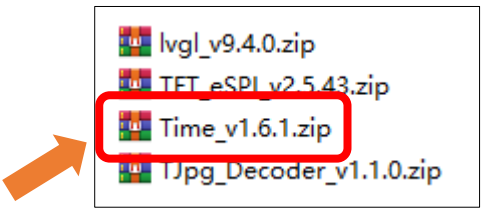
Sketch

Install Libraries

Click **Sketch** -> **Include Library** -> **Add .ZIP Library...**



Install **Time_v1.6.1.zip**



Open"**Sketch_06.3_TFT_Clock**" folder under **"Freenove_ESP32_Mini_TV\Sketch"** and double-click "**Sketch_06.3_TFT_Clock.ino**".

Sketch_06.3_TFT_Clock

The following is the program code:

```
#include <TFT_eSPI.h>
#include <WiFi.h>
#include <WiFiUdp.h>
#include <EEPROM.h>
#include <TimeLib.h>
#include <DNSServer.h>

/* ---------------------------------------------------------------------- */
/*                              Configuration                             */
/* ---------------------------------------------------------------------- */

// Pin Definitions
#define TFT_VCC       21
```

```
#define TOUCH_PIN      32
#define EEPROM_SIZE    1024
#define AP_SSID        "ESP32-Clock"
#define AP_IP          IPAddress(192, 168, 4, 1)

// UI Geometry
#define CENTER_X       120
#define CENTER_Y       120
#define RADIUS         100

#define COLOR_BG       0x0000
#define COLOR_SCALE    0xFFFF
#define COLOR_TEXT     0xFFFF
#define COLOR_HOUR     0xF800
#define COLOR_MIN      0x07E0
#define COLOR_SEC      0xFFE0
#define COLOR_CENTER   0xFFFF
#define COLOR_RING     0xAD55

// Global Constants
const char* ntpServer      = "pool.ntp.org";
const int timeZone         = 8;
const char* AUTHOR_NAME    = "Freenove";
const int TOUCH_THRESHOLD = 80;
const int PRESS_TIME       = 3000;

// Object Instances
TFT_eSPI tft = TFT_eSPI();
WiFiServer server(80);
DNSServer;
WiFiUDP ntpUDP;

// State Variables
bool timeSynced     = false;
bool isAPMode       = false;
int currentUI       = 1;
int lastUI          = -1;
bool isFirstDraw    = true;
int wifiScanCount   = 0;
float lastHAngle=0, lastMAngle=0, lastSAngle=0;

// Touch Interaction Variables
uint32_t Touch_Time = 0;
int Touch_data = 0, Touch_old_data = 0;
```

```
bool LONG_PRESS_FLAG = false;

// HTML/CSS for Web Configuration Portal
const char* CSS_BODY = R"raw(
<style>
    body {
        background: #202020;
        color: #fff;
        font-family: sans-serif;
        margin: 0;
        padding: 0;
    }
    #header {
        background: #333;
        padding: 15px;
        text-align: center;
    }
    h1 {
        margin: 0;
        font-size: 20px;
    }
    .list {
        max-height: 400px;
        overflow-y: auto;
    }
    .item {
        padding: 15px;
        border-bottom: 1px solid #444;
        display: flex;
        justify-content: space-between;
        cursor: pointer;
    }
    .item:active {
        background: #444;
    }
    .ssid {
        font-size: 16px;
        font-weight: bold;
    }
    .meta {
        font-size: 12px;
        color: #aaa;
    }
    .form {
```

```
            padding: 20px;
            background: #303030;
            text-align: center;
        }
        input {
            width: 100%;
            padding: 12px;
            margin: 10px 0;
            border: none;
            border-radius: 4px;
            font-size: 16px !important;
            box-sizing: border-box;
        }
        button {
            width: 100%;
            padding: 15px;
            background: #007BFF;
            color: #fff;
            border: none;
            border-radius: 4px;
            font-size: 18px;
            font-weight: bold;
            cursor: pointer;
        }
        a {
            color: #007BFF;
            text-decoration: none;
            display: block;
            margin-top: 10px;
            text-align: center;
        }
    </style>

    <script>
        function c(s) {
            document.getElementById('s').value = s;
            document.getElementById('p').focus();
        }
    </script>
    )raw";

    /* ---------------------------------------------------------------------- */
    /*                               Main Program                             */
    /* ---------------------------------------------------------------------- */
```

```
146
147 void setup() {
148     Serial.begin(115200);
149
150     // Initialize NVS storage
151     if (!EEPROM.begin(EEPROM_SIZE)) Serial.println("EEPROM Init Failed");
152
153     // Initialize Display
154     pinMode(TFT_VCC, OUTPUT);
155     digitalWrite(TFT_VCC, LOW);
156     tft.begin();
157     tft.setRotation(0);
158     tft.fillScreen(COLOR_BG);
159
160     // Initialize Touch and calibrate baseline
161     touchSetCycles(0xf000, 0x1000);
162     Touch_old_data = touchRead(TOUCH_PIN);
163
164     // Connection management
165     connectToWiFi();
166     ntpUDP.begin(123);
167 }
168
169 void loop() {
170     if (isAPMode) {
171       dnsServer.processNextRequest(); // Handle DNS queries for Captive Portal
172       handleWebServer();              // Process config portal requests
173     } else if (WiFi.isConnected()) {
174       static unsigned long lastNtpUpdate = 0;
175       // Periodic NTP synchronization
176       if (millis() - lastNtpUpdate > 3600000 || !timeSynced) {
177         syncNtpTime(timeZone);
178         lastNtpUpdate = millis();
179       }
180     }
181     checkTouchInput();
182     // Rendering logic only active after time synchronization
183     if (timeSynced) renderUI();
184     delay(5);
185 }
```

Code Explanation

Include the header file.

```
1 #include <TFT_eSPI.h>
2 #include <WiFi.h>
```

```
3  #include <WiFiUdp.h>
4  #include <EEPROM.h>
5  #include <TimeLib.h>
6  #include <DNSServer.h>
```

Preset the required variables the time zone and author name can be modified here.

```
35  const char* ntpServer     = "pool.ntp.org";
36  const int timeZone        = 8;
37  const char* AUTHOR_NAME   = "Freenove";
38  const int TOUCH_THRESHOLD = 80;
39  const int PRESS_TIME      = 3000;
```

Webpage interface design during network configuration

```
62  const char* CSS_BODY = R"raw(
63  <style>
...  ...
142  )raw";
```

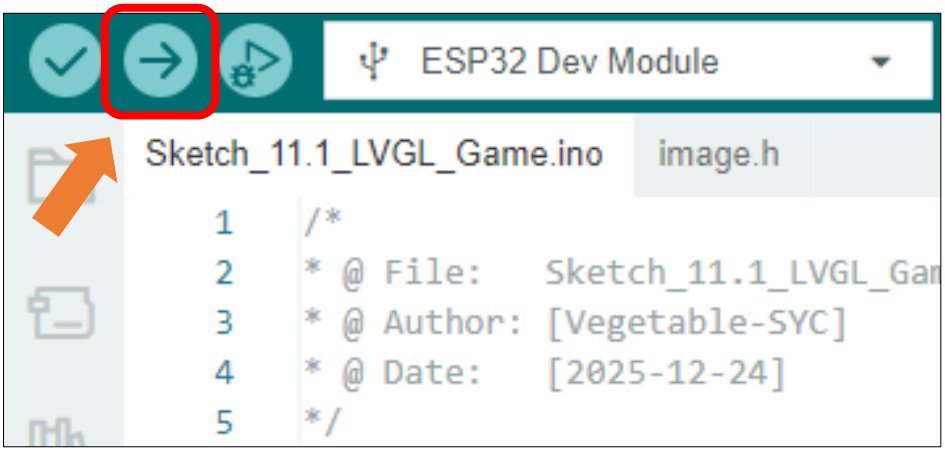
Initialize the screen.

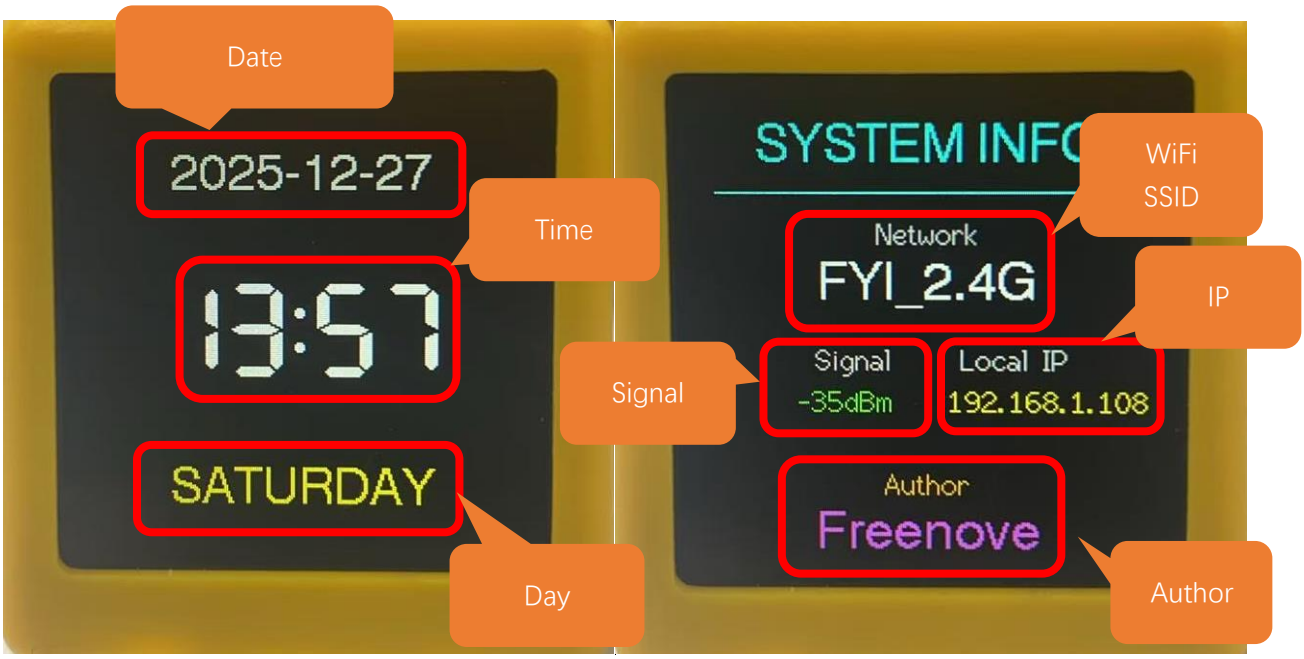
```
155  pinMode(TFT_VCC, OUTPUT);
156  digitalWrite(TFT_VCC, LOW);
157  tft.begin();
158  tft.setRotation(0);
159  tft.fillScreen(COLOR_BG);
```

Initialize the capative touch.

```
155  touchSetCycles(0xf000, 0x1000);
156  Touch_old_data = touchRead(TOUCH_PIN);
```

Click "**Upload**" to upload the code to Freenove ESP32 Mini TV




Date
2025-12-27
Time
13:57
SATURDAY
Day
SYSTEM INFO
WiFi
SSID
Network
FYI_2.4G
IP
Signal
Signal
-35dBm
Local IP
192.168.1.108
Author
Freenove
Author


Clock Face
Time
13:57:57

How to Use

Upon first boot, the device requires network configuration. The WiFi credentials will be saved to EEPROM for automatic connection on subsequent startups. To reset or change the network, simply long-press the capacitive touch area for 3 seconds. This will clear the stored configuration and initiate the setup process again.

1. On first power-up, wait until the following interface appears.

2. Connect to ESP32-Clock.

3. Wait for the network configuration page to appear.

Note: If the page does not load automatically, please open a browser and navigate to http://192.168.4.1.

4. Select your WiFi network in the configuration page and enter the password. Click

Important note: The WiFi network must be 2.4GHz; otherwise, the connection will fail.

5. the following screen will be displayed. You can tap the capacitive touch area to switch between interfaces.

Chapter 7 LVGL

Project 7.1 LVGL

Component List

Freenove ESP32 Mini TV 	USB cable x1 

Component knowledge

LVGL is a widely-used embedded GUI library that is implemented in pure C, making it highly portable and performant. It offers rich features and content, supporting both display and input devices such as touchscreens and keyboards.

	Features supported by LVGL
1	Powerful building blocks such as buttons, charts, lists, sliders, images, and more.
2	Advanced graphics with animation, anti-aliasing, opacity, and smooth scrolling.
3	Various input devices, such as touchpads, mice, keyboards, encoders, and more.
4	Multiple languages with UTF-8 encoding.
5	Multiple display types, including TFT and monochrome displays.
6	Fully customizable graphical elements.
7	LVGL can be used independently of any microcontroller or display hardware.
8	Highly extensible and can be configured to use very little memory (e.g. 64 kB of flash and 16 kB of RAM)
9	It can be used with or without an operating system, and supports external memory and GPUs as optional features.
10	Single-frame buffer operation, even with advanced graphics effects.
11	Written in C language to achieve maximum compatibility (compatible with C++ as well).
12	LVGL has a simulator that allows for embedded GUI design on a PC without any embedded hardware.
13	Resources to help developers quickly get started with the library, including tutorials, examples, and themes.
14	A wide range of resources.

Circuit

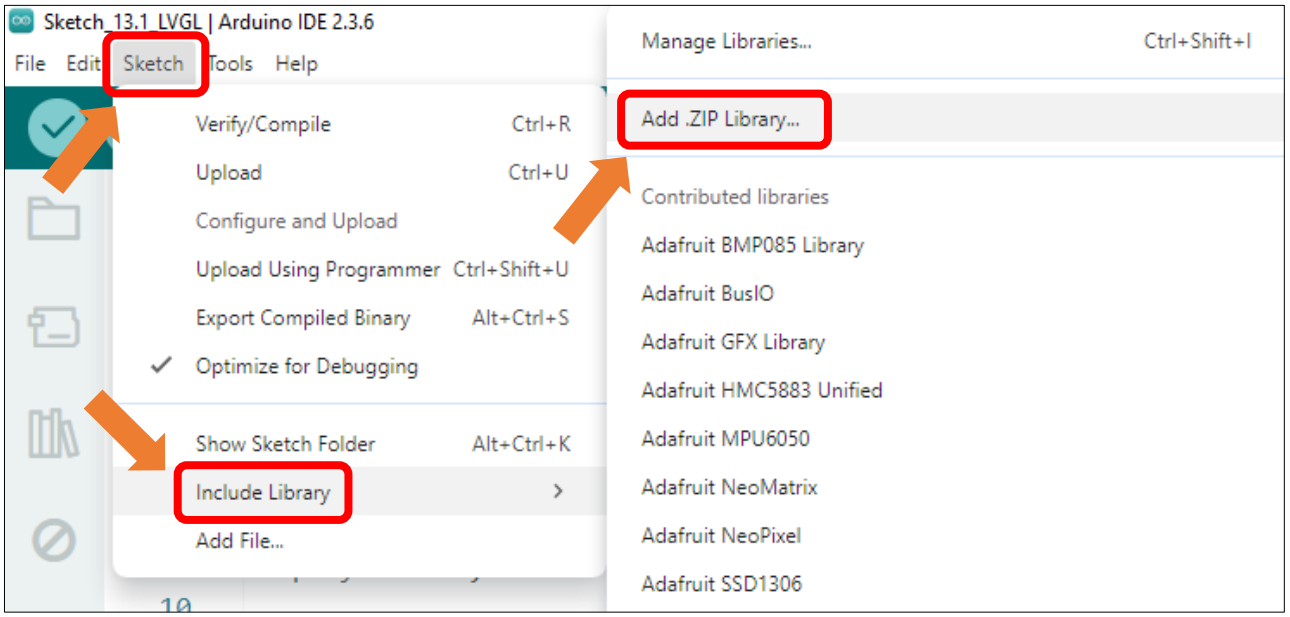
Connect Freenove ESP32 Mini TV to the computer with USB cable.

Sketch

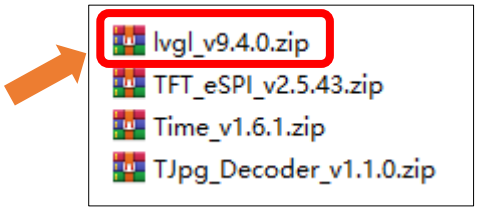
Install Libraries

Caution: It is critical to follow the installation steps below precisely to install the LVGL library. Our provided library comes pre-configured for the display. Using a version downloaded online or keeping an old one will prevent the screen from working. Please ensure you completely replace any other version with this one to guarantee correct driver operation.

Click **Sketch** -> **Include Library** -> **Add .ZIP Library...**



Install **lvgl_v9.4.0.zip**



Open"**Sketch_07.1_LVGL**" folder under **"Freenove_ESP32_Mini_TV\Sketch"** and double-click "**Sketch_07.1_LVGL.ino**".

Sketch_07.1_LVGL

The following is the program code:

```
#include <lvgl.h>
#include <TFT_eSPI.h>

/* Display resolution configuration */
#define SCREEN_WIDTH  240
#define SCREEN_HEIGHT 240

/* Hardware and driver initialization */
const int TFT_VCC = 21;
TFT_eSPI tft = TFT_eSPI();

/* LVGL display handles and draw buffers */
static lv_display_t * disp;
static uint8_t buf[SCREEN_WIDTH * SCREEN_HEIGHT / 10 * 2];

/**
  * Flush callback: Transfers pixel data from LVGL to the TFT controller
  */
void my_disp_flush(lv_display_t * disp, const lv_area_t * area, uint8_t * px_map) {
    uint32_t w = (area->x2 - area->x1 + 1);
    uint32_t h = (area->y2 - area->y1 + 1);

    tft.startWrite();
    tft.setAddrWindow(area->x1, area->y1, w, h);
    tft.pushColors((uint16_t *)px_map, w * h, true);
    tft.endWrite();

    lv_display_flush_ready(disp); // Indicate flushing finished
}

void setup() {
    Serial.begin(115200);

```

```
34      /* Initialize physical display */
35      tft.begin();
36
37      /* Initialize LVGL core library */
38      lv_init();
39
40      /* Setup display abstraction layer */
41      disp = lv_display_create(SCREEN_WIDTH, SCREEN_HEIGHT);
42      lv_display_set_flush_cb(disp, my_disp_flush);
43      lv_display_set_buffers(disp, buf, NULL, sizeof(buf), LV_DISPLAY_RENDER_MODE_PARTIAL);
44
45      /* UI initialization and version logging */
46      String LVGL_Arduino = "Hello Arduino! ";
47      LVGL_Arduino += String('V') + lv_version_major() + "." + lv_version_minor() + "." +
48  lv_version_patch();
49
50      Serial.println(LVGL_Arduino);
51      Serial.println("System Ready: LVGL initialized");
52
53      /* Create and center-align a label widget */
54      lv_obj_t *label = lv_label_create(lv_screen_active());
55      lv_label_set_text(label, LVGL_Arduino.c_str());
56      lv_obj_align(label, LV_ALIGN_CENTER, 0, 0);
57
58      /* Enable display power */
59      pinMode(TFT_VCC, OUTPUT);
60      digitalWrite(TFT_VCC, LOW);
61  }
62
63  void loop() {
64      /* Update library tick and process pending UI tasks */
65      lv_tick_inc(5);
66      lv_timer_handler();
67      delay(5);
68  }
```

Code Explanation

Include the header file.

```
1  #include <lvgl.h>
2  #include <TFT_eSPI.h>
```

Set the screen resolution.

```
5  #define SCREEN_WIDTH  240
6  #define SCREEN_HEIGHT 240
```

Initialize the screen.

```
35  tft.begin();
```

Initialize LVGL..

```
38  lv_init();
```

Create a text label and center it.

```
54  lv_obj_t *label = lv_label_create(lv_screen_active());
55  lv_label_set_text(label, LVGL_Arduino.c_str());
56  lv_obj_align(label, LV_ALIGN_CENTER, 0, 0);
```

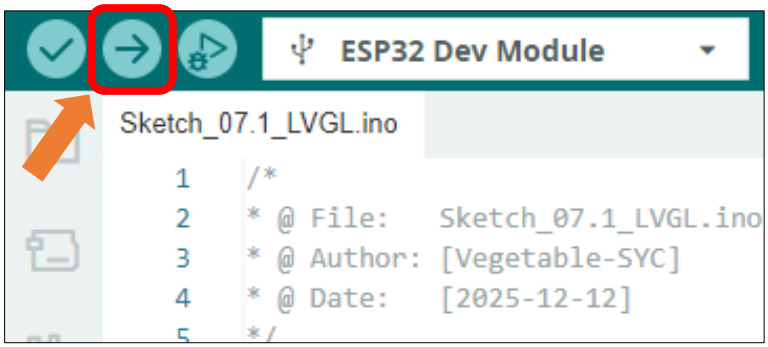
Enable the display's power supply pin.

```
59  pinMode(TFT_VCC, OUTPUT);
60  digitalWrite(TFT_VCC, LOW);
```

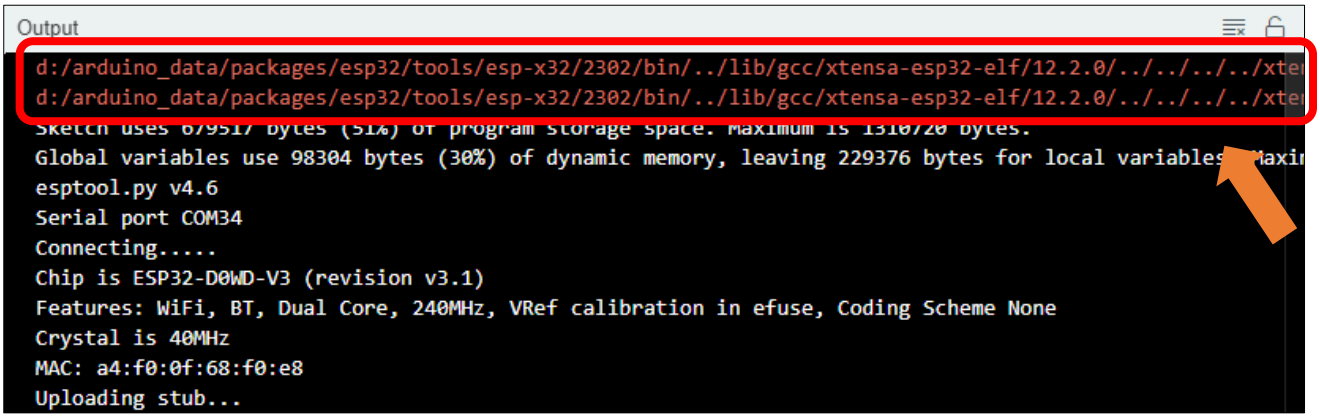
Handles UI tasks in a continuous loop.

```
65  lv_tick_inc(5);
66  lv_timer_handler();
67  delay(5);
```

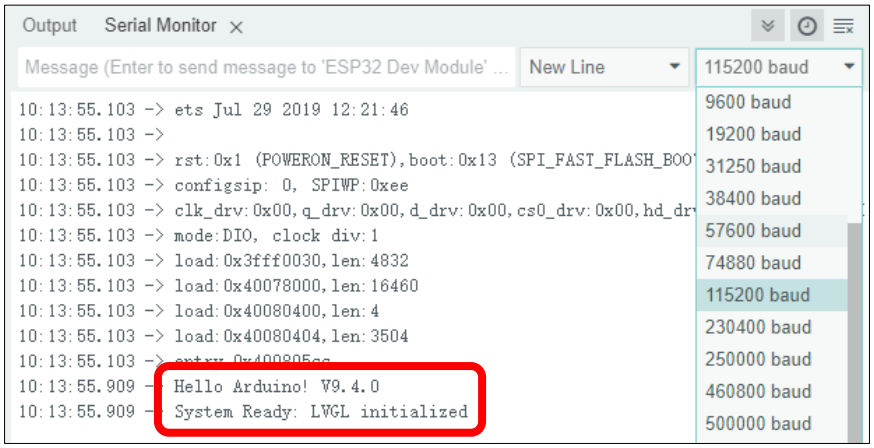
Click "**Upload**" to upload the code to Freenove ESP32 Mini TV. Set the baud rate to 115200.



Note: The following warning messages may appear in the console during compilation. They do not affect code execution and can be safely ignored.



Successful initialization information will be displayed on the serial monitor.



You will see the text reading "**Hello Arduino! V94.0**" on the display.

Reference

`lv_display_t * lv_display_create(int32_t hor_res, int32_t ver_res);`
This function is designed to create a device object with specified dimensions, define the canvas size, and return a handle to the object.
`lv_obj_t * lv_label_create(lv_obj_t * parent);`
This function creates a text label component.
`void lv_label_set_text(lv_obj_t * obj, const char * text);`
This function sets the static text to be displayed by the label control.

Chapter 8 Lvgl Touch

Project 8.1 Lvgl Touch

Having learned about capacitive touch in previous chapters, we will now combine it with LVGL to create more intuitive interactive effects on the screen.

Component List

Freenove ESP32 Mini TV 	USB cable x1 

Circuit

Connect Freenove ESP32 Mini TV to the computer with USB cable.

Sketch

Open“**Sketch_08.1_LVGL_Touch**” folder under **“Freenove_ESP32_Mini_TV\Sketch”** and double-click “**Sketch_08.1_LVGL_Touch.ino**”.

Sketch_08.1_LVGL_Touch

The following is the program code:

```
#include <lvgl.h>
#include <TFT_eSPI.h>

/* --- Configuration Parameters --- */
const int TOUCH_PIN = 32;          // Capacitive touch GPIO
const int TFT_VCC = 21;            // Power control pin
const int TOUCH_THRESHOLD = 80;    // Sensitivity threshold
const int PRESS_Time = 3000;       // Long press threshold (3000ms)
int touch_base_val = 0;            // Initial baseline value

#define SCREEN_WIDTH  240
#define SCREEN_HEIGHT 240

TFT_eSPI tft = TFT_eSPI();
static uint8_t buf[SCREEN_WIDTH * SCREEN_HEIGHT / 10 * 2]; // LVGL draw buffer

lv_obj_t * label_short;
lv_obj_t * label_long;
int short_press_num = 0;
int long_press_num = 0;

/* --- 1. Display Flush Callback --- */
void my_disp_flush(lv_display_t * disp, const lv_area_t * area, uint8_t * px_map) {
    uint32_t w = (area->x2 - area->x1 + 1);
    uint32_t h = (area->y2 - area->y1 + 1);
    tft.startWrite();
    tft.setAddrWindow(area->x1, area->y1, w, h);
    tft.pushColors((uint16_t *)px_map, w * h, true);
    tft.endWrite();
    lv_display_flush_ready(disp); // Inform LVGL flushing is complete
}

/* --- 2. Input Device Read Callback --- */
void my_touchpad_read(lv_indev_t * indev, lv_indev_data_t * data) {
    int val = touchRead(TOUCH_PIN);

    // Determine state based on baseline comparison
```

```
38      if (touch_base_val - val > TOUCH_THRESHOLD) {
39          data->state = LV_INDEV_STATE_PRESSED;
40      } else {
41          data->state = LV_INDEV_STATE_RELEASED;
42      }
43      data->btn_id = 0; // Use the first button mapping
44  }
45
46  /* --- 3. UI Event Callback --- */
47  void btn_event_cb(lv_event_t * e) {
48      lv_event_code_t code = lv_event_get_code(e);
49
50      if(code == LV_EVENT_SHORT_CLICKED) {
51          // Triggered upon releasing after a short press
52          short_press_num++;
53          lv_label_set_text_fmt(label_short, "Short Press: %d", short_press_num);
54          Serial.println("Short Press Detected");
55      }
56      else if(code == LV_EVENT_LONG_PRESSED) {
57          // Triggered upon releasing after a long press
58          long_press_num++;
59          lv_label_set_text_fmt(label_long, "Long Press: %d", long_press_num);
60          Serial.println("Long Press Detected (3s)");
61      }
62  }
63
64  void setup() {
65      Serial.begin(115200);
66
67      // Initialize capacitive touch hardware
68      touchSetCycles(0xf000, 0x1000);
69      touch_base_val = touchRead(TOUCH_PIN);
70
71      tft.begin();
72      lv_init();
73
74      // Register LVGL display driver
75      lv_display_t * disp = lv_display_create(SCREEN_WIDTH, SCREEN_HEIGHT);
76      lv_display_set_flush_cb(disp, my_disp_flush);
77      lv_display_set_buffers(disp, buf, NULL, sizeof(buf), LV_DISPLAY_RENDER_MODE_PARTIAL);
78
79      // Register Input Device (Button Type for single touch point)
80      lv_indev_t * indev = lv_indev_create();
81      lv_indev_set_type(indev, LV_INDEV_TYPE_BUTTON);
```

```
82      lv_indev_set_read_cb(indev, my_touchpad_read);
83
84      // Set LVGL long press detection threshold
85      lv_indev_set_long_press_time(indev, PRESS_Time);
86
87      // Map the hardware touch sensor to a virtual screen coordinate
88      static const lv_point_t btn_points[] = { {120, 150} }; // Coordinates within button area
89      lv_indev_set_button_points(indev, btn_points);
90
91      // Create User Interface - Interactive Button
92      lv_obj_t * btn = lv_button_create(lv_screen_active());
93      lv_obj_set_size(btn, 180, 80);
94      lv_obj_align(btn, LV_ALIGN_CENTER, 0, 30);
95
96      // Listen for all interaction events
97      lv_obj_add_event_cb(btn, btn_event_cb, LV_EVENT_ALL, NULL);
98
99      lv_obj_t * btn_label = lv_label_create(btn);
100     lv_label_set_text(btn_label, "Touch Pin 32");
101     lv_obj_center(btn_label);
102
103     // Create Feedback Labels
104     label_short = lv_label_create(lv_screen_active());
105     lv_label_set_text(label_short, "Short Press: 0");
106     lv_obj_align(label_short, LV_ALIGN_TOP_MID, 0, 40);
107
108     label_long = lv_label_create(lv_screen_active());
109     lv_label_set_text(label_long, "Long Press: 0");
110     lv_obj_align(label_long, LV_ALIGN_TOP_MID, 0, 70);
111
112     // Power up display
113     pinMode(TFT_VCC, OUTPUT);
114     digitalWrite(TFT_VCC, LOW);
115 }
116
117 void loop() {
118     // Process UI tasks
119     lv_timer_handler();
120
121     // Handle LVGL internal tick timing
122     static uint32_t last_tick = 0;
123     uint32_t current_ms = millis();
124     lv_tick_inc(current_ms - last_tick);
125     last_tick = current_ms;
```

```
126
127       delay(5);
128   }
```

Code Explanation

Include the necessary header files.

```
1   #include <lvgl.h>
2   #include <TFT_eSPI.h>
```

Preset basic parameters.

```
5   const int TOUCH_PIN = 32;           // Capacitive touch GPIO
6   const int TFT_VCC = 21;             // Power control pin
7   const int TOUCH_THRESHOLD = 80;     // Sensitivity threshold
8   const int PRESS_Time = 3000;          // Long press threshold (3000ms)
```

Define the screen resolution.

```
11  #define SCREEN_WIDTH  240
12  #define SCREEN_HEIGHT 240
```

Initialize and calinrate the touch.

```
68  touchSetCycles(0xf000, 0x1000);
69  touch_base_val = touchRead(TOUCH_PIN);
```

Initialize the screen and LVGL.

```
71  tft.begin();
72  lv_init();
```

Draw the UI.

```
74  // Register LVGL display driver
75  lv_display_t * disp = lv_display_create(SCREEN_WIDTH, SCREEN_HEIGHT);
76  lv_display_set_flush_cb(disp, my_disp_flush);
77  lv_display_set_buffers(disp, buf, NULL, sizeof(buf), LV_DISPLAY_RENDER_MODE_PARTIAL);
78
79  // Register Input Device (Button Type for single touch point)
80  lv_indev_t * indev = lv_indev_create();
81  lv_indev_set_type(indev, LV_INDEV_TYPE_BUTTON);
82  lv_indev_set_read_cb(indev, my_touchpad_read);
83
84  // Set LVGL long press detection threshold
85  lv_indev_set_long_press_time(indev, PRESS_Time);
86
87  // Map the hardware touch sensor to a virtual screen coordinate
88  static const lv_point_t btn_points[] = { {120, 150} }; // Coordinates within button area
89  lv_indev_set_button_points(indev, btn_points);
90
91  // Create User Interface - Interactive Button
92  lv_obj_t * btn = lv_button_create(lv_screen_active());
93  lv_obj_set_size(btn, 180, 80);
94  lv_obj_align(btn, LV_ALIGN_CENTER, 0, 30);
95
```

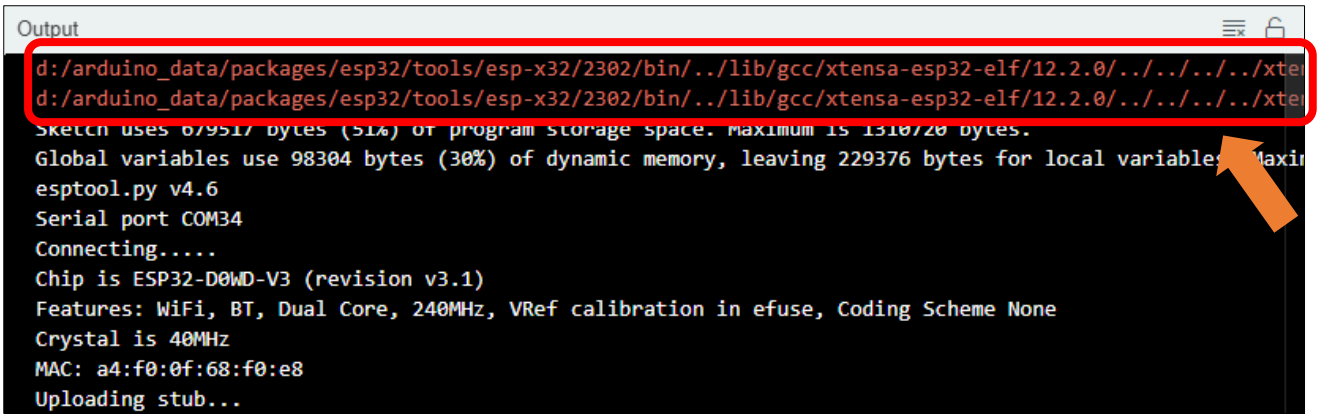
```
96  // Listen for all interaction events
97  lv_obj_add_event_cb(btn, btn_event_cb, LV_EVENT_ALL, NULL);
98
99  lv_obj_t * btn_label = lv_label_create(btn);
100 lv_label_set_text(btn_label, "Touch Pin 32");
101 lv_obj_center(btn_label);
102
103 // Create Feedback Labels
104 label_short = lv_label_create(lv_screen_active());
105 lv_label_set_text(label_short, "Short Press: 0");
106 lv_obj_align(label_short, LV_ALIGN_TOP_MID, 0, 40);
107
108 label_long = lv_label_create(lv_screen_active());
109 lv_label_set_text(label_long, "Long Press: 0");
110 lv_obj_align(label_long, LV_ALIGN_TOP_MID, 0, 70);
```

Enable the screen power supply pin.

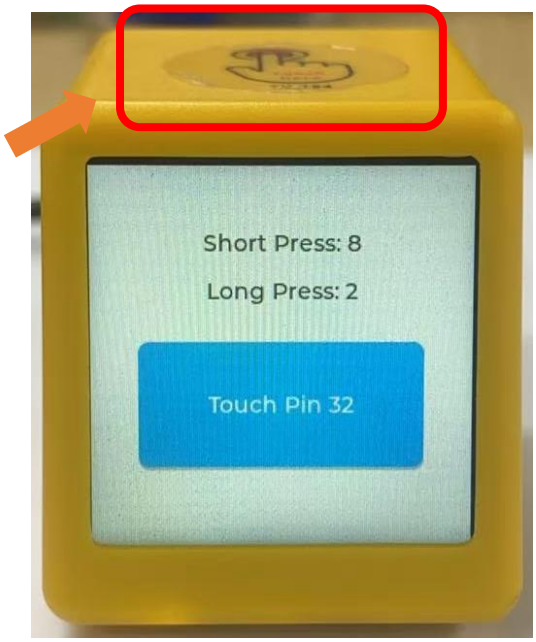
```
113 pinMode(TFT_VCC, OUTPUT);
114 digitalWrite(TFT_VCC, LOW);
```

Click "**Upload**" to upload the code to Freenove ESP32 Mini TV. Set the baud rate to 115200.

Note: The following warning messages may appear in the console during compilation. They do not affect code execution and can be safely ignored.



When touching the capacitive touch area of the Freenode ESP32 Mini TV with a finger, a touch event lasting longer than 3 seconds is considered a long press; otherwise, it is a short press. The on-screen button displays a click animation when pressed.



Reference

`lv_obj_t * lv_button_create(lv_obj_t * parent);`
This function is used to create a button component.
`lv_event_dsc_t * lv_obj_add_event_cb(lv_obj_t * obj, lv_event_cb_t event_cb, lv_event_code_t filter, void * user_data);`
This function is used to bind an event callback function to an object, enabling it to listen for and respond to specific interactive behaviors.

Chapter 9 Lvgl Picture

Project 9.1 Lvgl Picture

Component List

Freenove ESP32 Mini TV 	USB cable x1 

Circuit

Connect Freenove ESP32 Mini TV to the computer with USB cable.

Sketch

Open "**Sketch_09.1_LVGL_Picture**" folder under **"Freenove_ESP32_Mini_TV\Sketch"** and double-click "**Sketch_09.1_LVGL_Picture.ino**".

Sketch_09.1_LVGL_Picture

The following is the program code:

```
1  #include <lvgl.h>
2  #include <TFT_eSPI.h>
```

```
#include <image.h>

/* Display and asset dimensions */
#define IMG_W  240
#define IMG_H  240
#define SCREEN_WIDTH  240
#define SCREEN_HEIGHT 240

/* Hardware pin configuration */
const int TFT_VCC = 21;
TFT_eSPI tft = TFT_eSPI();

/* LVGL rendering buffer */
static uint8_t draw_buf[SCREEN_WIDTH * SCREEN_HEIGHT / 10 * 2];

/* LVGL Image Descriptor mapping the C-array asset */
const lv_image_dsc_t lv_img = {
    .header = {
        .magic = LV_IMAGE_HEADER_MAGIC,
        .cf = LV_COLOR_FORMAT_RGB565, // Color format: 16-bit RGB
        .flags = 0,
        .w = IMG_W,
        .h = IMG_H,
        .stride = IMG_W * 2,          // Bytes per line
        .reserved_2 = 0
    },
    .data_size = IMG_W * IMG_H * 2,
    .data = img_asset,                // Pointer to external image data
};

/**
  * Display flushing callback: Transfers pixels from LVGL to hardware
  */
void my_disp_flush(lv_display_t * disp, const lv_area_t * area, uint8_t * px_map) {
    uint32_t w = (area->x2 - area->x1 + 1);
    uint32_t h = (area->y2 - area->y1 + 1);

    tft.startWrite();
    tft.setAddrWindow(area->x1, area->y1, w, h);
    tft.pushColors((uint16_t *)px_map, w * h, true);
    tft.endWrite();

    lv_display_flush_ready(disp); // Inform library flushing is complete
}
```

```
47
48  void setup() {
49      Serial.begin(115200);
50
51      /* Hardware and library initialization */
52      tft.begin();
53      lv_init();
54
55      /* Create and configure display abstraction layer */
56      lv_display_t * disp = lv_display_create(SCREEN_WIDTH, SCREEN_HEIGHT);
57      lv_display_set_flush_cb(disp, my_disp_flush);
58      lv_display_set_buffers(disp, draw_buf, NULL, sizeof(draw_buf),
59  LV_DISPLAY_RENDER_MODE_PARTIAL);
60
61      /* Initialize UI components: Image display */
62      lv_obj_t * img_obj = lv_image_create(lv_screen_active());
63      lv_image_set_src(img_obj, &lv_img);
64      lv_obj_center(img_obj);
65
66      /* Status label configuration */
67      lv_obj_t * label = lv_label_create(lv_screen_active());
68      lv_label_set_text(label, "LVGL v9 Image Display");
69      lv_obj_align(label, LV_ALIGN_BOTTOM_MID, 0, -20);
70      lv_obj_set_style_text_color(label, lv_color_white(), 0);
71
72      /* Enable display power */
73      pinMode(TFT_VCC, OUTPUT);
74      digitalWrite(TFT_VCC, LOW);
75
76      Serial.println("System initialized: LVGL asset display ready.");
77  }
78
79  void loop() {
80      /* Process timer events and system task handlers */
81      lv_timer_handler();
82
83      /* Advance internal library tick counts */
84      static uint32_t last_tick = 0;
85      uint32_t current_ms = millis();
86      lv_tick_inc(current_ms - last_tick);
87      last_tick = current_ms;
88
89      delay(5);
90  }
```

Code Explanation

Include the necessary header files

```
1  #include <lvgl.h>
2  #include <TFT_eSPI.h>
3  #include <image.h>
```

Define image size

```
6  #define IMG_W  240
7  #define IMG_H  240
```

Define screen resolution

```
8  #define SCREEN_WIDTH  240
9  #define SCREEN_HEIGHT 240
```

Set the baudrate to 115200

```
8  Serial.begin(115200);
```

Drawing pictures

```
61 /* Initialize UI components: Image display */
62 lv_obj_t * img_obj = lv_image_create(lv_screen_active());
63 lv_image_set_src(img_obj, &lv_img);
64 lv_obj_center(img_obj);
65
66 /* Status label configuration */
67 lv_obj_t * label = lv_label_create(lv_screen_active());
68 lv_label_set_text(label, "LVGL v9 Image Display");
69 lv_obj_align(label, LV_ALIGN_BOTTOM_MID, 0, -20);
70 lv_obj_set_style_text_color(label, lv_color_white(), 0);
```

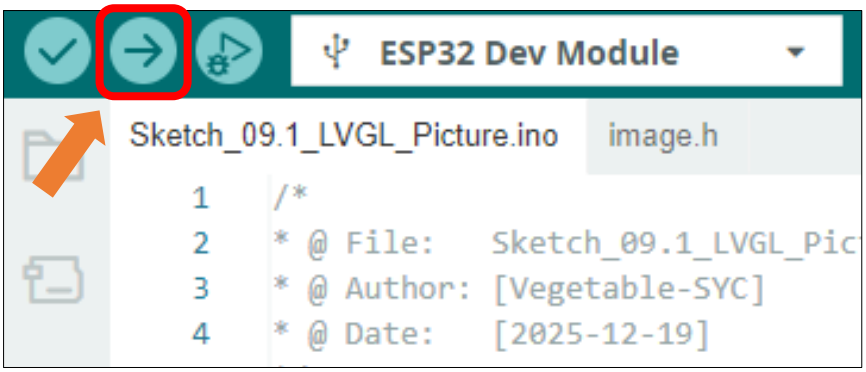
Enable screen power supply pin

```
72 /* Enable display power */
73 pinMode(TFT_VCC, OUTPUT);
74 digitalWrite(TFT_VCC, LOW);
```

Loop through UI tasks

```
80 /* Process timer events and system task handlers */
81 lv_timer_handler();
82
83 /* Advance internal library tick counts */
84 static uint32_t last_tick = 0;
85 uint32_t current_ms = millis();
86 lv_tick_inc(current_ms - last_tick);
87 last_tick = current_ms;
88
89 delay(5);
```

Click "**Upload**" to upload the code to Freenove ESP32 Mini TV



After the code is uploaded, an image will be displayed on the TFT screen

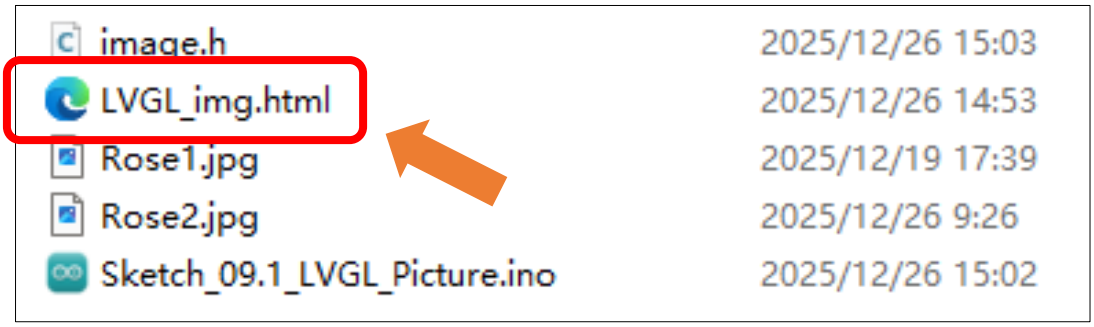
Reference

`lv_obj_t * lv_image_create(lv_obj_t * parent);`
This function is used to create an image component.
`void lv_image_set_src(lv_obj_t * obj, const void * src);`
This function is used to provide a data source for the image component, thereby displaying the specific image content.

Custom image display

You can customize the image displayed on the display according to your personal preferences.

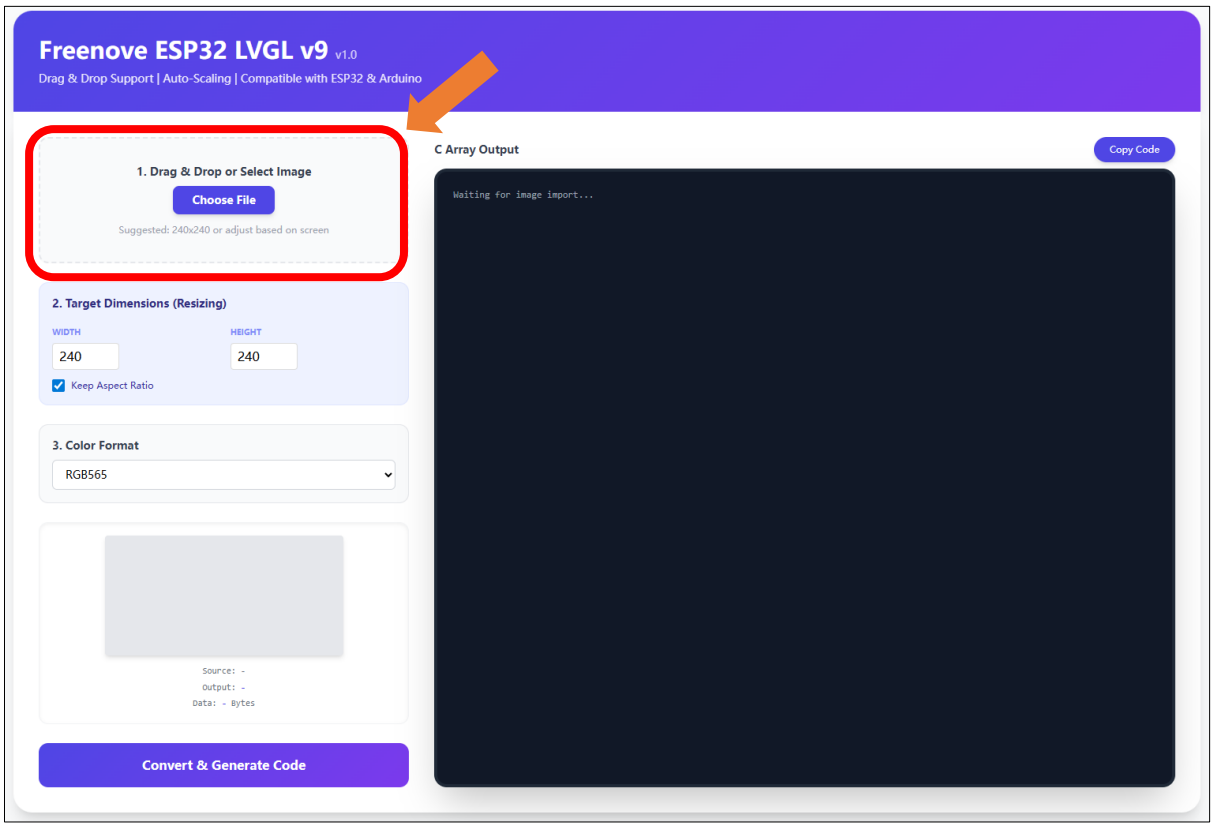
First, open **Freenove_ESP32_Mini_TV\Sketch\Sketch_09.1_LVGL_Picture\LVGL_img.html**



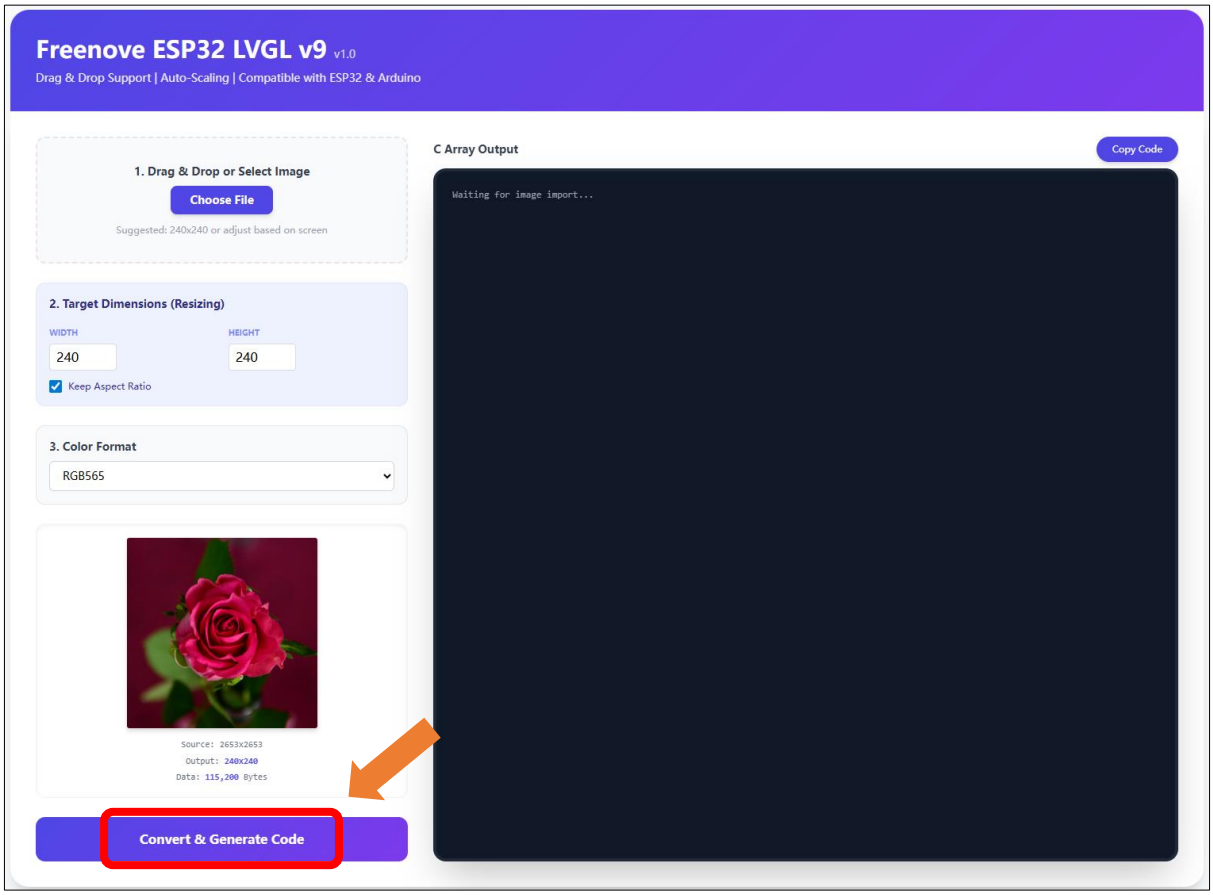
For optimal compatibility, please use Microsoft Edge or Google Chrome. You can open the file by dragging it directly into the browser window, or by right clicking it and selecting"Open with".

If you have any concerns, please feel free to contact us via support@freenove.com

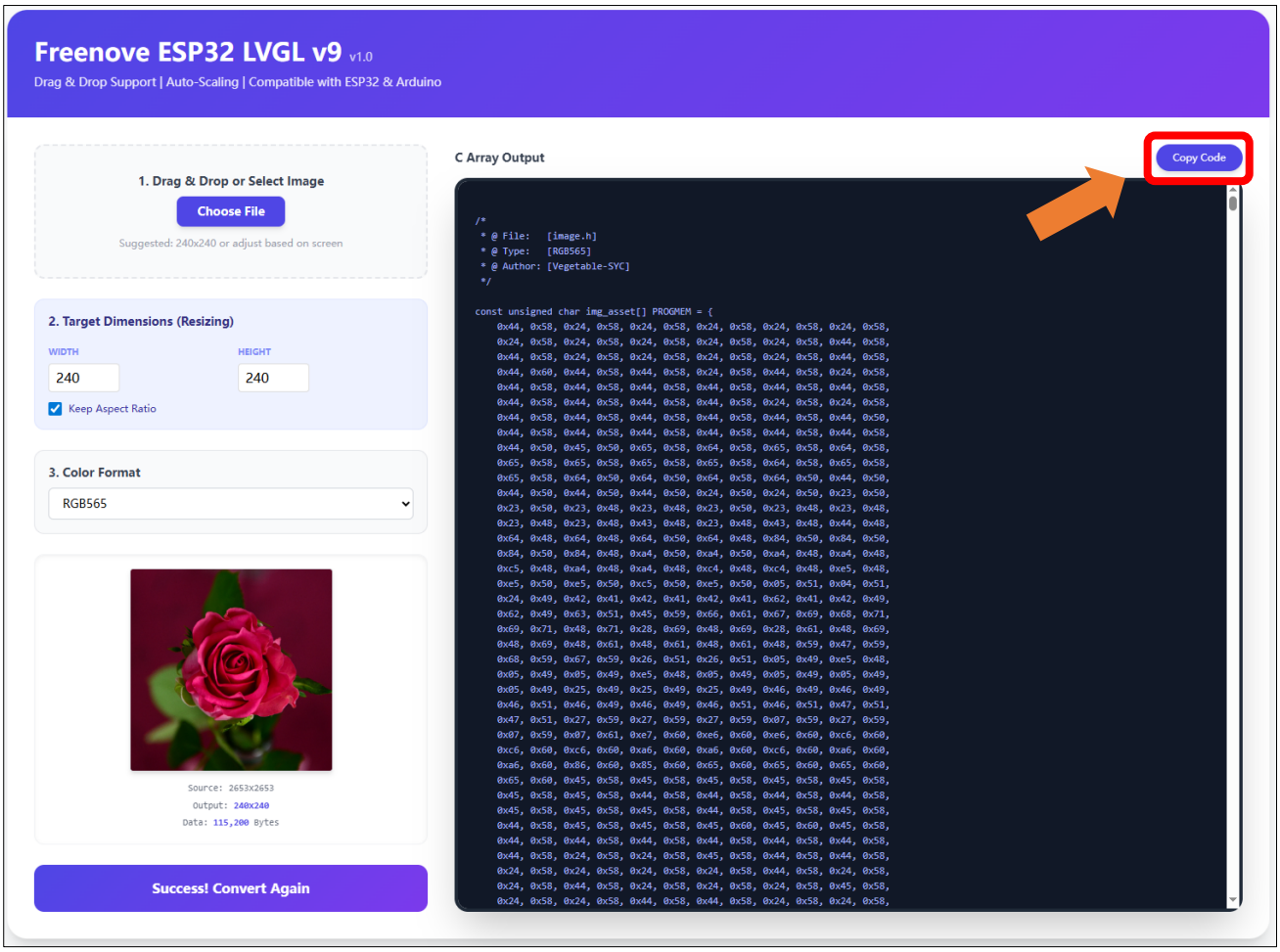
Click the "**Drag and drop any image**" area to select an image file, or directly drag and drop a file onto it.



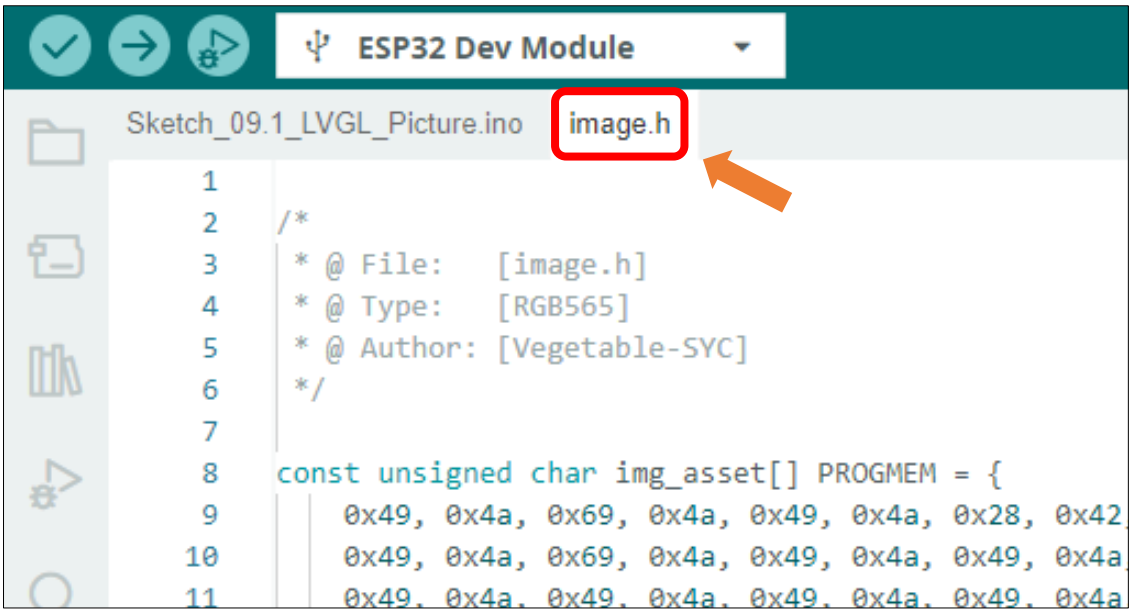
With the default settings, click "**Convert & Generate Code**" to generate the array.



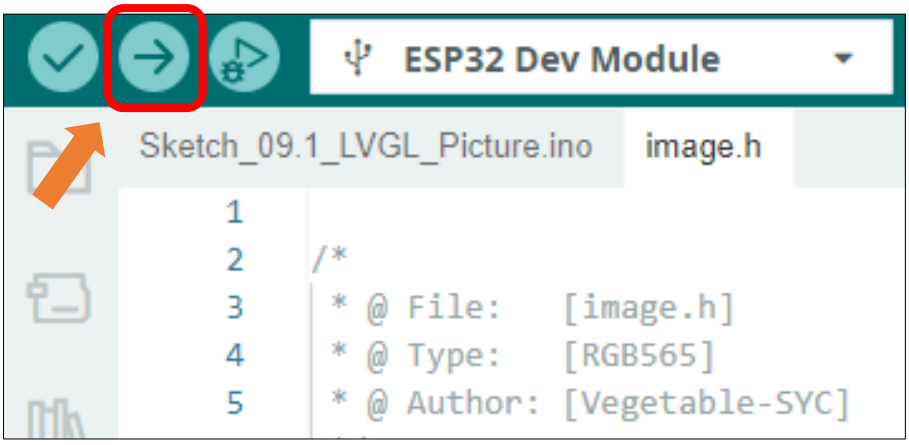
Click "**Copy Code**" to copy the arrays.



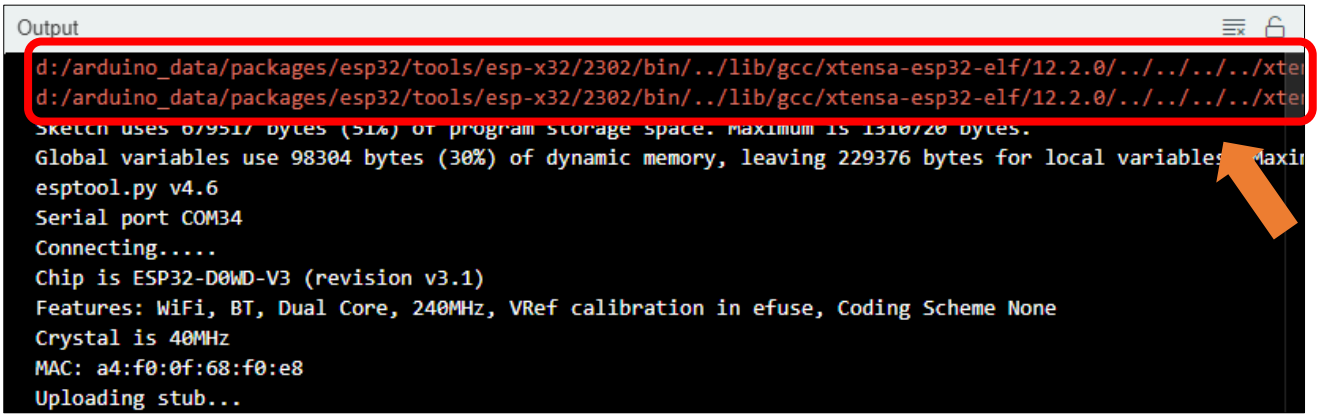
Replace all the contect in the **image.h** file with the copied arrays.



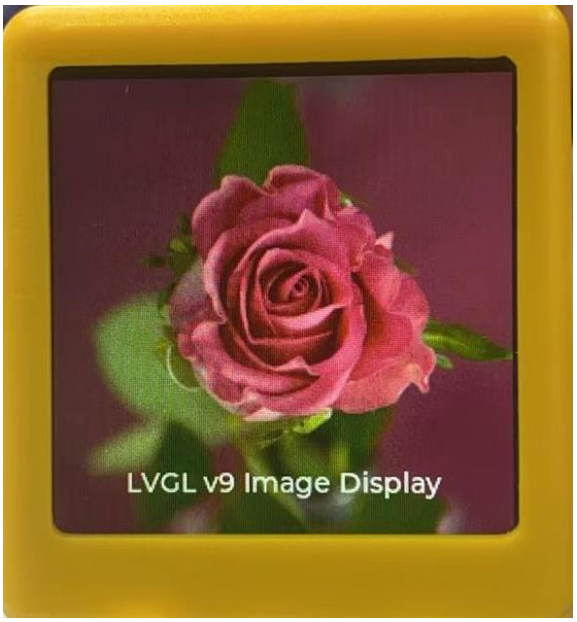
Click "**Upload**" to upload the code to Freenove ESP32 Mini TV. Set the baud rate to 115200.



Note: The following warning messages may appear in the console during compilation. They do not affect code execution and can be safely ignored.



After uploading the code, the custom image will be displayed on the screen.



Chapter 10 Lvgl Timer

Project 10.1 Lvgl Timer

Component List

Freenove ESP32 Mini TV 	USB cable x1 

Circuit

Connect Freenove ESP32 Mini TV to the computer with USB cable.

Sketch

Open“**Sketch_10.1_LVGL_Timer**” folder under **“Freenove_ESP32_Mini_TV\Sketch”** and double-click “**Sketch_10.1_LVGL_Timer.ino**”.

Sketch_10.1_LVGL_Timer

The following is the program code:

```
1  #include <lvgl.h>
2  #include <TFT_eSPI.h>
```

```
3
4   /* Hardware mapping & Constants */
5   const int TOUCH_PIN = 32;          // Capacitive touch GPIO
6   const int TFT_VCC = 21;            // Display power control
7   const int TOUCH_THRESHOLD = 80;    // Sensitivity calibration
8   const int PRESS_TIME = 3000;       // Reset threshold (ms)
9   int touch_base_val = 0;            // Auto-calibrated baseline
10
11  /* Display specifications */
12  #define SCREEN_WIDTH  240
13  #define SCREEN_HEIGHT 240
14
15  TFT_eSPI tft = TFT_eSPI();
16
17  /* Memory-aligned draw buffer for optimized DMA/rendering */
18  static uint32_t buf_aligned[ (SCREEN_WIDTH * SCREEN_HEIGHT / 10 * 2) / 4 + 1 ];
19  uint8_t * draw_buf_ptr = (uint8_t *)buf_aligned;
20
21  /* Global state machine & variables */
22  bool is_running = false;
23  uint32_t elapsed_time = 0;
24  uint32_t last_millis = 0;
25
26  /* UI Object handles */
27  lv_obj_t * label_time;
28  lv_obj_t * label_status;
29  lv_obj_t * arc_deco;
30
31  /**
32    * Flush callback: Bridges LVGL software rendering to hardware SPI
33    */
34  void my_disp_flush(lv_display_t * disp, const lv_area_t * area, uint8_t * px_map) {
35      uint32_t w = (area->x2 - area->x1 + 1);
36      uint32_t h = (area->y2 - area->y1 + 1);
37      tft.startWrite();
38      tft.setAddrWindow(area->x1, area->y1, w, h);
39      tft.pushColors((uint16_t *)px_map, w * h, true);
40      tft.endWrite();
41      lv_display_flush_ready(disp);
42  }
43
44  /**
45    * Input device callback: Logic for capacitive touch state acquisition
46    */
```

```
void my_touchpad_read(lv_indev_t * indev, lv_indev_data_t * data) {
    int val = touchRead(TOUCH_PIN);
    if (touch_base_val - val > TOUCH_THRESHOLD) {
        data->state = LV_INDEV_STATE_PRESSED;
    } else {
        data->state = LV_INDEV_STATE_RELEASED;
    }
    data->btn_id = 0;
}

/**
 * Timer callback: Manages chronological calculations and visual effects
 */
void timer_refresh_cb(lv_timer_t * timer) {
    if (is_running) {
        uint32_t now = millis();
        elapsed_time += (now - last_millis);
        last_millis = now;

        /* Calculate clock components */
        uint32_t sec = (elapsed_time / 1000) % 60;
        uint32_t min = (elapsed_time / 60000) % 60;
        uint32_t ms = (elapsed_time % 1000) / 100;
        lv_label_set_text_fmt(label_time, "%02d:%02d.%d", min, sec, ms);

        /* Procedural arc animation */
        static uint16_t angle = 0;
        angle = (angle + 4) % 360;
        lv_arc_set_rotation(arc_deco, angle);
    }
}

/**
 * Interaction event manager: Handles toggle and reset logic
 */
void btn_event_cb(lv_event_t * e) {
    lv_event_code_t code = lv_event_get_code(e);

    if(code == LV_EVENT_SHORT_CLICKED) {
        is_running = !is_running;
        if(is_running) {
            last_millis = millis();
            lv_label_set_text(label_status, "RUNNING");
            lv_obj_set_style_text_color(label_status, lv_palette_main(LV_PALETTE_GREEN), 0);
```

```
91              lv_obj_set_style_arc_color(arc_deco, lv_palette_main(LV_PALETTE_GREEN),
92  LV_PART_INDICATOR);
93          } else {
94              lv_label_set_text(label_status, "PAUSED");
95              lv_obj_set_style_text_color(label_status, lv_palette_main(LV_PALETTE_ORANGE), 0);
96              lv_obj_set_style_arc_color(arc_deco, lv_palette_main(LV_PALETTE_ORANGE),
97  LV_PART_INDICATOR);
98          }
99      }
100     else if(code == LV_EVENT_LONG_PRESSED) {
101         /* System Reset Sequence */
102         is_running = false;
103         elapsed_time = 0;
104         lv_label_set_text(label_time, "00:00.0");
105         lv_label_set_text(label_status, "READY");
106         lv_obj_set_style_text_color(label_status, lv_palette_main(LV_PALETTE_CYAN), 0);
107         lv_obj_set_style_arc_color(arc_deco, lv_palette_main(LV_PALETTE_CYAN),
108 LV_PART_INDICATOR);
109         Serial.println("System Reset Successful");
110     }
111 }
112
113 void setup() {
114     Serial.begin(115200);
115
116     /* Hardware Layer Init */
117     tft.begin();
118     tft.fillScreen(TFT_BLACK);
119     touchSetCycles(0xf000, 0x1000);
120     touch_base_val = touchRead(TOUCH_PIN);
121
122     /* Framework Layer Init */
123     lv_init();
124
125     /* Display Registration */
126     lv_display_t * disp = lv_display_create(SCREEN_WIDTH, SCREEN_HEIGHT);
127     lv_display_set_flush_cb(disp, my_disp_flush);
128     lv_display_set_buffers(disp, draw_buf_ptr, NULL, (SCREEN_WIDTH * SCREEN_HEIGHT / 10) * 2,
129 LV_DISPLAY_RENDER_MODE_PARTIAL);
130
131     /* Input Device Configuration */
132     lv_indev_t * indev = lv_indev_create();
133     lv_indev_set_type(indev, LV_INDEV_TYPE_BUTTON);
134     lv_indev_set_read_cb(indev, my_touchpad_read);
```

```
135         lv_indev_set_long_press_time(indev, PRESS_TIME);
136
137         /* Map hardware sensor to logical button area */
138         static const lv_point_t btn_points[] = { {120, 120} };
139         lv_indev_set_button_points(indev, btn_points);
140
141         /* UI Theme & Orchestration */
142         lv_obj_t * scr = lv_screen_active();
143         lv_obj_set_style_bg_color(scr, lv_color_hex(0x000000), 0);
144
145         /* Stylized Decorative Arc */
146         arc_deco = lv_arc_create(scr);
147         lv_obj_set_size(arc_deco, 220, 220);
148         lv_arc_set_range(arc_deco, 0, 100);
149         lv_arc_set_value(arc_deco, 15);
150         lv_arc_set_bg_angles(arc_deco, 0, 360);
151         lv_obj_set_style_arc_width(arc_deco, 6, LV_PART_MAIN);
152         lv_obj_set_style_arc_width(arc_deco, 6, LV_PART_INDICATOR);
153         lv_obj_set_style_arc_color(arc_deco, lv_color_hex(0x1A1A1A), LV_PART_MAIN);
154         lv_obj_set_style_arc_color(arc_deco, lv_palette_main(LV_PALETTE_CYAN),
155 LV_PART_INDICATOR);
156         lv_obj_remove_style(arc_deco, NULL, LV_PART_KNOB);
157         lv_obj_center(arc_deco);
158
159         /* Button Interaction */
160         lv_obj_t * btn_full = lv_button_create(scr);
161         lv_obj_set_size(btn_full, SCREEN_WIDTH, SCREEN_HEIGHT);
162         lv_obj_set_style_bg_opa(btn_full, 0, 0);
163         lv_obj_set_style_border_opa(btn_full, 0, 0);
164         lv_obj_set_style_shadow_opa(btn_full, 0, 0);
165         lv_obj_add_event_cb(btn_full, btn_event_cb, LV_EVENT_ALL, NULL);
166
167         /* Chrono Readout Setup */
168         label_time = lv_label_create(scr);
169         lv_label_set_text(label_time, "00:00.0");
170         lv_obj_set_style_text_font(label_time, &lv_font_montserrat_40, 0);
171         lv_obj_set_style_text_color(label_time, lv_color_white(), 0);
172         lv_obj_align(label_time, LV_ALIGN_TOP_LEFT, 50, 95);
173         lv_obj_set_style_text_align(label_time, LV_TEXT_ALIGN_LEFT, 0);
174
175         /* Secondary Information Readout */
176         label_status = lv_label_create(scr);
177         lv_label_set_text(label_status, "READY");
178         lv_obj_set_style_text_font(label_status, &lv_font_montserrat_14, 0);
```

```
179       lv_obj_set_style_text_color(label_status, lv_palette_main(LV_PALETTE_CYAN), 0);
180       lv_obj_align(label_status, LV_ALIGN_CENTER, 0, 45);
181
182       /* Design separation line */
183       lv_obj_t * line = lv_obj_create(scr);
184       lv_obj_set_size(line, 80, 2);
185       lv_obj_set_style_bg_color(line, lv_color_hex(0x333333), 0);
186       lv_obj_set_style_border_opa(line, 0, 0);
187       lv_obj_align(line, LV_ALIGN_CENTER, 0, 20);
188
189       /* Primary Application Task Initiation */
190       lv_timer_create(timer_refresh_cb, 50, NULL);
191
192       /* TFT Power Engagement */
193       pinMode(TFT_VCC, OUTPUT);
194       digitalWrite(TFT_VCC, LOW);
195  }
196
197  void loop() {
198       /* Handle pending framework tasks and signal management */
199       lv_timer_handler();
200
201       /* System synchronization - synchronize library clock to hardware millis */
202       static uint32_t last_tick = 0;
203       uint32_t current_ms = millis();
204       lv_tick_inc(current_ms - last_tick);
205       last_tick = current_ms;
206
207       delay(5);
208  }
```

Code Explanation

Include the necessary header files

```
1  #include <lvgl.h>
2  #include <TFT_eSPI.h>
```

Preset capacitive touch related parameters

```
5  const int TOUCH_PIN = 32;          // Capacitive touch GPIO
6  const int TFT_VCC = 21;            // Display power control
7  const int TOUCH_THRESHOLD = 80;    // Sensitivity calibration
8  const int PRESS_TIME = 3000;       // Reset threshold (ms)
```

Define screen resolution

```
12 #define SCREEN_WIDTH  240
13 #define SCREEN_HEIGHT 240
```

Draw the outer ring of the timer

```
146 /* Stylized Decorative Arc */
```

```
147 arc_deco = lv_arc_create(scr);
148 lv_obj_set_size(arc_deco, 220, 220);
149 lv_arc_set_range(arc_deco, 0, 100);
150 lv_arc_set_value(arc_deco, 15);
151 lv_arc_set_bg_angles(arc_deco, 0, 360);
152 lv_obj_set_style_arc_width(arc_deco, 6, LV_PART_MAIN);
153 lv_obj_set_style_arc_width(arc_deco, 6, LV_PART_INDICATOR);
154 lv_obj_set_style_arc_color(arc_deco, lv_color_hex(0x1A1A1A), LV_PART_MAIN);
155 lv_obj_set_style_arc_color(arc_deco, lv_palette_main(LV_PALETTE_CYAN), LV_PART_INDICATOR);
156 lv_obj_remove_style(arc_deco, NULL, LV_PART_KNOB);
157 lv_obj_center(arc_deco);
158
159 /* Button Interaction */
160 lv_obj_t * btn_full = lv_button_create(scr);
161 lv_obj_set_size(btn_full, SCREEN_WIDTH, SCREEN_HEIGHT);
162 lv_obj_set_style_bg_opa(btn_full, 0, 0);
163 lv_obj_set_style_border_opa(btn_full, 0, 0);
164 lv_obj_set_style_shadow_opa(btn_full, 0, 0);
165 lv_obj_add_event_cb(btn_full, btn_event_cb, LV_EVENT_ALL, NULL);
166
167 /* Chrono Readout Setup */
168 label_time = lv_label_create(scr);
169 lv_label_set_text(label_time, "00:00.0");
170 lv_obj_set_style_text_font(label_time, &lv_font_montserrat_40, 0);
171 lv_obj_set_style_text_color(label_time, lv_color_white(), 0);
172 lv_obj_align(label_time, LV_ALIGN_TOP_LEFT, 50, 95);
173 lv_obj_set_style_text_align(label_time, LV_TEXT_ALIGN_LEFT, 0);
174
175 /* Secondary Information Readout */
176 label_status = lv_label_create(scr);
177 lv_label_set_text(label_status, "READY");
178 lv_obj_set_style_text_font(label_status, &lv_font_montserrat_14, 0);
179 lv_obj_set_style_text_color(label_status, lv_palette_main(LV_PALETTE_CYAN), 0);
180 lv_obj_align(label_status, LV_ALIGN_CENTER, 0, 45);
181
182 /* Design separation line */
183 lv_obj_t * line = lv_obj_create(scr);
184 lv_obj_set_size(line, 80, 2);
185 lv_obj_set_style_bg_color(line, lv_color_hex(0x333333), 0);
186 lv_obj_set_style_border_opa(line, 0, 0);
187 lv_obj_align(line, LV_ALIGN_CENTER, 0, 20);
```

Create a timer task

```
190 lv_timer_create(timer_refresh_cb, 50, NULL);
```

Enable screen power supply pin

```
193  /* TFT Power Engagement */
194  pinMode(TFT_VCC, OUTPUT);
195  digitalWrite(TFT_VCC, LOW);
```

Loop through UI tasks

```
198  /* Handle pending framework tasks and signal management */
199  lv_timer_handler();
200
201  /* System synchronization - synchronize library clock to hardware millis */
202  static uint32_t last_tick = 0;
203  uint32_t current_ms = millis();
204  lv_tick_inc(current_ms - last_tick);
205  last_tick = current_ms;
206
207  delay(5);
```

Click "**Upload**" to upload the code to Freenove ESP32 Mini TV

Touch the capacitive touch area of the Freenove ESP32 Mini TV with your finger:

Short press: Start/Pause timer;

Long press: Reset timer.

Chapter 11 LVGL Game

Having explored key concepts like EEPROM, capacitive touch, TFT display, and LVGL, we will now synthesize this knowledge to create a classic and entertaining mini-game: Flappy Bird.

Project 11.1 Lvgl Game

Component List

Freenove ESP32 Mini TV 	USB cable x1 

Circuit

Connect Freenove ESP32 Mini TV to the computer with USB cable.

Sketch

Open“**Sketch_11.1_LVGL_Game**” folder under **“Freenove_ESP32_Mini_TV\Sketch”** and double-click “**Sketch_11.1_LVGL_Game.ino**”.

Sketch_11.1_LVGL_Game

The following is the program code:

```
#include <lvgl.h>
#include <TFT_eSPI.h>
#include "image.h"
#include <EEPROM.h>

/* --- Hardware Configuration --- */
const int TOUCH_PIN = 32;
const int TFT_VCC = 21;
const int TOUCH_THRESHOLD = 80;
#define SCREEN_WIDTH  240
#define SCREEN_HEIGHT 240
#define EEPROM_SIZE 4

/* --- Graphics Instances --- */
TFT_eSPI tft = TFT_eSPI();
static uint32_t buf_aligned[ (SCREEN_WIDTH * SCREEN_HEIGHT / 10 * 2) / 4 + 4 ];

/* --- LVGL Image Descriptor --- */
const lv_image_dsc_t img_bg = {
   .header = {
     .magic = LV_IMAGE_HEADER_MAGIC,
     .cf = LV_COLOR_FORMAT_RGB565,
     .flags = 0,
     .w = 240,
     .h = 240,
     .stride = 240 * 2,
     .reserved_2 = 0,
   },
   .data_size = sizeof(img_bg_data),
   .data = img_bg_data,
   .reserved = NULL,
};

/* --- Game Physics & Constants --- */
const int GRAVITY = 1;
const int JUMP_FORCE = -9;
const int PIPE_SPEED = 4;
```

```
38  const int PIPE_GAP = 100;
39  const int PIPE_WIDTH = 40;
40
41  /* --- Game State Variables --- */
42  int bird_y = 120;
43  int bird_vel = 0;
44  int pipe_x = 240;
45  int pipe_gap_y = 100;
46  int score = 0;
47  bool game_running = false;
48  int touch_base_val = 0;
49  int best_score = 0;
50
51  /* --- UI Object References --- */
52  lv_obj_t * bird = NULL;
53  lv_obj_t * pipe_top = NULL;
54  lv_obj_t * pipe_bottom = NULL;
55  lv_obj_t * label_score = NULL;
56  lv_obj_t * label_msg = NULL;
57
58  /**
59    * @brief Load best score from EEPROM
60    */
61  void load_best_score() {
62      EEPROM.begin(EEPROM_SIZE);
63      EEPROM.get(0, best_score);
64      if (best_score < 0 || best_score > 9999) {
65          best_score = 0;
66      }
67  }
68
69  /**
70    * @brief Handle game termination logic
71    */
72  void game_over_action() {
73      game_running = false;
74
75      bool is_new_record = false;
76      if (score > best_score) {
77          best_score = score;
78          EEPROM.put(0, best_score);
79          EEPROM.commit();
80          is_new_record = true;
81      }
```

```
82
83          lv_label_set_text_fmt(label_msg,
84              "%s\nSCORE: %d\nBEST: %d",
85              is_new_record ? "#008000 NEW RECORD!#" : "#FF0000 GAME OVER#",
86              score,
87              best_score);
88
89          lv_label_set_recolor(label_msg, true);
90          lv_obj_remove_flag(label_msg, LV_OBJ_FLAG_HIDDEN);
91      }
92
93      /**
94        * @brief LVGL Display flushing callback
95        */
96      void my_disp_flush(lv_display_t * disp, const lv_area_t * area, uint8_t * px_map) {
97          uint32_t w = (area->x2 - area->x1 + 1);
98          uint32_t h = (area->y2 - area->y1 + 1);
99          tft.startWrite();
100         tft.setAddrWindow(area->x1, area->y1, w, h);
101         tft.pushColors((uint16_t *)px_map, w * h, true);
102         tft.endWrite();
103         lv_display_flush_ready(disp);
104     }
105
106     /**
107       * @brief Touch input reading callback
108       */
109     void my_touchpad_read(lv_indev_t * indev, lv_indev_data_t * data) {
110         int val = touchRead(TOUCH_PIN);
111         if (touch_base_val - val > TOUCH_THRESHOLD) {
112             data->state = LV_INDEV_STATE_PRESSED;
113         } else {
114             data->state = LV_INDEV_STATE_RELEASED;
115         }
116     }
117
118     /**
119       * @brief Main game loop timer callback
120       */
121     void game_loop_cb(lv_timer_t * timer) {
122         if (!game_running || bird == NULL) return;
123
124         // Apply gravity and update bird position
125         bird_vel += GRAVITY;
```

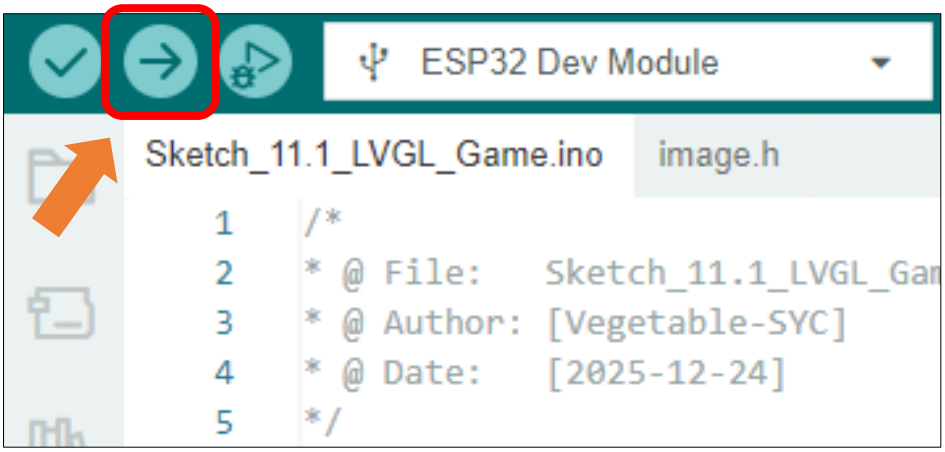
```
126         bird_y += bird_vel;
127         lv_obj_set_y(bird, bird_y);
128 
129         // Update pipe position and handle scrolling
130         pipe_x -= PIPE_SPEED;
131         if (pipe_x < -PIPE_WIDTH) {
132             pipe_x = SCREEN_WIDTH;
133             pipe_gap_y = 30 + rand() % 80;
134             score++;
135             lv_label_set_text_fmt(label_score, "%d", score);
136         }
137 
138         // Refresh pipe UI elements
139         if(pipe_top && pipe_bottom) {
140             lv_obj_set_x(pipe_top, pipe_x);
141             lv_obj_set_x(pipe_bottom, pipe_x);
142             lv_obj_set_height(pipe_top, pipe_gap_y);
143             lv_obj_set_y(pipe_bottom, pipe_gap_y + PIPE_GAP);
144             lv_obj_set_height(pipe_bottom, SCREEN_HEIGHT - (pipe_gap_y + PIPE_GAP));
145         }
146 
147         // Boundary collision detection
148         if (bird_y < 0 || bird_y > (SCREEN_HEIGHT - 20)) {
149             game_running = false;
150             lv_obj_remove_flag(label_msg, LV_OBJ_FLAG_HIDDEN);
151             lv_label_set_text(label_msg, "CRASHED!\nTap to Restart");
152             game_over_action();
153         }
154 
155         // Pipe collision detection logic
156         if (pipe_x < 60 && (pipe_x + PIPE_WIDTH) > 40) {
157             if (bird_y < pipe_gap_y || (bird_y + 18) > (pipe_gap_y + PIPE_GAP)) {
158                 game_running = false;
159                 lv_obj_remove_flag(label_msg, LV_OBJ_FLAG_HIDDEN);
160                 lv_label_set_text(label_msg, "HIT PIPE!\nTap to Restart");
161                 game_over_action();
162             }
163         }
164 }
165 
166 /**
167   * @brief UI event handler for touch interactions
168   */
169 void event_handler(lv_event_t * e) {
```

```
170         if (lv_event_get_code(e) == LV_EVENT_PRESSED) {
171             if (!game_running) {
172                 // Reset game state on restart
173                 bird_y = 100; bird_vel = 0; pipe_x = 240; score = 0;
174                 lv_label_set_text(label_score, "0");
175                 lv_obj_add_flag(label_msg, LV_OBJ_FLAG_HIDDEN);
176                 game_running = true;
177             } else {
178                 bird_vel = -7.5;
179             }
180         }
181 }
182
183 void setup() {
184     Serial.begin(115200);
185     delay(1000);
186
187     load_best_score();
188     Serial.printf("Best score loaded: %d\n", best_score);
189
190     // Hardware Pin Setup
191     pinMode(TFT_VCC, OUTPUT);
192     digitalWrite(TFT_VCC, LOW);
193     delay(100);
194
195     // Display Driver Setup
196     tft.begin();
197     tft.fillScreen(TFT_BLACK);
198
199     // Touch Calibration
200     touchSetCycles(0xf000, 0x1000);
201     touch_base_val = touchRead(TOUCH_PIN);
202
203     // LVGL Framework Initialization
204     lv_init();
205     Serial.println("LVGL Init OK");
206
207     // Display Interface Configuration
208     lv_display_t * disp = lv_display_create(SCREEN_WIDTH, SCREEN_HEIGHT);
209     if(!disp) return;
210     lv_display_set_flush_cb(disp, my_disp_flush);
211     lv_display_set_buffers(disp, (uint8_t *)buf_aligned, NULL, sizeof(buf_aligned),
212 LV_DISPLAY_RENDER_MODE_PARTIAL);
213
```

```
214     // Input Device Registration
215     lv_indev_t * indev = lv_indev_create();
216     lv_indev_set_type(indev, LV_INDEV_TYPE_BUTTON);
217     lv_indev_set_read_cb(indev, my_touchpad_read);
218     static const lv_point_t btn_pts[] = {{120, 120}};
219     lv_indev_set_button_points(indev, btn_pts);
220
221     // UI Layout - Background
222     lv_obj_set_style_bg_color(lv_screen_active(), lv_color_hex(0x050510), 0);
223     lv_obj_t * bg_img = lv_image_create(lv_screen_active());
224     lv_image_set_src(bg_img, &img_bg);
225     lv_obj_center(bg_img);
226
227     // Button
228     lv_obj_t * bg_btn = lv_button_create(lv_screen_active());
229     lv_obj_set_size(bg_btn, SCREEN_WIDTH, SCREEN_HEIGHT);
230     lv_obj_set_style_bg_opa(bg_btn, 0, 0);
231     lv_obj_set_style_border_opa(bg_btn, 0, 0);
232     lv_obj_add_event_cb(bg_btn, event_handler, LV_EVENT_ALL, NULL);
233
234     // UI Layout - Pipes
235     pipe_top = lv_obj_create(lv_screen_active());
236     lv_obj_set_size(pipe_top, PIPE_WIDTH, 100);
237     lv_obj_set_style_bg_color(pipe_top, lv_palette_main(LV_PALETTE_GREEN), 0);
238     lv_obj_set_style_border_color(pipe_top, lv_palette_main(LV_PALETTE_GREEN), 0);
239     lv_obj_set_style_border_width(pipe_top, 2, 0);
240
241     pipe_bottom = lv_obj_create(lv_screen_active());
242     lv_obj_set_size(pipe_bottom, PIPE_WIDTH, 100);
243     lv_obj_set_style_bg_color(pipe_bottom, lv_palette_main(LV_PALETTE_GREEN), 0);
244     lv_obj_set_style_border_color(pipe_bottom, lv_palette_main(LV_PALETTE_GREEN), 0);
245     lv_obj_set_style_border_width(pipe_bottom, 2, 0);
246
247     // UI Layout - Character (Bird)
248     bird = lv_obj_create(lv_screen_active());
249     lv_obj_set_size(bird, 18, 18);
250     lv_obj_set_style_radius(bird, LV_RADIUS_CIRCLE, 0);
251     lv_obj_set_style_bg_color(bird, lv_palette_main(LV_PALETTE_PINK), 0);
252     lv_obj_set_x(bird, 40);
253     lv_obj_set_scrollbar_mode(bird, LV_SCROLLBAR_MODE_OFF);
254
255     // UI Layout - Score
256     label_score = lv_label_create(lv_screen_active());
257     lv_label_set_text(label_score, "0");
```

```
258         lv_obj_set_style_text_font(label_score, &lv_font_montserrat_40, 0);
259         lv_obj_set_style_text_color(label_score, lv_color_white(), 0);
260         lv_obj_set_style_text_opa(label_score, 255, 0);
261         lv_obj_align(label_score, LV_ALIGN_TOP_MID, 0, 20);
262
263         // UI Layout - Information
264         label_msg = lv_label_create(lv_screen_active());
265         lv_label_set_recolor(label_msg, true);
266         lv_obj_set_style_text_align(label_msg, LV_TEXT_ALIGN_CENTER, 0);
267         lv_label_set_text_fmt(label_msg,
268             "#FF3399 CYBER BIRD#\n\n#FF0000 BEST SCORE: %d#\n#000000 Touch to Fly#",
270             best_score);
271         lv_obj_set_style_text_color(label_msg, lv_color_white(), 0);
272         lv_obj_center(label_msg);
273
274         // Start timer
275         lv_timer_create(game_loop_cb, 30, NULL);
276         Serial.println("Setup Complete");
277 }
278
279 void loop() {
280         // Refresh LVGL tasks
281         lv_timer_handler();
282         static uint32_t last_tick = 0;
283         uint32_t current_ms = millis();
284         lv_tick_inc(current_ms - last_tick);
285         last_tick = current_ms;
286         delay(5);
287 }
```

Click "**Upload**" to upload the code to Freenove ESP32 Mini TV



Game Rules

To start the game, tap the capacitive touch area after powering on the device.

Control the ball by briefly tapping the touch zone to make it rise; release to let it fall. Navigate through obstacle pillars to score points—each successful pass earns one point. The game ends if the ball hits any obstacle or the screen edges.

A new high score is automatically saved to EEPROM and persists after restart, preserving your best achievement.

Chapter 12 LVGL Weather Clock

Project 12.1 Weather Clock

Component List

Freenove ESP32 Mini TV 	USB cable x1 

Circuit

Connect Freenove ESP32 Mini TV to the computer with USB cable.

Sketch

Open“**Sketch_12.1_LVGL_Weather_Clock**” folder under **“Freenove_ESP32_Mini_TV\Sketch”** and double-click “**Sketch_12.1_LVGL_Weather_Clock.ino**”.

Sketch_12.1_LVGL_Weather_Clock

The following is the program code:

```
/*
* @ File:   Sketch_12.1_LVGL_Weather_Clock.ino
* @ Author: [Vegetable-SYC]
* @ Date:   [2026-08-07]
*/

#include "debug_log.h"
#include "lvgl_driver.h"
#include "wifi_setup.h"
#include "config_store.h"
#include "ntp_time.h"
#include "weather.h"
#include "ui_clock.h"
#include "touch_button.h"

#ifndef WEATHER_REFRESH_MS
#define WEATHER_REFRESH_MS (10UL * 60UL * 1000UL)
#endif

static uint32_t last_time_ms = 0;
static uint32_t last_weather_ms = 0;
static bool ui_ready = false;

static void boot_status_live(const char *text) {
    ui_clock_show_boot(text);
}

/** Show a boot page **/
static void boot_page(BootPage page, const char *detail, uint32_t hold_ms = 450) {
    ui_clock_show_boot_page(page, detail);
    uint32_t start = millis();
    while (millis() - start < hold_ms) {
        lvgl_driver_handler();
        delay(5);
    }
}

```

```
38  /** Fetch weather **/
39  static bool refresh_weather(void) {
40      if (!ui_ready) {
41          boot_page(BOOT_PAGE_WEATHER, "Locating city\nby public IP...", 300);
42      }
43
44      bool ok = weather_update_auto();
45
46      if (ok && ui_ready) {
47          ui_clock_update_weather(weather_get());
48      } else if (!ok && !ui_ready) {
49          boot_page(BOOT_PAGE_WEATHER, "Weather failed\nWill retry later", 600);
50      }
51
52      last_weather_ms = millis();
53      return ok;
54  }
55
56  static void on_theme_long_press(void) {
57      uint8_t next = ui_clock_toggle_theme();
58      config_store_save_theme(next);
59      DBG_PRINTF("[UI] Theme -> %s\n", next == THEME_LIGHT ? "light" : "dark");
60  }
61
62  static void on_face_long_press(void) {
63      uint8_t next = ui_clock_toggle_face();
64      config_store_save_ui_face(next);
65      const char *name = "weather";
66      if (next == UI_FACE_ANALOG) {
67          name = "analog";
68      } else if (next == UI_FACE_XL) {
69          name = "xl";
70      }
71      DBG_PRINTF("[UI] Face -> %s\n", name);
72  }
73
74  void setup() {
75      DBG_BEGIN(115200);
76      delay(200);
77
78      lvgl_driver_init();
79      touch_button_begin();
80      config_store_begin();
81      ui_clock_set_theme(config_store_get().theme);
```

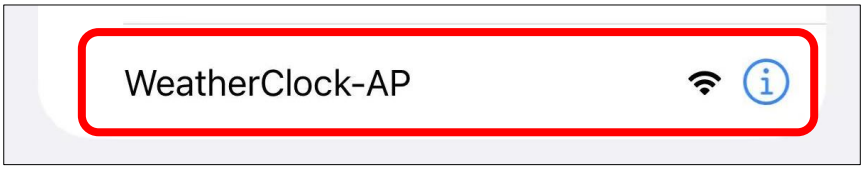
```
82
83      boot_page(BOOT_PAGE_START, "Weather Clock\nStarting system...", 500);
84      DBG_PRINTLN("System Ready");
85
86      boot_page(BOOT_PAGE_WIFI, "Preparing WiFi...", 350);
87      if (!wifi_setup_begin(boot_status_live)) {
88          boot_page(BOOT_PAGE_WIFI, "WiFi setup failed\nReset to retry", 0);
89          return;
90      }
91      boot_page(BOOT_PAGE_WIFI, "WiFi connected", 400);
92
93      boot_page(BOOT_PAGE_NTP, "Syncing time\nvia NTP...", 350);
94      if (!ntp_time_begin(config_store_get().timezone)) {
95          boot_page(BOOT_PAGE_NTP, "NTP sync failed\nClock may be wrong", 800);
96      } else {
97          boot_page(BOOT_PAGE_NTP, "Time synced OK", 400);
98      }
99
100     boot_page(BOOT_PAGE_WEATHER, "Fetching weather...", 300);
101     refresh_weather();
102     if (weather_get().valid) {
103         const WeatherInfo &w = weather_get();
104         char msg[96];
105         snprintf(msg, sizeof(msg), "Got weather\n%s  %.0fC",
106                  w.resolved_name[0] ? w.resolved_name : "OK",
107                  w.temperature_c);
108         boot_page(BOOT_PAGE_WEATHER, msg, 500);
109     }
110
111     boot_page(BOOT_PAGE_READY, "Building UI...", 350);
112     ui_clock_create();
113     ui_clock_set_face(config_store_get().ui_face);
114     ui_ready = true;
115     ui_clock_update_time();
116     if (weather_get().valid) {
117         ui_clock_update_weather(weather_get());
118     }
119 }
120
121 void loop() {
122     lvgl_driver_handler();
123
124     TouchEvent ev = touch_button_poll();
125     if (ev == TOUCH_EVENT_THEME) {
```

```
126             on_theme_long_press();
127         } else if (ev == TOUCH_EVENT_FACE) {
128             on_face_long_press();
129         }
130
131         uint32_t now = millis();
132
133         if (ui_ready && (now - last_time_ms >= 1000)) {
134             last_time_ms = now;
135             ui_clock_update_time();
136             ntp_time_print();
137         }
138
139         if (ui_ready && (now - last_weather_ms >= WEATHER_REFRESH_MS)) {
140             refresh_weather();
141         }
142
143         delay(5);
144     }
```

Click "**Upload**" to upload the code to the Freenove ESP32 Mini TV

After the code is uploaded successfully, the Freenove ESP32 Mini TV will automatically enter WiFi configuration mode, as shown below:

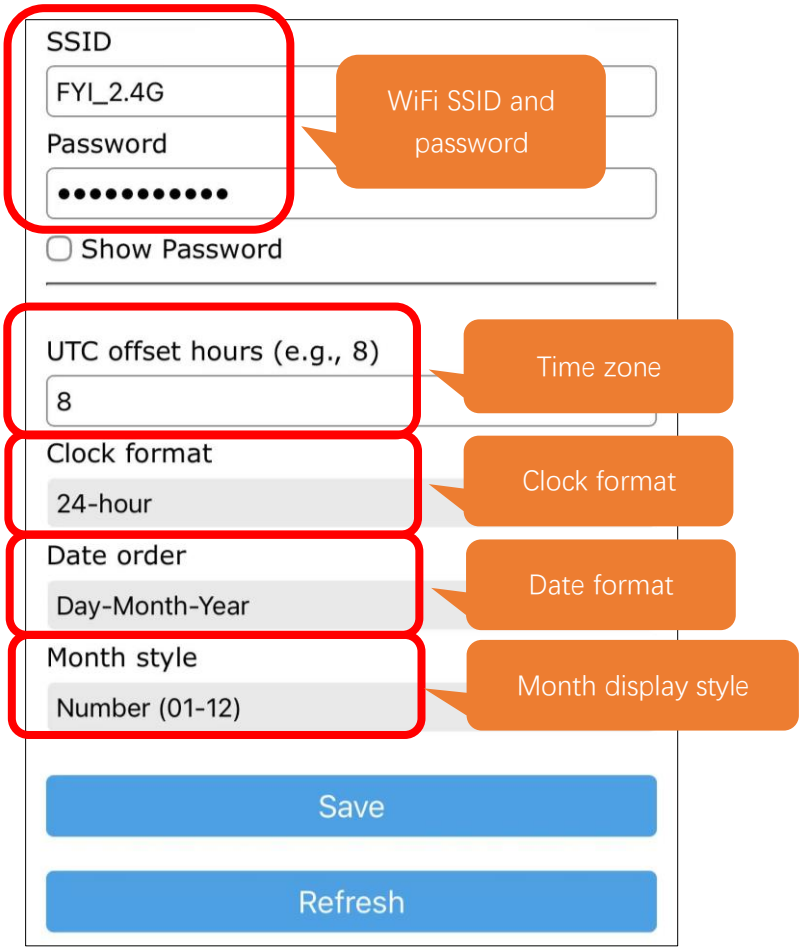
Connect your phone or computer to the WiFi hotspot "WeatherClock-AP". No password is required for this hotspot.



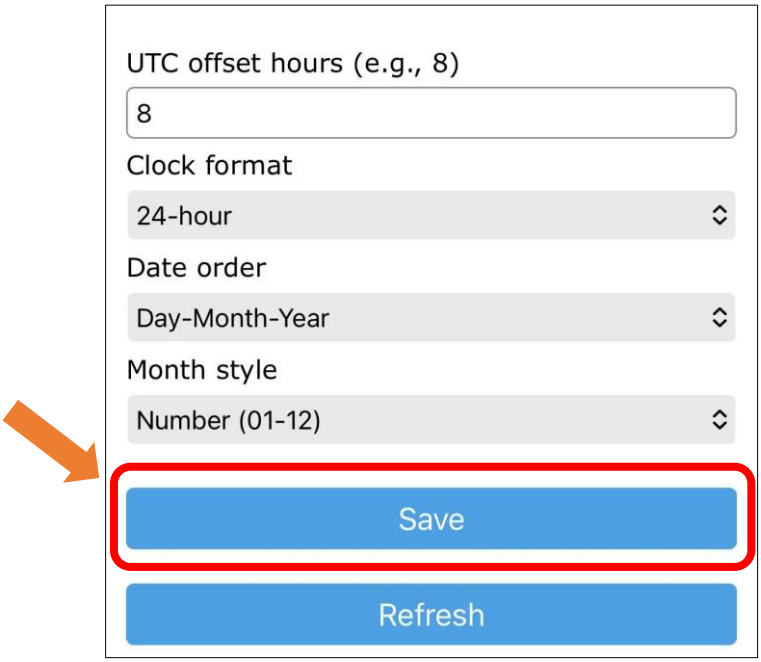
After connecting, the following configuration page will automatically appear. If it does not appear, open a browser and visit 192.168.4.1.

Click **Configure WiFi**.

Configure the following parameters based on your actual settings:



Click Save to save the settings.



After the configuration is completed, the device will automatically connect to the Internet and retrieve weather information. **Please wait patiently during this process.**

The Freenove Weather Clock features three built-in clock faces and two themes (black and white), as shown below:

Tap the touch button to switch between clock faces. Press and hold the touch button for 3 seconds to switch between themes.